数据结构
知识点手册

配《数据结构》互动课件 · 按教材小节重编

芦老师聊AI　编

校本讲义　第 一 版　·　2026-10-14　·　共 38 节，已写 38 节

说明　本手册是互动课件的配套教材：正文按教材小节组织，每节末标注对应的动画（站内序号与直达链接）。所有插图取自课件 `render()` 的画布输出，与课堂画面同源；正文、例题与习题为手写，例题的演算过程均经逐轮核算。

数据结构（严蔚敏、吴伟民编著，C 语言版）教材章节编号与 408 计算机学科专业基础综合考试大纲为编排依据。

许可：CC BY-NC-SA 4.0 —— 免费用于课堂教学与个人学习，署名、转发需注明出处，不得商用。

前　言

这门课难的地方往往不在“记不住”，而在“看不见”：算法每一步到底改了什么，教材给的是静态的图，板书给的是某一瞬间的表。配套课件把每个算法真的执行一遍，逐帧画出来；本手册则负责另一半——把每个知识点的定义、前提、方法、复杂度与易错处用书面语固定下来，供课前预习、课后复习与考前速查。

两者分工明确：**手册是正文，动画是插图**。手册可以独立读完，不依赖任何设备；每节末的“配套动画”只是给出一个入口，想看清某一步为什么这样改时再打开。

全书按教材的章-节顺序编排，共 38 节。各节体例统一为：**概念与前提 → 方法描述 → 正确性与复杂度 → 手工演算的标准写法 → 典型例题 → 易错点与考点提示 → 习题**。习题答案集中在附录 A。

目 录

第 1 章 绪论与算法分析 5

1.1 数据结构的基本概念 5

1.2 算法及其评价 8

第 2 章 线性表 12

2.1 线性表的顺序表示与实现 12

2.2 线性表的链式表示与实现 14

2.3 线性表的应用：有序表归并与多项式相加 17

2.4 链表典型问题 19

第 3 章 栈和队列 22

3.1 栈的定义、存储与实现 22

3.2 队列的定义、假溢出与循环队列 25

3.3 栈与递归 27

3.4 栈和队列的应用 29

第 4 章 串、数组和广义表 33

4.1 串与模式匹配 33

4.2 数组的压缩存储 35

4.3 广义表 37

第 5 章 树和二叉树 40

5.1 树的定义与存储结构 40

5.2 二叉树的定义、性质与顺序存储 42

5.3 二叉树的遍历与线索化 44

5.4 树、森林与二叉树的转换 47

5.5 哈夫曼树及其应用 49

5.6 堆与优先队列 51

5.7 并查集 53

第 6 章 图 56

6.1 图的定义与术语 56

6.2 图的存储结构 58

6.3 图的遍历 61

6.4 最小生成树 64

6.5 最短路径 68

6.6 拓扑排序与关键路径 72

第 7 章 查找 77

7.1 查找的基本概念与线性表上的查找 77

7.2 树形查找：二叉排序树与平衡二叉树 80

7.3 多路查找树：B 树与 B+ 树 83

7.4 红黑树 85
7.5 散列查找 87

第 8 章 排序 91
8.1 排序的基本概念 91
8.2 插入排序 93
8.3 交换排序 96
8.4 选择排序 99
8.5 归并排序与基数排序 102
8.6 外部排序 105
8.7 排序方法的综合比较 108

第 1 章　绪论与算法分析

1.1　数据结构的基本概念

本节配套动画：第 1 项《数据结构基本概念：术语层级、四种逻辑结构、两种存储结构与 ADT》 · 408 大纲 一(一) 数据结构基本概念（数据/数据元素/数据项、四种逻辑结构、存储结构（顺序/链式为主，另有索引/散列）、抽象数据类型）

> **定义**
> **数据**是信息的载体；**数据元素**是数据的基本单位，在程序中通常作为一个整体处理；**数据项**是构成数据元素的、有独立含义的最小单位；**数据对象**是性质相同的数据元素的集合，是数据的一个子集。

四者的包含关系是：数据 ⊇ 数据对象 ⊇ 数据元素 ⊇ 数据项。一张学生成绩表是数据对象，表里**一行**是一个数据元素，行内的学号、姓名、各科成绩各是一个数据项。判据只有一句话：**能不能再拆**——能拆成“姓名 + 学号”的就不是最小单位。

1.1.1　逻辑结构：分类只看关系

> **定义**
> 逻辑结构是数据元素之间的逻辑关系，与它们在计算机里如何存放无关。按“每个元素最多能和几个元素直接相关”分为四类。

结构	前驱上限	后继上限	典型例子
集合	0	0（只有归属关系）	并查集里“同属一堆”的元素
线性	1	1	数组、链表、栈、队列
树形	1	多个	二叉树、哈夫曼树、堆
图状 / 网状	多个	多个	网络拓扑、AOV / AOE 网

四类中**只有线性结构是线性的**，集合、树形、图状都属非线性；考试问“哪些是非线性结构”时，标准答案通常是**树形与图状**两项。

> **易错**
> 边数不能用来分类。n 个顶点的线性结构和 n 个顶点的树都是 **n − 1** 条边，边数相同而关系完全不同；分类依据是“几对几”的对应关系。

1.1.2　存储结构：四种，不是两种

存储结构（物理结构）是逻辑结构在计算机中的映像，大纲列**四种**：

方法	相邻关系靠什么表达	说明
顺序存储	**物理位置相邻**	元素挤在连续单元里，可随机存取
链式存储	**指针**	物理上可散落，增删不必搬动
索引存储	附加一张索引表	关键字 → 地址，块内仍可顺序
散列存储	由关键字**算**地址	查找接近 O(1)

教材主讲前两种，但“只有两种”是错的表述。**顺序存储不等于逻辑上是线性结构**：完全二叉树可以顺序存进数组（`a[i]` 的孩子在 `a[2i]`、`a[2i+1]`），物理相邻只是实现手段。

1.1.3 数据结构的三要素与 ADT

数据结构是**集合 + 关系 + 运算**三方面，形式化地定义为二元组 **Data_Structure = (D, S)**，其中 D 是数据对象的有限集，S 是 D 上关系的有限集；加上运算即构成抽象数据类型：

> **ADT**
> **ADT = (D, S, P)**，D 为数据对象，S 为 D 上的关系集合，P 为对 D 的基本操作集合。

写 ADT 声明时只出现数据与操作，**不出现数组、指针、下标、结构体**这些实现词。同一个 ADT（如栈）既可有顺序实现也可有链式实现，上层算法只认 push / pop / 判空，不认底层怎么存——这正是“算法分析与数据无关、与规模有关”的前提。

典型例题

例 1 指出下列各项的层级：① 全班同学的成绩记录；② 某同学的“高等数学”成绩；③ 某位同学的成绩单。

解 ② 是数据项（已拆到字段）；③ 是数据元素（一行记录，仍可拆出姓名与各科成绩）；① 是数据对象（同型数据元素的集合）。

例 2 “数据的逻辑结构在计算机中的存放形式”指的是什么？下列哪种结构与“元素之间的物理相邻”无关？

解 前者是**存储结构**。链式存储与物理相邻无关——它用指针表达关系；顺序存储才依赖物理相邻。

例 3 设栈 ADT 只暴露 InitStack、Push、Pop、GetTop、StackEmpty。学生问“它内部是数组还是链表”，这属于哪一层？

解 属于实现层，不在 ADT 中。两种实现都满足这五个操作，对使用者没有区别。

易错点与考点提示

- “数据元素”与“数据项”混用：判据是能否再拆，不是体积大小。
- 说“存储结构只有顺序与链式两种”：大纲是四种。
- 说“顺序存储就是线性结构”：完全二叉树顺序存进数组仍是树形。
- 在 ADT 声明里写实现词（数组 / 指针 / 下标），这是最常见的扣分写法。

- 逻辑结构决定**算法**，存储结构决定**算法的实现效率**——同一逻辑结构换存储，时间性能可能完全不同（线性表顺序 vs 链式即为例）。

习题

1. 用“前驱 / 后继上限”说明为什么树形与图状都不属于线性结构。
2. n 个结点的树有 n − 1 条边，n 个结点的线性表也有 n − 1 条边。据此能否说“两者逻辑结构相同”？
3. 把“数据结构 = 逻辑结构 + 存储结构 + 运算”这句话与 ADT 三元组对应起来，指出 P 对应哪一部分。

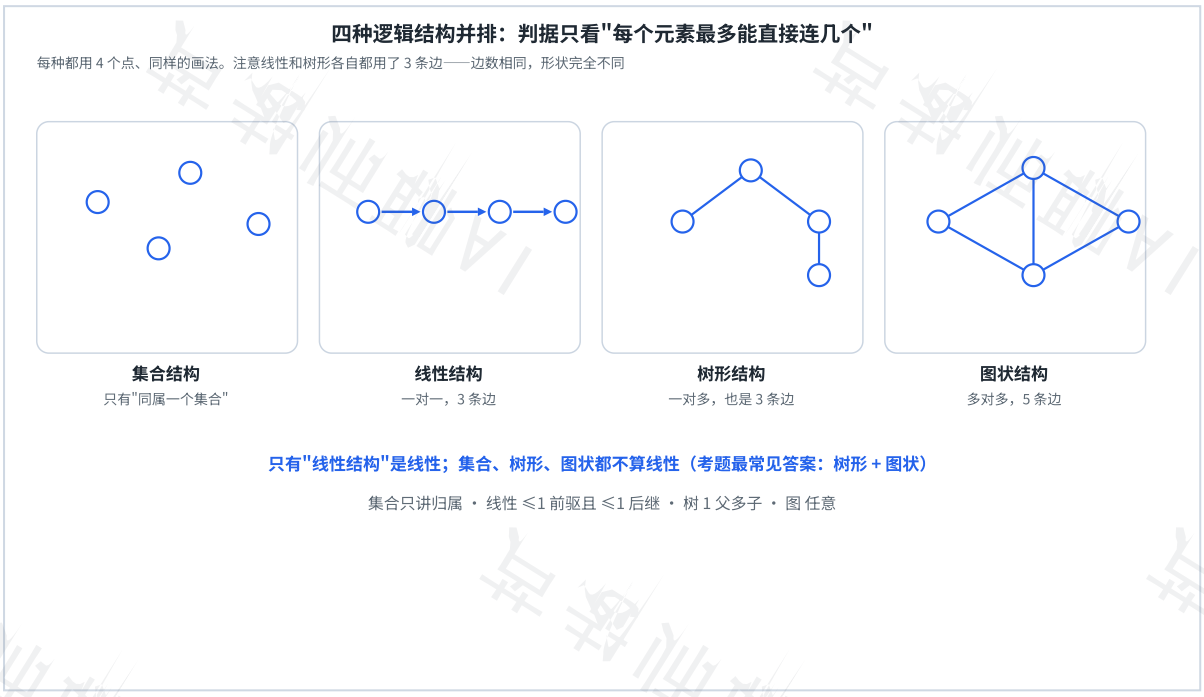


图 1-1　四张缩略图并排，判据统一成一句话：看每个元素最多能直接连几个。

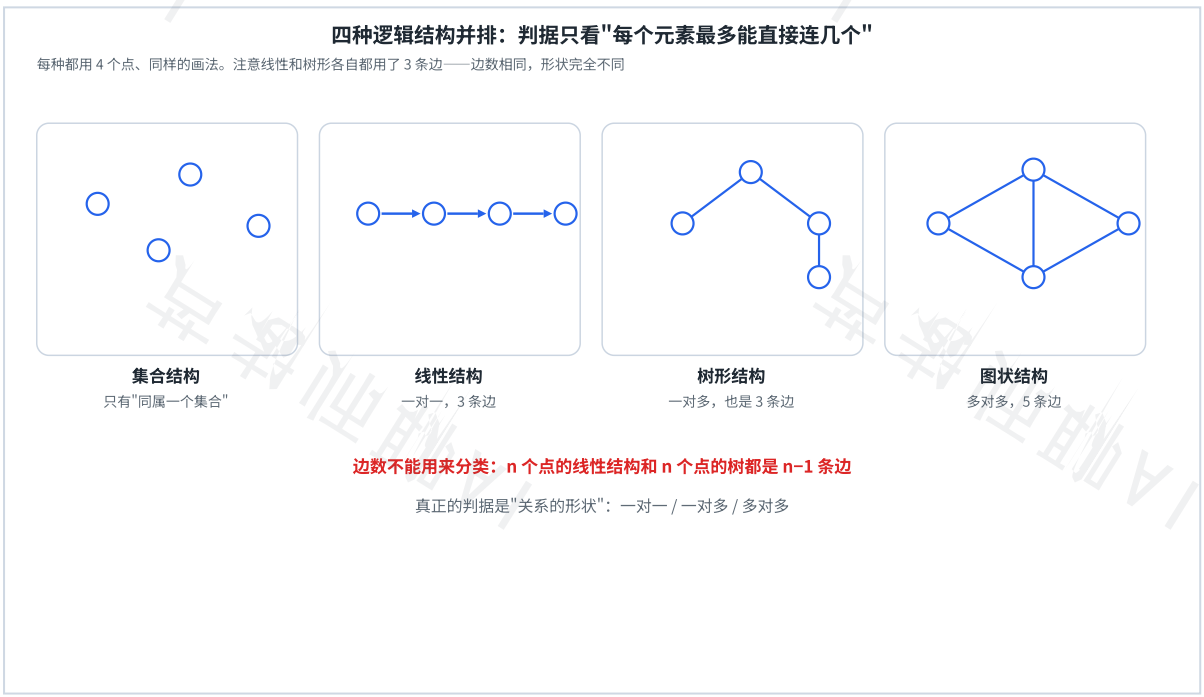


图 1-2　★ 最后拆掉一个错判据：边数不能用来分类。

1.2 算法及其评价

本节配套动画：第 2 项《时间复杂度可视化》、第 4 项《怎么算时间复杂度：四步法与 O(log₂n)》、第 3 项《空间复杂度：辅助空间与递归栈》 · 教材 1.4 算法与算法分析（增长速度分级、渐近符号）

> **定义**
> 算法是对特定问题求解步骤的一种描述，具有**有穷性、确定性、可行性、输入（≥0 个）、输出（≥1 个）**五项特性。评价算法主要看时间复杂度 T(n) 与空间复杂度 S(n)，二者都是**问题规模 n 的函数**。

1.2.1 怎么算时间复杂度：四步法

1. 找出**基本操作**（最内层循环里执行次数最多的那句赋值或比较）；
2. 用频度 T(n) 表示它执行多少次；
3. 只保留**最高阶项**，丢掉系数与低阶项；
4. 写成 O(·)，并用**上界**定义理解：若存在 c、n_0，使 $n > n_0$ 时 $T(n) \le c \cdot f(n)$，则 $T(n) = O(f(n))$。

常见量级由低到高：$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$。

实测对照（n 从 1 增到 16 时各量级的取值）：

n	O(1)	$O(\log_2 n)$	O(n)	$O(n \log_2 n)$	$O(n^2)$	$O(2^n)$
1	1	0	1	0	1	2
2	1	1	2	2	4	4
15	1	3.9	15	58.6	225	32768
16	1	4	16	64	256	**65536**

n = 16 时指数阶已到 65536——**指数阶算法在真实规模上不可行**，这就是“多项式时间 / 非多项式时间”这条分界的现实意义。

1.2.2 对数是怎么来的

循环变量每轮**翻倍**（或折半）时，轮数就是对数。设 `for (i = 1; i < n; i *= 2)`，n = 32：

i 依次为 1 → 2 → 4 → 8 → 16 →（32 时退出），共 **5 轮**，解不等式 2^k ≥ 32 得 $k = \lceil \log_2 32 \rceil = 5$。n 翻到 64 也才 6 轮——**n 翻倍、轮数只加 1**，这就是对数增长，也是折半查找快的根本原因。

嵌套循环要相乘：外层 n、内层 log n，合起来 O(n log n)。

1.2.3 空间复杂度与“原地”

> **定义**
> S(n) 是算法执行过程中**所需辅助空间**的实例数量级，**不含输入本身占用的空间**。

把长度 6 的数组反转，两种写法时间都是 O(n)：

写法	辅助空间	S(n)
Reverse1：先 malloc 一个等长数组 b[]，倒着填	6 格	**O(n)**
Reverse2：两头对撞，只借一个临时变量 t	1 格	**O(1)**

时间一样、结果一样，只有空间一栏分出高下——这就是“**原地算法**”（辅助空间 O(1)）这个词的意义。

递归栈要单独算。递归程序每层调用都要压入工作记录（参数、局部变量、返回地址），故空间复杂度按**最大递归深度**计：汉诺塔 n = 3 时总步数 $2^3 - 1 = 7$，但栈深只有 3，所以 S(n) = O(n)、T(n) = $O(2^n)$。

典型例题

例 1　求下列代码段的时间复杂度：`for (i = 1; i <= n; i++) for (j = 1; j <= i; j++) x++;`

解　内层频度 $\Sigma i = n(n+1)/2$，故 T(n) = **$O(n^2)$**。

例 2　`for (i = 0; i < n; i++) for (j = 0; j < n; j += i + 1) x++;` 的复杂度？

解　第 i 轮内层执行 $\lceil n/(i+1) \rceil$ 次，总频度 $n\cdot(1 + 1/2 + \ldots + 1/n) = n\cdot O(\log n)$，故 **O(n log n)**。

例 3　某算法“最好情况 O(n)、最坏 $O(n^2)$、平均 $O(n^2)$”，下列哪种说法正确：(a) 它的时间复杂度是 O(n)；(b) 说复杂度默认指最坏情况。

解　(a) 错、(b) 对。未加说明时，时间复杂度指**最坏情况**的上界；分析平均情况需给定输入分布。

易错点与考点提示

- 时间复杂度不是“运行秒数”，是**基本操作次数**的量级；同一算法在不同机器上秒数不同、量级相同。
- 只保留最高阶、丢掉系数，但**加法不能丢**：$T(n) = n^2 + n$ 与 $T(n) = n^2$ 同阶，而 $n^2 + n^3$ 是 $O(n^3)$。
- 空间复杂度不算输入占用的空间，只算辅助空间——这是“原地”判定的关键。
- 递归的空间按栈深算，不能只看局部变量个数。
- 有穷性是算法的必备特性：死循环的程序不是算法。
- O 是**上界**，Ω 是下界，Θ 是同阶；教材与考题默认用 O 表述。

习题

1. 求 `for (i = 0; i*i < n; i++) x++;` 的复杂度。
2. 设 $T(n) = 2T(n/2) + n$，$T(1) = 1$，求 T(n) 的量级（归并排序的递推式）。
3. 说明为什么“汉诺塔的时间是 $O(2^n)$、空间是 O(n)”。



图 1-3　第 5 轮：i = 16 × 2 = 32 = 2^5。

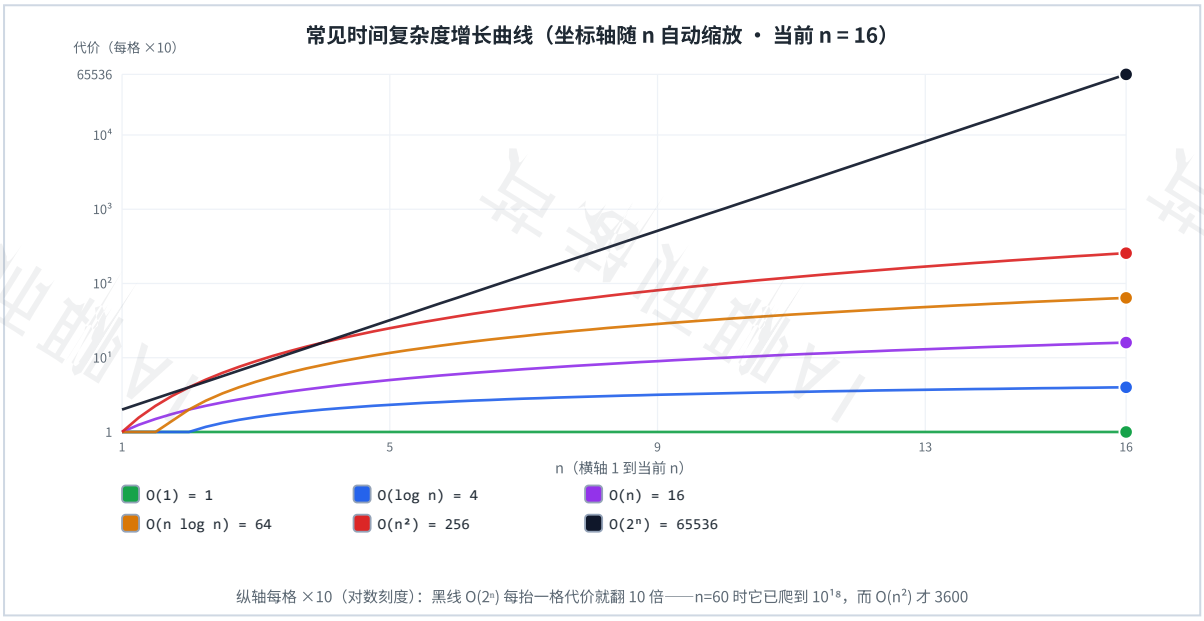


图 1-4　★ n = 16：O(1) = 1，O(log n) = 4，O(n) = 16，O(n log n) = 64，O(n²) = 256，O(2ⁿ) = 65536。



图 1-5 ★ 同样把数组反转：Reverse1 辅助 O(n)、Reverse2 辅助 O(1)。

第 2 章　线性表

2.1　线性表的顺序表示与实现

本节配套动画：第 5 项《顺序表的插入与删除》、第 6 项《顺序表基本操作合集》 · 教材 2.4 线性表的顺序表示和实现（插入/删除、移动次数分析）

> **定义**
> 线性表是 n（n ≥ 0）个**数据元素有限序列** ($a_1, a_2, \ldots, a_n$)，除首元素外每个都有唯一前驱、除末元素外每个都有唯一后继。n = 0 时是空表。位序从 **1** 开始，而数组下标从 0 开始，二者差 1。

2.1.1　顺序表的存储与随机存取

顺序表用**一段地址连续的存储单元**依次存放元素：`L.elem[i-1]` 存第 i 个元素，因此

- **按位（下标）存取是 O(1)**——这是顺序结构最大的优点；
- 逻辑相邻即物理相邻，不必额外存指针，存储密度高；
- 需要**预先分配**连续空间（`MAXSIZE`），插入删除要搬动元素。

2.1.2　插入：为什么必须从后往前移

在**第 i 个位置**插入 e（1 ≤ i ≤ n + 1），要把第 n, n−1, …, i 个元素依次后移一格，空出位置再放入 e，表长加 1。

实例：L = (25, 12, 47, 89, 36, 14)，n = 6，在第 3 位插入 33。

步	动作	已移动
1	下标 5 的 14 移到下标 6	1
2	下标 4 的 36 → 下标 5	2
3	下标 3 的 89 → 下标 4	3
4	下标 2 的 47 → 下标 3	4
5	把 33 放入下标 2（第 3 位）	—

结果 L = (25, 12, **33**, 47, 89, 36, 14)，共移动 **4 = n − i + 1** 个元素。

> **平均性能**
> 在表长 n 的线性表上等概率地在 n + 1 个位置插入，平均移动次数为 **n/2**；最好（在表尾插）0 次，最坏（在表头插）n 次。删除同理，平均 **(n − 1)/2**。故插入删除的时间复杂度都是 **O(n)**。

必须从后往前：若从前往后移，先移动的元素会覆盖尚未搬走的元素，数据丢失。

2.1.3　基本操作的代价

操作	复杂度	说明
按位查找 GetElem(i)	**O(1)**	直接算地址
按值查找 LocateElem(e)	O(n)	顺序扫描，等概率成功 ASL = (n + 1)/2
插入 / 删除	O(n)	需移动元素
求表长 Length()	O(1)	表长存在结构体里

按值查找实例：在上表中找 47，比较 25、12、47 三次命中，返回位序 3。等概率下 ASL = (n + 1)/2 = 3.5（n = 6）。想做到 O(1) 查找，得换第 7 章的散列表。

典型例题

例 1 线性表有 100 个元素，等概率地在任意位置插入，平均要移动多少个元素？若改为等概率删除呢？

解 插入平均 n/2 = **50**；删除平均 (n − 1)/2 = **49.5**。

例 2 设顺序表长度为 n，在第 i 个元素之前插入需要移动 n − i + 1 个元素。问在哪个位置插入代价最小、哪个最大？

解 i = n + 1（表尾）移动 0 个，代价最小；i = 1（表头）移动 n 个，代价最大。

例 3 为什么“顺序表支持随机存取”不等于“顺序表查找很快”？

解 随机存取指**按位序**取元素 O(1)；而**按值**查找仍要逐个比较，O(n)。二者是不同的操作，别混为一谈。

易错点与考点提示

- **位序从 1 起、下标从 0 起**：第 i 个元素在下标 i − 1，插入位置 i 合法区间是 1 ≤ i ≤ n + 1，越界必须报错。
- 插入前先判**表满**（length == MAXSIZE），删除前先判**表空**与位置合法。
- 顺序表的“存储密度”为 1（不存指针），链式表小于 1；这是空间利用率的比较点。
- 顺序结构要求**地址连续**，即使逻辑上能放下，也可能因内存碎片而分配失败。
- 频繁按位访问 + 很少插删 → 顺序表；频繁插删 + 需要大量移动 → 链表。

习题

1. 在长度为 n 的顺序表的**最后一个位置之后**插入元素，需要移动几个元素？在**第一个位置**插入呢？
2. 长度为 7 的顺序表 (25,12,47,89,36,14,20)，删除第 2 个元素要移动几个？写出结果序列。
3. 说明顺序表做 GetElem(i) 为 O(1) 的**前提条件**是什么。



图 2-1 后移：j = 2，满足 j ≥ i−1 = 2。



图 2-2 ★ 小结：按值查找 O(n)。

2.2 线性表的链式表示与实现

本节配套动画：第 7 项《单链表的插入与删除》、第 8 项《单链表基本操作合集》、第 9 项《双向链表与循环链表》 · 教材 2.5 线性表的链式表示和实现（先连后断、与顺序表对比）

> **定义**
> 链式存储用**指针**表达逻辑关系：每个结点含数据域与指针域，结点可散落在内存各处。单链表 $L \to a_1 \to a_2 \to \dots \to a_n \to \wedge$ 。

2.2.1 头结点的作用

链表的第一个结点通常是**头结点**（不存数据，或存表长），其 next 指向首元结点。引入它有两个好处：

1. **第 1 个位置的操作与其他位置一致**——插入删除都要改“前驱的 next”，有了头结点，插到第 1 位时前驱就是头结点，不必特判；

2. 空表与非空表的处理统一（判空条件是 `L->next == NULL`，而不是 `L == NULL`）。

注意区分：**头指针**（必需，是入口）、**头结点**（可选，为了操作一致）、**首元结点**（存 a_1 的那个）。

2.2.2 插入与删除：先连后断

在第 i 个结点之前插入 s：先找到第 i − 1 个结点 p（查找 O(n)），再改两个指针（O(1)）。

```
s->next = p->next;      /* ① 先连：新结点接上后半段 */
p->next = s;            /* ② 后断：前驱改指新结点    */
```

实例：L = (25, 12, 47, 89, 36, 14)，在第 3 位插入 33。p 从头结点起后移 2 次停在 12；先 `s->next = p->next`（s 接上 47），再 `p->next = s`。结果 (25, 12, **33**, 47, 89, 36, 14)。

> **为什么顺序不能反**
> 若先执行 `p->next = s`，则 47 及其后所有结点的地址信息当场丢失，整条后半链再也找不回来。这与顺序表“必须从后往前移”是同一种性质的错误：**先破坏、后读取**。

删除第 i 个结点：找到其前驱 p，`q = p->next; p->next = q->next; free(q);` ——同样只需改指针，不移动任何元素。

2.2.3 双向链表与循环链表

双向链表每结点含 `prior` 与 `next` 两个指针，代价是多一个指针域，收益是**可以从尾结点倒着走**、且删除时不必再从头找前驱。在结点 p 之前插入 s 要改**四条链**：

步	语句	若提前执行会怎样
1	`s->prior = p->prior`	先读 20 的地址
2	`(p->prior)->next = s`	仍依赖第 1 步读到的 20
3	`s->next = p`	—
4	`p->prior = s`	**若最先执行，20 的信息当场丢失**，第 1、2 步无从下手

循环链表：单循环让尾结点的 next 指回头结点；判“到尾”的条件由 `p->next == NULL` 改为 `p->next == L`。双向循环链表则首尾相接，可从任一结点走到任何结点。

2.2.4 顺序表与链表的选型

比较项	顺序表	单链表
存取方式	随机存取 O(1)	顺序存取，按位 O(n)
插入 / 删除	平均移动 n/2 个	只改指针 O(1)（但找位置 O(n)）
空间	需预分配连续区，可能浪费或溢出	按需申请，另付指针开销
存储密度	1	< 1

典型例题

例 1　在带头结点的单链表中查找第 i 个结点，其时间复杂度是多少？若链表按值有序，查找某个值呢？

解　都是 **O(n)**——链表没有随机存取，即使有序也只能顺序走（二分查找要求随机存取，故**有序链表不能折半**）。

例 2　已知指针 p 指向单链表中的某结点（不是首元结点也不是尾结点），且**不知道头指针**，如何删除 p 所指结点？

解　把 p 的**后继**结点的内容复制到 p，再摘掉后继：`q = p->next; p->data = q->data; p->next = q->next; free(q);`，时间 O(1)。此法对尾结点不适用。

例 3　双向链表删除结点 p（不带头结点、p 非首尾）需要改几条链？

解　**两条**：`p->prior->next = p->next; p->next->prior = p->prior;` 再释放 p。比单链表省去“找前驱”的 O(n)。

易错点与考点提示

- 头指针 / 头结点 / 首元结点是三个不同概念，选择题常在此设陷。
- 插入必须**先连后断**，双向插入必须**先动新结点、最后才改 p->prior**。
- 判空：带头结点时是 `L->next == NULL`；不带头结点时是 `L == NULL`。
- 链表**不支持随机存取**，所以按位访问、折半查找都不适用。
- 删除结点必须拿到它的**前驱**（单链表），这是“为什么很多算法都要多走一步找 pre”的原因。
- 静态链表用数组模拟，游标代替指针，适用于没有指针的高级语言或需要整块交换的场合。

习题

1. 说明为什么“在已知结点 p 之后插入一个新结点”是 O(1)，而在 p 之前插入是 O(n)（单链表）。
2. 长度为 n 的单链表，要在第 i 个位置删除结点，需要移动/遍历几个指针？写出等概率下的平均时间量级。
3. 循环双链表在指针 p 所指结点之前插入 s，写出四条修改语句。

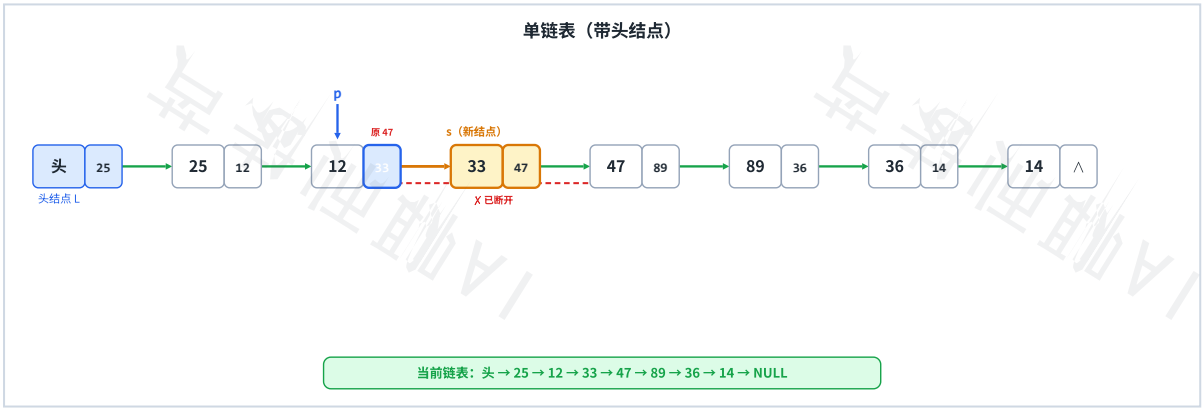


图 2-3　② 后断：p->next = s。

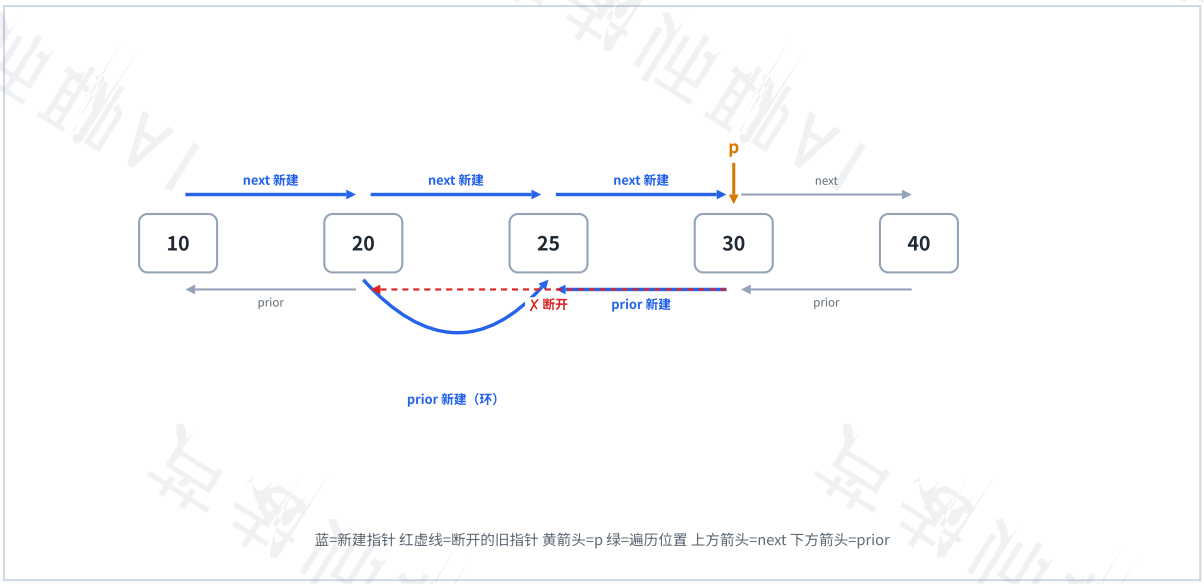


图 2-4 ④ p->prior = s：最后才改 30 的 prior（30→20 的旧 prior 断开，红色虚线）。

2.3 线性表的应用：有序表归并与多项式相加

本节配套动画：第 10 项《合并有序表 LA+LB→LC》、第 11 项《一元多项式相加（有序链表）》 · 教材 2.7 线性表应用（双指针归并，O(m+n)）

> **核心**
> 本节解决两类“两个有序序列合成一个”的问题：有序表归并、一元多项式相加。它们的共同结构是**双指针各走各的、只前进不回头**，因此线性时间。

2.3.1 有序表归并

设 LA = (3, 5, 8, 11)、LB = (2, 6, 8, 9, 15)，i、j 分别指向两表当前头部，每次比较取**较小者**接入 LC，被取的一方指针后移；任一表走完后，把另一表的**剩余部分整段接上**。

步	对视	取走	LC 当前
1	3 vs 2	2	2
2	3 vs 6	3	2, 3
3	5 vs 6	5	2, 3, 5
4	8 vs 6	6	2, 3, 5, 6
5	8 vs 8	8（取 LA 的，保证稳定）	2, 3, 5, 6, 8
…			2, 3, 5, 6, 8, 8, 9, 11, 15

结果 LC = (2, 3, 5, 6, 8, 8, 9, 11, 15)，共 9 个元素、**仅比较 8 次**。

> **结论**
> 合并长度 m 与 n 的两个有序表，比较次数在 **min(m, n)** 到 **m + n − 1** 之间，时间复杂度 O(m + n)。用链表实现时“接入”只是改指针，不必搬动元素；用顺序表实现则需要额外 O(m + n) 空间。

2.3.2 一元多项式的表示与相加

多项式用**系数链表**表示，每结点存 `(coef, exp)` 并按指数**升序**排列，稀疏时远省空间。相加即“有序归并 + 同类项合并”：

- 指数不等 → 取指数小的一项进结果，该指针后移；
- 指数相等 → 系数相加，**和不为零才保留**，两个指针同时后移；
- 一方走完 → 另一方剩余项整段接入。

实例：$A(x) = 7 + 3x + 9x^8 + 5x^{17}$，$B(x) = 8x + 22x^7 - 9x^8$。

步	处理	结果项
1	A 指数 0 < B 指数 1	7
2	$3x^1$ 与 $8x^1$ 同类	$11x$
3	B 的 $22x^7$ 指数最小	$22x^7$
4	$9x^8$ 与 $-9x^8$ 同类，系数和为 **0**	丢弃
5	B 已尽，A 余下 $5x^{17}$ 直接并入	$5x^{17}$

得 $A + B = \mathbf{7 + 11x + 22x^7 + 5x^{17}}$，全程比较 4 次。

典型例题

例 1 两个长度分别为 m、n 的有序表归并，最多比较多少次？最少呢？

解：最多 $m + n - 1$（每次只消耗一个元素，最后一次比较后余下一段直接接上）；最少 $\min(m, n)$（一个表的元素全部小于另一个表的最小元素）。

例 2 为什么多项式用**链表**而不是顺序数组按下标存指数？

解：按下标存要求指数连续且范围有限，稀疏多项式（如最高次 10^6 只有几项）会浪费巨量空间；链表只存非零项，且删除中间项（系数抵消为 0）不必搬动其他项。

例 3 归并两个有序链表时若“相等时取 LB 的”，结果是否仍有序？稳定性如何？

解：仍有序（相等值取哪边都不破坏次序），但**不稳定**——相等元素的相对先后被颠倒。要稳定必须固定“相等时取左边”。

易错点与考点提示

- 归并的输入必须**已经有序**，否则结果不保证有序。
- 多项式相加中“系数抵消为 0”的项**不能留在结果里**，这是最常见的漏写。
- 用顺序表做归并需要辅助空间 $O(m + n)$；用链表只需改指针，空间 $O(1)$。
- 相等时取哪一个决定稳定性，写算法时要明确约定。
- 这一“两指针各走各的”技巧在第 8 章归并排序、第 6 章求并集等处反复出现。

习题

1. 把 LA = (1, 3, 5)、LB = (2, 4, 6, 8, 10) 归并，写出每步的比较对象与总比较次数。
2. A = $3x^4 + 2x^2$，B = $-2x^2 + 5x^6$，求 A + B 并说明哪一步发生了“丢弃”。
3. 若要求归并结果按指数**降序**，输入链表应满足什么条件？



图 2-5　合并完成：LC =（2, 3, 5, 6, 8, 8, 9, 11, 15），共 9 个元素、仅比较 8 次。

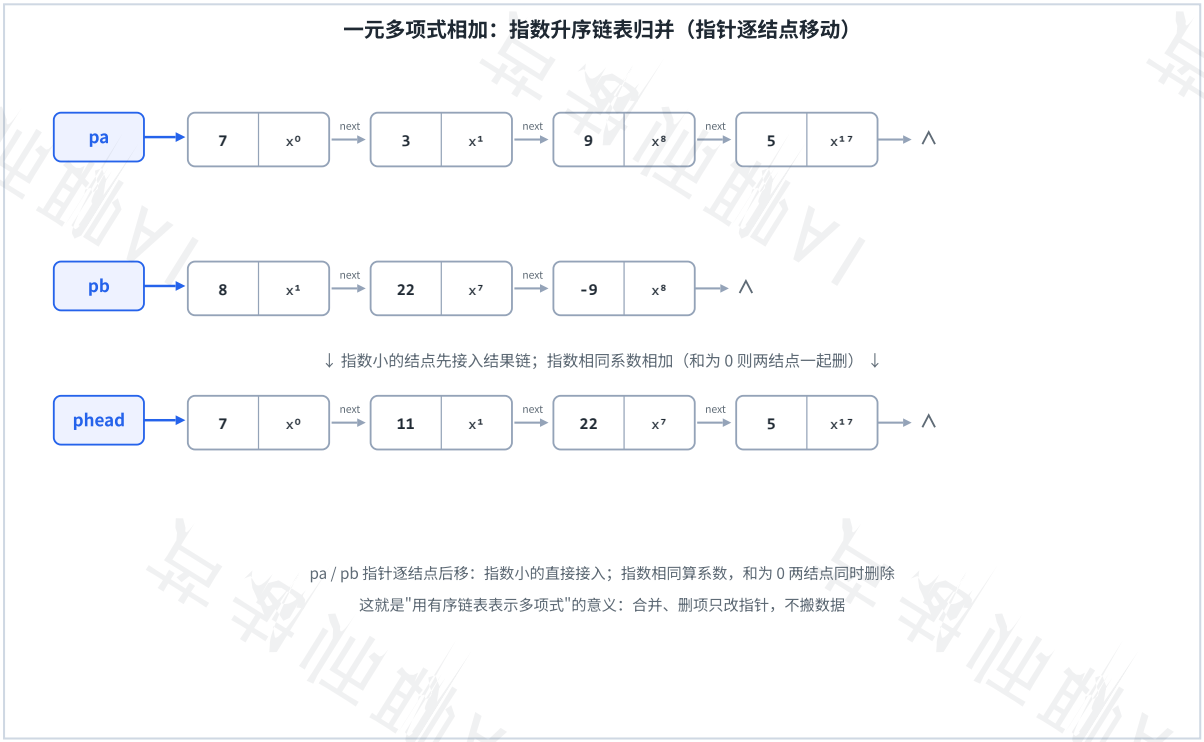


图 2-6　★ A(x) + B(x) = 7 + 11x + 22x^7 + 5x^17。

2.4　链表典型问题

本节配套动画：第 12 项《链表三道经典题：逆置 / 约瑟夫环 / 找公共结点》　·　教材 2.5 延伸实验题（只改指针、循环链表、长度差对齐）

> **要点**
> 链表考题的共性是**只改指针、不新建结点**。三道必做题：链表逆置、约瑟夫环、两链表公共结点。

2.4.1 原地逆置

原表 L → 25 → 12 → 47 → 89 → 36 → ∧，要求逆置为 L → 36 → 89 → 47 → 12 → 25。

做法是**头插法**：先把 L->next 置空（把 L 变成一张空表），然后反复“摘下原表的第一个结点，插到新表头”。

轮	摘下的结点	新表（L 之后）
1	25	25
2	12	12 → 25
3	47	47 → 12 → 25
4	89	89 → 47 → 12 → 25
5	36	**36 → 89 → 47 → 12 → 25**

整个过程只动了 5 次摘挂，**没有新建也没有释放**结点：空间 O(1)，时间 O(n)。

2.4.2 约瑟夫环

n 个人围成一圈、从某人起报数，报到 k 的出列，从其下家重新报数。用**循环链表**自然建模：尾结点指回头结点形成环，出列即 `p->next = q->next`（q 为被删者），然后 p 后移、计数到 k 时删除。

n = 7、k = 3 时的出列次序为 3 → 6 → 2 → 7 → 5 → 1 → 4（最后一个剩下）。

2.4.3 两链表的公共结点

两条单链表若相交，则从交点起其后所有结点**全部共享**（形状是一个 Y，不是 X）。求公共结点的关键是**长度差对齐**：

1. 分别求两表长 m、n；
2. 让长表先走 |m − n| 步；
3. 两指针同步后移，第一个相同的结点即公共结点（比较的是**结点地址**，不是 data 值）。

若两表长度分别为 7、2、5（公共部分长 2），长表先走 7 − 2 = 5 步后同步比较，可在 O(m + n) 时间内找到。

典型例题

例 1 逆置时为什么必须先执行 `L->next = NULL`？

解 头插法要求新表初始为空。若不先断开，第一个被摘下的结点插回时其 next 仍指向原后继，会形成环或把原表结构搅乱。

例 2 判断两个单链表是否相交，最快方法是什么？相交后如何找第一个公共结点？

解 判交：分别走到尾，比较**尾结点地址**是否相同，O(m + n)、O(1) 空间。找公共结点：先求两表长度、让长表走差值步，再同步走，第一个地址相同处即为所求。

例 3 约瑟夫环问题中，若用数组模拟而不是循环链表，代价有何不同？

解 数组需要标记“已出列”并反复跳过，每轮最坏扫全表，总 $O(n \cdot k)$ 或 $O(n^2)$；循环链表删除是 O(1)，但需要 n 个结点的空间。空间换时间的方向相反。

易错点与考点提示

- 逆置后**必须**把原表尾结点的 next 置空，否则最后一个结点仍指向已被摘走的结点，形成环。
- 找公共结点比较的是**指针（结点地址）**；若只比 data 相同，值重复时会误判。
- 相交链表必呈 Y 形：若出现 X 形说明有两个后继，违背单链表定义。
- 约瑟夫环的计数起点容易差 1，写代码前先手工模拟 n = 5、k = 2 的小例子。
- 快慢指针（求中点、判环、找环入口）同属“只改指针”这一族技巧。

习题

1. 写出对 L → a → b → c → ∧ 做原地逆置的每一轮指针变化（用语句表示）。
2. 两条链表长度 6 与 9、公共部分长 3，求两指针各需走多少步才能对齐比较。
3. 设计一个 O(n) 时间、O(1) 空间的算法，判断单链表是否有环，并说明思路。



图 2-7 ★ 逆置完成：L → 36 → 89 → 47 → 12 → 25 → ∧。

第 3 章　栈和队列

3.1　栈的定义、存储与实现

本节配套动画：第 13 项《顺序栈：进栈与出栈》、第 14 项《链栈与链队列：栈和队列的链式实现》 · 教材 3.3 栈的表示和操作的实现（栈空/栈满判定、上溢/下溢）

> **定义**
> 栈是**只允许在一端（栈顶）插入和删除**的线性表，后进先出 **LIFO**。基本操作 InitStack、Push、Pop、GetTop、StackEmpty 均为 O(1)。

3.1.1　顺序栈

用一组连续存储单元存放自栈底到栈顶的元素，同时附设指针 `top` 指示栈顶位置。约定 **top 指向“下一个可插入位置”**，于是：

状态	判定条件
栈空	`top == base`（即 top − base = 0）
栈满	`top - base == stacksize`
栈长	`top - base`
进栈	`*S.top++ = x`（先放元素，top 再加 1）
出栈	`x = *--S.top`（top 先减 1，再取元素）

实例：容量 6 的空栈依次进栈 A、B、C、D、E、F。每进一个，top − base 由 0 增到 6；第 6 个 F 入栈后栈恰好填满，**再多一个元素就上溢**。此时栈顶元素是 F，栈底是 A——后进者位置更高。

> **易错**
> 出栈前要判**栈空**（下溢），进栈前要判**栈满**（上溢）。顺序栈的“上溢”是运行时错误，与“数组定义时越界”不是一回事。

3.1.2　链栈与链队列

链栈在**表头**做插入删除最快（头指针即栈顶指针），因此链栈**不需要头结点**，push / pop 都是 O(1) 且不会上溢（除非内存耗尽）。

链队列必须同时设 `front` 与 `rear` 两个指针：只在表头出队是 O(1)，但若没有 rear，入队还要从头走到尾，失去意义。带头结点时：

操作	语句要点
入队	`rear->next = s; rear = s;`
出队	`p = front->next; front->next = p->next;` 然后释放 p
判空	`front == rear`

> **必须特判**
> 若被删除的结点**恰好是 rear 所指**（队里只剩一个元素），出队后必须令 `rear = front`，否则 rear 指向已被释放的内存，后续入队就写飞。这是链队列最经典的考点。

预置 A、B、C、D、E 依次入队后逐个出队，输出序列仍是 A → B → C → D → E，与入队顺序一致（**FIFO**），与栈相反。

3.1.3 栈与队列的判据对照

	栈	队列
进出端	同一端	一端进、另一端出
次序	后进先出	先进先出
顺序结构痛点	上溢（可共享空间解决）	**假溢出**（循环队列解决）
链式结构	链栈：表头操作	链队列：需 front + rear，出队要特判 rear

典型例题

例 1 设进栈序列为 1, 2, 3, 4，下列哪个不可能是出栈序列？(a) 1234 (b) 2143 (c) 4123 (d) 3214

解 (c) 不可能。4 第一个出栈意味着 1、2、3 都还在栈中且此刻栈顶是 3，下一个出栈必须是 3 而非 1。**判定方法**：对出栈序列中每个元素，其后所有比它小的元素必须按递降排列。

例 2 n 个元素进栈，可能的出栈序列种数是多少？

解 **卡特兰数** C(n) = C(2n, n) / (n + 1)。n = 4 时为 14 种。

例 3 链队列中，若删除结点后不检查 `p == rear`，会出现什么问题？

解 rear 成为**悬空指针**（指向已释放内存）。此时再入队，`rear->next = s` 会写入已释放区域，程序行为未定义。

易错点与考点提示

- top 的约定（指向栈顶元素 or 下一个空位）决定 `*S.top++` 与 `*--S.top` 的写法，两种约定下判满条件不同，做题先确认约定。
- 栈适合“最近优先”（递归、表达式、回溯、括号匹配），队列适合“先到优先”（层次遍历、缓冲区、任务调度）。
- 链栈不需要头结点、链队列需要头结点——原因都是“表头操作是否统一”。
- 判栈空用 `top == base`（顺序栈），判队空用 `front == rear`（循环队列须另办法，见 3.2）。

- 出栈序列合法性判断题，用“比它小的后续元素必须递降”最快。

习题

1. 进栈序列 a, b, c, d，若出栈序列为 a, c, b, d，写出每一步的进 / 出栈操作。
2. 用容量 4 的顺序栈（top 指向下一空位）依次执行 push×5、pop×3，问最终栈内元素个数与是否发生溢出。
3. 链队列里现有 3 个元素，连续出队 3 次，front 与 rear 各停在何处？



图 3-1 进栈 B：检查 top − base = 1 < stacksize = 6，未满，可以入栈。

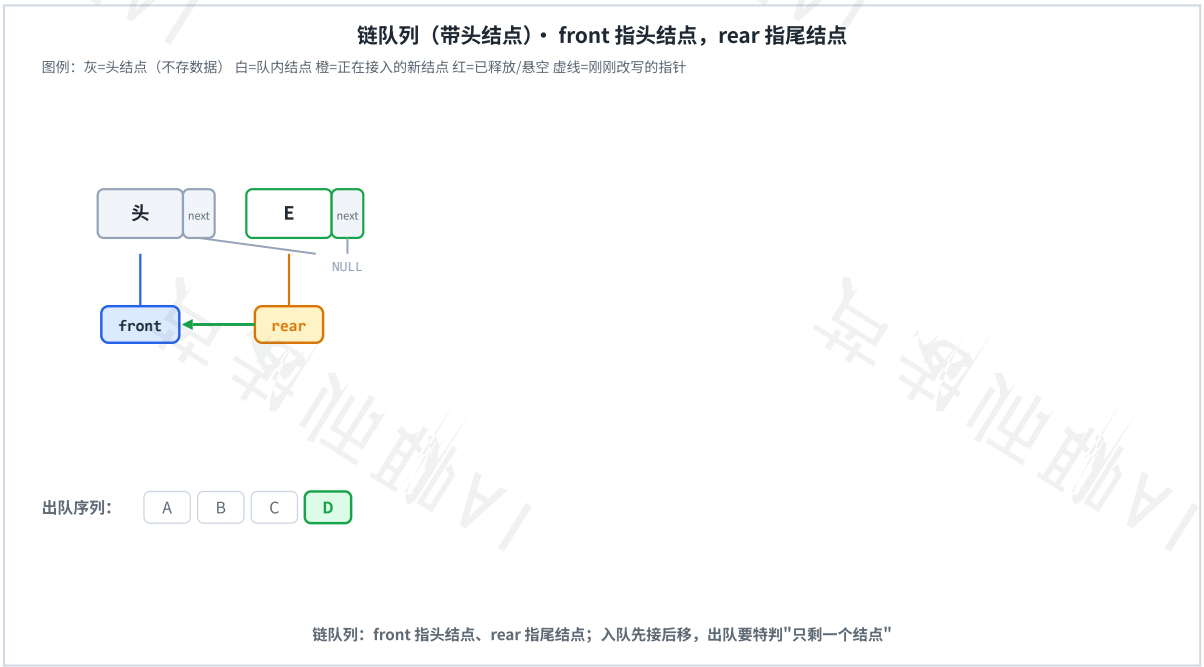


图 3-2 ★ 特判：刚才摘掉的 p 正是 rear 指着的那个结点——队里已经没东西了，必须 rear = front，否则 rear 就指向被释放的内存。

3.2 队列的定义、假溢出与循环队列

本节配套动画：第 16 项《假溢出与循环队列：入队与出队》 · 教材 3.5 队列的表示和操作的实现（假溢出、队空/队满三方案）

> **问题**
> 顺序队列用数组实现、front 与 rear 各自单调后移，出队并不真正释放空间——当 rear 冲到数组上界而前面已空出若干格时，就会报“队满”，这就是**假溢出**。解决办法是把数组首尾相连，构成**循环队列**。

3.2.1 取模是全部技巧的来源

指针一律取模后移：

```
rear  = (rear  + 1) % M;
front = (front + 1) % M;
队长  = (rear - front + M) % M;
```

M 为数组长度。加 M 再取模是为了处理 rear < front 的情形（如 rear = 0、front = 2、M = 6 时队长 = (0 − 2 + 6) % 6 = **4**）。

3.2.2 判空与判满：三种方案

队空与队满都可能出现 `front == rear`，必须区分。三种经典方案：

方案	队空	队满	可存元素数	代价
① 少用一个单元	front == rear	(rear + 1) % M == front	**M − 1**	浪费一格，最常用
② 增设 size	size == 0	size == M	M	多一个变量，增减要维护
③ 增设 tag	front == rear 且 tag == 0	front == rear 且 tag == 1	M	开销最小，但要在每次删除后置 tag = 0、每次插入后置 tag = 1

> **实验要求**
> 上机实验通常指定实现 **tag 方案**——它不浪费空间也不额外计数，但必须保证 tag 的语义是“最近一次操作是插入还是删除”。

实例（M = 6，采用方案①）：初始 front = rear = 0，队长 0。依次入队 a、b、c、d、e（5 个，容量上限 M − 1 = 5），再出队两个：front = (0 + 1) % 6 → (1 + 1) % 6 = **2**，rear = 5，队长 = (5 − 2 + 6) % 6 = 3。此时若再入队一个，(rear + 1) % 6 = 0 == front，判**队满**——尽管数组里还有空格，这正是“少用一格”换来的判别力。

3.2.3 出队腾出的格子可以复用

循环队列的关键收益：出队使 front 后移，那些格子重新成为可用空间。对比“① 假溢出引入”场景可以看到，非循环版本里 rear 到 M − 1 就再也进不去了，而循环版本绕回下标 0 继续用。

典型例题

例 1 循环队列 Q（数组长度 6），front = 3、rear = 0，采用“少用一格”方案，问队列中有几个元素？还能入队吗？

解 队长 = (0 − 3 + 6) % 6 = **3** 个。判满条件 (rear + 1) % M = 1 ≠ front = 3，故**可以**入队。

例 2 若数组长度 M、要求能存放 M 个元素，可选哪些方案？

解 方案②（size）与方案③（tag）都能存满 M 个；方案①必须浪费一格，最多存 M − 1 个。

例 3 设 front 指向队头元素、rear 指向队尾元素的**下一个**位置，写出入队与出队语句。

解 入队：`Q.data[rear] = x; rear = (rear + 1) % M;`；出队：`x = Q.data[front]; front = (front + 1) % M;`。若 rear 改为“指向队尾元素”，则入队要先 `rear = (rear + 1) % M` 再赋值——**约定不同，语句顺序就不同**。

易错点与考点提示

- 队长公式必须写 `(rear - front + M) % M`，漏掉 + M 在 rear < front 时算错。
- “少用一格”方案的容量是 **M − 1**，题目问“能存几个元素”最常在这里丢分。
- front / rear 的约定（指向元素本身还是下一个空位）必须先确认，再写操作语句。
- 假溢出是**顺序队列（非循环）**的问题，不是循环队列的问题；循环队列满时是**真溢出**。
- 判满与判空条件不要混用：tag 方案下 `front == rear` 既可能空也可能满，靠 tag 区分。

习题

1. M = 8、front = 6、rear = 3（少用一格方案），求队长与还能容纳的元素数。
2. 依次对空循环队列（M = 4，少用一格）执行 入、入、出、入、入，写出每步后的 front、rear 与队长。
3. 为什么链队列没有假溢出问题？

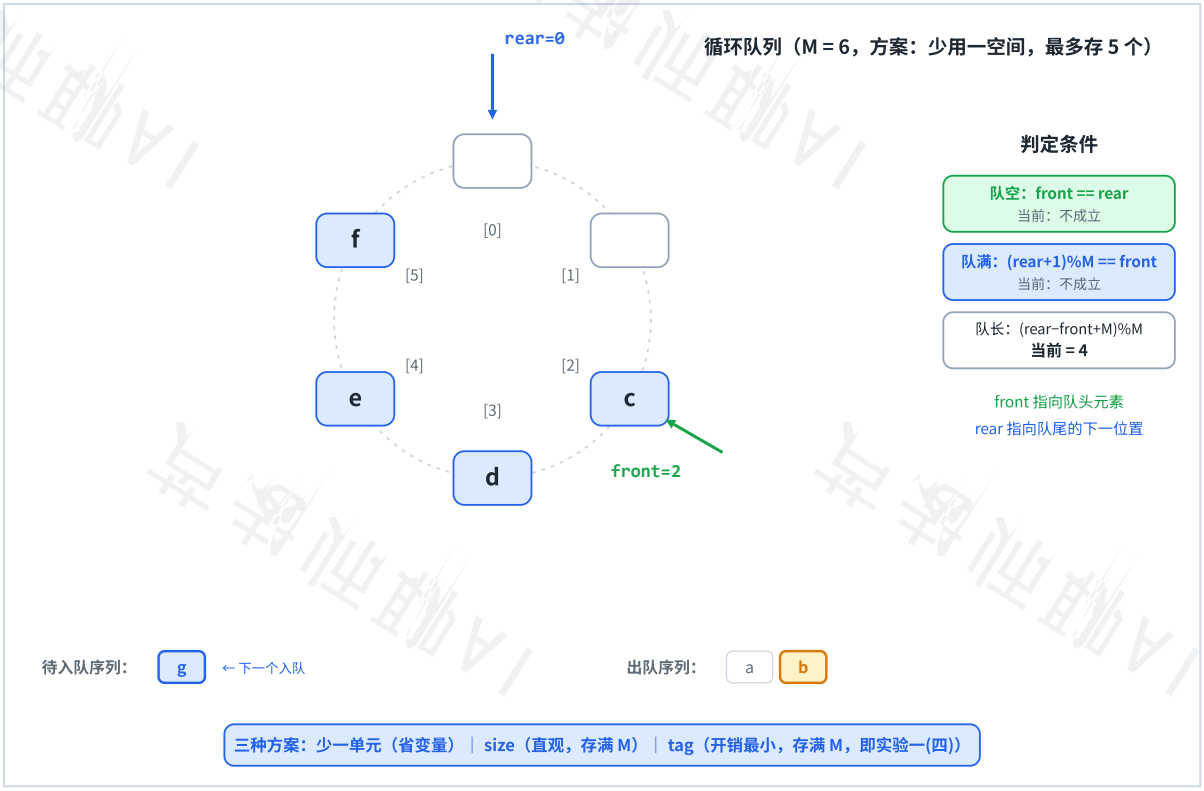


图 3-3　★ 演示结束。当前队长 = (rear − front + M) % M = (0 − 2 + 6) % 6 = 4。

3.3　栈与递归

本节配套动画：第 15 项《递归调用栈：汉诺塔》　·　教材 3.4 栈与递归（递归工作栈）

> **核心**
> 递归的本质是**系统帮你维护一个栈**：每次调用把参数、局部变量与返回地址压入递归工作栈，返回时弹栈恢复现场。看懂栈的压入弹出，才看得懂递归为什么正确、代价在哪里。

3.3.1　汉诺塔

n 个盘按大小叠在 A 柱，要求借助 B 柱全部移到 C 柱，每次只移 1 个盘、大盘不能压小盘。

递归拆解：要把 n 号（最大）盘从 A 移到 C，必须先把上面 n − 1 个盘从 A 移到 B（以 C 为辅助），再把最大盘移到 C，最后把 B 上的 n − 1 个盘移回 C（以 A 为辅助）：

```
hanoi(n, A, C, B):
    if n == 1: 直接把 A 顶移到 C
    else: hanoi(n-1, A, B, C);  移动 A→C;  hanoi(n-1, B, C, A)
```

步数满足 f(n) = 2f(n − 1) + 1、f(1) = 1，解得 **$f(n) = 2^n - 1$**。

n = 3 的完整过程（7 步）：A→C、A→B、C→B、A→C、B→A、B→C、A→C。递归调用最大栈深 3 = n。

量	值（n = 3）	一般式
移动步数	7	$2^n - 1$
最大栈深	3	n
时间复杂度	—	$O(2^n)$
空间复杂度	—	O(n)

3.3.2 递归工作栈里到底存了什么

每层调用压入一条**工作记录**：实参（n 与三个柱名）、局部变量、**返回地址**（回到上一层的哪一行）。因此：

- 递归能“回到原地继续”，靠的是栈里保存的返回地址；
- 递归的栈深 = 递归树的最大高度，这决定了空间复杂度；
- 把每层的 n 与起止柱都压进栈，就能手工模拟递归——这也是把递归改写成非递归（显式栈）的标准做法。

3.3.3 递归 → 非递归的一般方法

任何递归都可写成“自己维护一个栈”的非递归形式：把待处理的问题实例压栈，循环弹栈处理、需要递归时把子问题压栈。汉诺塔的非递归版本每次比较“能否把当前最小可移动盘挪一步”，步数仍是 $2^n - 1$，但省去了函数调用开销。

典型例题

例 1 4 个盘的汉诺塔需要多少步？若机器每秒执行 10^9 步，需要多久？

解 $2^4 - 1 = \mathbf{15}$ 步。64 个盘要 $2^{64} - 1 \approx 1.8 \times 10^{19}$ 步，按 10^9 步/秒约 580 多**亿年**——指数阶算法在真实规模上不可行的典型例证。

例 2 递归函数 `f(n) = n == 0 ? 1 : f(n - 1) + 2;` 的时间与空间复杂度？

解 每层做一次常数工作、共 n 层，T(n) = **O(n)**，栈深 n 故 S(n) = **O(n)**。

例 3 为什么“递归的空间复杂度按栈深算，而不是按局部变量总数算”？

解 同一时刻栈里只有从根到当前结点这一条路径上的记录，共“最大深度”层；每层记录大小是常数，故总空间 = O(栈深)。

易错点与考点提示

- 递归必须有**边界条件**，否则不满足算法的有穷性（死循环）。
- 汉诺塔的步数 $2^n - 1$ 与栈深 n 是两个不同的量，别把空间也写成指数。
- 斐波那契的朴素递归是 $O(2^n)$（大量重复子问题），记忆化或递推可降到 O(n)——“递归慢”通常慢在重复计算，不是慢在调用本身。
- 尾递归（递归调用是最后一步且不参与计算）通常可改写成循环，空间从 O(n) 降到 O(1)。

- 手工模拟递归时，一定要把“返回地址”记下来，否则会漏掉第二次递归调用。

习题

1. 写出 n = 2 时汉诺塔的完整移动序列与栈深变化。
2. 递推 T(n) = 2T(n − 1) + 1、T(1) = 1，验证 T(4) = 15。
3. 用“自己维护栈”的方法，把 `void pre(Tree t){ if(t){ visit(t); pre(t->lchild); pre(t->rchild); } }` 改写成非递归先序遍历，写出核心循环。

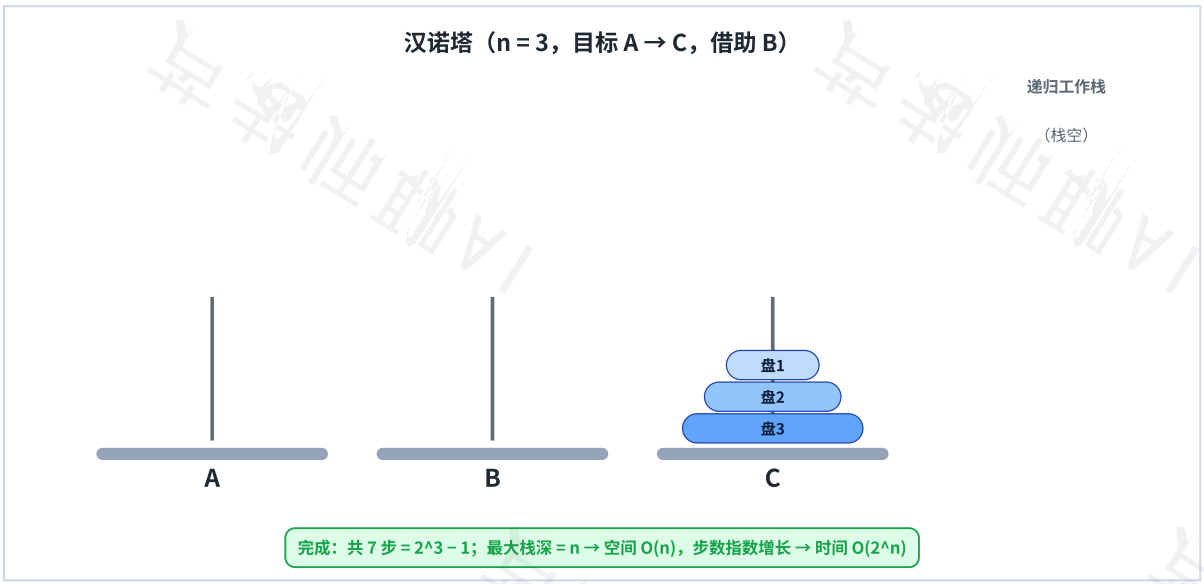


图 3-4 完成！共移动 7 步 = 2^3 − 1。

3.4 栈和队列的应用

本节配套动画：第 17 项《括号匹配：栈的应用》、第 18 项《表达式求值：双栈法（案例3.3）》、第 19 项《中缀转后缀与后缀式求值（逆波兰）》、第 20 项《数制转换（栈的应用）》、第 21 项《迷宫求解（栈与回溯）》、第 22 项《队列的应用：打印机任务排队》 · 教材 3.6 案例分析与实现（案例3.2 括号匹配）

> **核心**
> 栈的四类典型应用：括号匹配、表达式求值与转换、数制转换、回溯（迷宫）。它们的共同点是“**最近出现的对象最先被处理**”。

3.4.1 括号匹配

规则：左括号一律进栈；遇到右括号时，它必须与**栈顶**的左括号配对——最后出现的左括号最先被匹配（LIFO）。扫描结束时栈必须为空。

实例 `([()])`：

读入	动作	栈（底→顶）
(	进栈	(
[	进栈	([
(	进栈	([(
)	与栈顶 (配对 √ 弹出	([
]	与栈顶 [配对 √ 弹出	(
)	与栈顶 (配对 √ 弹出	（空）

扫描结束且栈空 → **匹配成功**。三种失败情形：遇右括号时栈空（多右括号）、栈顶不匹配（类型交叉，如 `([)]`）、扫描完栈非空（多左括号）。

3.4.2 表达式求值：双栈法

设 **OPTR** 运算符栈（栈底压入结束符 `#`）与 **OPND** 操作数栈，从左向右扫描：

- 读入**操作数** → 压入 OPND；
- 读入**运算符** → 与栈顶运算符比优先级：栈顶**优先级低**则压栈，否则**弹出栈顶运算符连同两个操作数计算**，结果压回 OPND，并继续与新栈顶比较（当前运算符暂不消费）。

实例 `3*(7-2)`：读 3 → OPND(3)；读 `*`（高于 #）→ OPTR(#, *)；读 7 → OPND(3,7)；读 `-`（低于 `*`）→ OPTR(#, *, -)；读 2 → OPND(3,7,2)；读 `)` → 弹 `-` 算 7 − 2 = 5 → OPND(3,5)；括号出栈；扫描结束，弹 `*` 算 3 × 5 = **15**。共 2 次弹栈计算，时间 O(n)。

3.4.3 中缀转后缀（逆波兰式）

规则：操作数**直接输出**；运算符进栈前，先把栈里“该先算的”（优先级不低于它的）弹出输出；右括号则一直弹到左括号。

实例 `12/(4-2)+3*5-8/4` → 后缀式 `12 4 2 - / 3 5 * + 8 4 / -`（13 个 token）。后缀式**没有括号**，运算顺序完全由位置决定，一遍扫描即可求值：遇操作数压栈、遇运算符弹两个算完压回。

3.4.4 数制转换与回溯

数制转换：`N = (N div d) × d + (N mod d)`，每轮余数是当前最低位，故**除 d 取余、余数逆序输出**。逆序正靠栈：

1348 → 八进制：1348 % 8 = 4（入栈）、168 % 8 = 0、21 % 8 = 5、2 % 8 = 2 → 弹出顺序 2 5 0 4，得 $(1348)_{10} = (2504)_{8}$。

迷宫求解 = 深度优先 + 回溯：把“当前位置与已试方向”压栈，走通则继续，撞墙则弹栈退回上一个岔口换方向。栈中保存的正是“从入口到现在”的完整路径——与第 6 章的 DFS 同构。

典型例题

例 1 把中缀式 `A + B * C - D / E` 转为后缀式。

解 `A B C * + D E / -`。乘除先出栈，加减同级时把前一个运算符先弹出。

例 2 后缀式 `6 2 + 3 * 4 2 / +` 的值是多少？

解 (6+2)×3 + 4/2 = 24 + 2 = **26**。

例 3 $(1348)_{10}$ 转十六进制是多少？说明为什么余数必须逆序输出。

解 1348 % 16 = 4，84 % 16 = 4，5 % 16 = 5 → 逆序得 **$(544)_{16}$**。因为每轮取到的是**当前最低位**，先取到的应排在最后，天然与“先进后出”吻合。

易错点与考点提示

- 括号匹配的失败有三种，答题要写全（栈空、不匹配、扫描完非空）。
- 双栈求值里“栈顶运算符优先级**低**才压栈”，等号情形要先算——左结合性的来源。
- 中缀转后缀时**左括号入栈但不弹出**，右括号才成对消失；括号本身不出现在后缀式里。
- 数制转换的余数顺序、栈的输出顺序是**逆序**关系，别写成正序。
- 表达式扫描一遍即 O(n)，每个运算符与括号至多进出栈各一次。

习题

1. 判断 `((a+b)]` 在哪一步失败、属于哪种情形。
2. 写出 `a*(b+c)-d/e` 的后缀式。
3. 用双栈法求 `6/(2*(3-1))+5`，列出每次弹栈计算的算式。



图 3-5　★ 两栈都只剩 "#"，扫描结束——OPND 栈顶即为结果：3*(7-2) = 15。



图 3-6 ★ 转换完成：12/(4−2)+3*5−8/4 → 后缀式 12 4 2 − / 3 5 * + 8 4 / −。



图 3-7 t=2：P3（4 页）提交。

第 4 章　串、数组和广义表

4.1　串与模式匹配

本节配套动画：第 23 项《模式匹配：BF 与 KMP》 · 教材 4.3 串的模式匹配（BF 回退对比 KMP 不回退、next 数组）

> **定义**
> 串（字符串）是由零个或多个字符组成的有限序列。串的模式匹配：在主串 S 中找第一个与模式 T 完全相同的子串，返回其**位置**。

4.1.1　BF 算法：主串指针要回溯

Brute-Force 从主串第 1 位起，逐趟与模式对齐比较；一旦失配，**主串指针回退到本趟起点的下一位**，模式指针回到 1，重新比。

设 S = acabaabaabcacaabc、T = abaabcac（长 8）。BF 共比较 **20 次**才在第 6 位找到匹配。回溯是 BF 慢的根源：前面已经确认相等的字符被白白重比。

4.1.2　KMP：主串指针从不回溯

关键观察：失配时，模式串中**已经匹配上的那一段**自身可能含有“前缀 = 后缀”的结构，于是可以把它整体滑到下一个可能位置，而不必让主串指针后退。

> **next 数组**
> next[j] = 模式串前 j − 1 个字符的**最长相等前后缀长度 + 1**。含义：模式第 j 位失配时，j 滑到 next[j] 继续比，主串指针 i **不动**。

实例：T = `abaabcac`，逐位求 next：

j	1	2	3	4	5	6	7	8
T[j]	a	b	a	a	b	c	a	c
前 j−1 个字符	—	a	ab	aba	abaa	abaab	abaabc	abaabca
最长相等前后缀长	—	0	0	1	1	2	0	1
next[j]	**1**	1	1	2	2	3	1	2

结果对照：同一组数据 BF 比较 20 次、KMP 比较 **15** 次，都在主串第 6 位匹配成功。

4.1.3　复杂度与适用性

算法	预处理	匹配	主串指针	说明
BF	无	O((n − m + 1)·m) 最坏	回溯	实现简单，平均性能尚可
KMP	next 数组 O(m)	**O(n)**	不回溯	模式前缀重复越多、主串越长，优势越大

next 数组只需对同一模式**算一次**，多次查找可复用——这是 KMP 在“一个模式、多个文本”场景下额外的好处。

典型例题

例 1 求模式 `abcabaa` 的 next 数组。

解 逐位算前缀表：j=1 → 1；j=2（“a”，0）→ 1；j=3（“ab”，0）→ 1；j=4（“abc”，0）→ 1；j=5（“abca”，前缀 a = 后缀 a，1）→ 2；j=6（“abcab”，“ab” = “ab”，2）→ 3；j=7（“abcaba”，“a” = “a”，1）→ 2。故 next = **[1, 1, 1, 1, 2, 3, 2]**。

例 2 为什么 KMP 的主串指针不需要回溯？

解 因为失配时已知 S 的某一段与 T 的前缀相同，而 T 的那一段自身有相等的前后缀——把 T 滑动到“后缀与前缀对齐”的位置即可继续，S 中被跳过的那些位置**不可能**成为匹配起点（否则与“最长相等前后缀”矛盾）。

例 3 若模式串所有字符互不相同，next 数组是什么形状？

解 next[j] = 1（对所有 j > 1），因为任何长度的前缀都没有相等的后缀。此时 KMP 退化为“失配就把模式整体右移一位”，与 BF 的滑动量相同，但主串指针仍不回溯。

易错点与考点提示

- next 有**下标从 1 起（值 ≥ 1）**与**从 0 起（部分教材记为 −1）**两套约定，做题必须先确认，两者数值差 1。
- 定义是“前 **j − 1** 个字符”的最长相等前后缀 **+ 1**，漏掉 −1 或漏掉 +1 是最常见的计算错误。
- “相等前后缀”必须是**真前缀 / 真后缀**（不能取整串自身）。
- KMP 的匹配位置指**主串中子串的首位**，别写成失配位置。
- KMP 快在**主串指针不回溯**，而不是“比较次数一定少于 BF”——短模式、随机文本上两者接近。

习题

1. 求模式 `aabaaab` 的 next 数组（下标从 1 起）。
2. 主串 `ababaaababaa`、模式 `abaab`，用 KMP 写出第一次失配后模式滑到的位置。
3. 说明 BF 最坏情况下为什么是 O(n·m)。



图 4-1 next = [0, 1, 1, 2, 2, 3, 1, 2]。



图 4-2 BF 结束：成功：匹配位置 = 主串第 6 位，共比较 20 次。

4.2 数组的压缩存储

本节配套动画：第 24 项《矩阵压缩存储》 · 教材 4.5/4.6 特殊矩阵与稀疏矩阵（对称矩阵映射、三元组快速转置）

> **动机**
> 当数组中大量元素取值固定（零元素或常数元素）时，为它们分配存储单元是浪费。**压缩存储**只为非零元素（或不同值元素）分配空间，并为它们设计**位置映射公式**——理想情况下仍能 O(1) 随机存取。

4.2.1 对称矩阵

n 阶对称矩阵满足 a[i][j] = a[j][i]，只需存下三角（含对角线）共 **n(n + 1)/2** 个元素，压缩率 1 − (n+1)/(2n) ≈ **50%**。

把下三角按行优先存进一维数组 sa（下标从 0 起），则元素 a[i][j]（i、j 从 1 起）：

> **映射**
> 当 i ≥ j 时 **k = i(i − 1)/2 + (j − 1)**；当 i < j 时取对称位置 a[j][i]，即 k = j(j − 1)/2 + (i − 1)。

实例：4 阶对称矩阵，查 m[3][2]。因 3 ≥ 2，k = 3 × 2 / 2 + 2 − 1 = **4**（第 3 行之前有 1 + 2 = 3 个元素，本行内是第 2 个 → 下标 4）。读 sa[4] 即得，与矩阵中 m[3][2] 一致，**存取都是 O(1)**。

4.2.2 三角矩阵与对角矩阵

类型	存储量	说明		
下三角 + 常数上三角	n(n+1)/2 + 1	多存一个常数单元 C		
三对角（次对角线）矩阵	3n − 2	a[i][j] 非零仅当 |	i − j|	≤ 1，映射 k = 2(i − 1) + j − 1
上三角	n(n+1)/2	与下三角对称处理		

4.2.3 稀疏矩阵：三元组与十字链表

稀疏矩阵的非零元素远少于 n × m，压缩方式有两类：

① 三元组顺序表：每个非零元存 `(行 i，列 j，值 v)`，另存行数、列数、非零元个数 total。优点是省空间、便于转置；缺点是**不能随机存取**任意 a[i][j]，且做加法时若结果非零元个数未知，需要预留最大空间。

② 十字链表：每个非零元一个结点，含 `row / col / value / right / down`，同一行的结点串成行链、同一列串成列链，行头与列头各一张表。适合**会改变结构**的运算（加法、相乘），因为可以随时插入或删除结点。

快速转置：普通转置对每个元素扫全表，O(n × m)。设 `num[col]` 为第 col 列非零元个数、`cpot[col]` 为它在转置后三元组中的**起始位置**，则

```
cpot[1] = 1;  cpot[col] = cpot[col-1] + num[col-1];
```

每个元素只需一次定位，复杂度降为 **O(m + t)**（t 为非零元个数）。

典型例题

例 1　设 100 阶对称矩阵按下三角存入一维数组（下标从 0 起），元素 a[60][20]（i > j）存放在下标几？

解　k = 60 × 59 / 2 + (20 − 1) = 1770 + 19 = **1789**。

例 2　一个 n 阶三对角矩阵按行存入一维数组，元素 a[i][j]（|i − j| ≤ 1）的下标是什么？

解　前 i − 1 行共存 3(i − 1) − 1 = 3i − 4 个元素（第一行只 2 个），本行内偏移为 j − i + 1，故 **k = 3i − 4 + (j − i + 1) − 1 + 1 = 2i + j − 3**（下标从 1 起时）；下标从 0 起则再减 1。

例 3　为什么稀疏矩阵用三元组存储做加法不方便？

解　加法结果的**非零元个数事先不知道**，需要按最坏情况（2t）预留空间；而按行定位还要在两个表之间反复比较列号。改用十字链表（或行逻辑链接的三元组）即可动态增删结点。

易错点与考点提示

- 压缩存储的核心是**位置公式**，公式错一位就全错；推导时先数“前 i − 1 行共有几个元素”。
- 对称矩阵压缩后**失去随机存取能力**的说法是错的——只要公式对，仍是 O(1)。
- 三元组顺序表**不能**按 a[i][j] 直接取元素，只能顺序查找，这是它与“压缩后仍可随机存取”的区别所在。
- 稀疏矩阵的“稀疏”没有严格定义，工程上以非零元占比判断；考题给出三元组即默认稀疏。
- 快速转置的两个辅助数组 num 与 cpot 必须先统计再前缀和，顺序不能颠倒。

习题

1. 6 阶对称矩阵需存多少个元素？压缩后节省百分之多少？
2. 下三角（含对角线）按行优先存入 sa[0..20]，求 a[5][3] 的下标。
3. 一个 1000 × 1000 的矩阵有 1000 个非零元，用三元组（每元 3 个域，每域 4 字节）存储约占多少空间？与完整存储相比节省多少？



图 4-3 一般结论：对角矩阵、三角矩阵、对称矩阵都可用"位置公式"压进一维数组；

4.3 广义表

本节配套动画：第 25 项《广义表：链式存储与表头/表尾/长度/深度》 · 教材 4.4 广义表（tag/sub/tp 三域结点、GetHead/GetTail、长度与递归深度）

> **定义**
> 广义表 GL = $(a_1, a_2, \ldots, a_n)$，其中每个元素要么是**原子**（不可再分），要么是一个**子表**。广义表是线性表的推广：允许元素本身是表，因而可以递归定义。零个元素的广义表记作 `()`，长度为 0。

4.3.1 长度与深度：两个不同的量

概念	定义	例
长度	最外层元素的**个数** n	Length((a,(b,c),(),d)) = **4**
深度	括号嵌套的最大层数：原子深度 0，空表深度 1	Depth((a,(b,c),(),d)) = **2**

深度递推式：`Depth(原子) = 0`，`Depth(表) = 1 + max{Depth(各元素)}`。

实例：求 Depth((a,(b,c),(),d))。外层贡献 1 层，各元素 a → 0、(b,c) → 1、() → 1、d → 0，最大值 1，故结果 = 1 + 1 = **2**，递归共访问 7 个结点。

4.3.2 表头与表尾

> **GetHead / GetTail**
> 非空广义表的**表头**是第一个元素 a_1，可以是原子也可以是子表；**表尾**是其余元素组成的**表**（一定带括号，一定是表）。

表	表头	表尾
(a, (b, c), (), d)	a	((b,c), (), d)
((a, b))	(a, b)	( ) 即空表
(a)	a	( )

易错点：表尾是“其余元素组成的表”，所以 `GetTail((a,b,c)) = (b,c)` 而不是 `b`；而空表没有表头表尾。

4.3.3 链式存储：靠 tag 域区分

每个结点含三个域：

```
tag | data / sublist | link
```

- `tag = 0`：该结点是**原子**，data 域存元素值；
- `tag = 1`：该结点是**子表**，sublist 域指向子表的第一个结点；
- `link` 指向同层的下一个结点。

因此“是原子还是子表”不需要猜，看 tag 即可——这是广义表链表结构的精髓。求深度、求长度都靠沿 link 横向走 + 沿 sublist 纵向钻的递归完成。

典型例题

例 1 写出 A = (a, B)，B = (A, ()) 的长度与深度。

解 A 的长度 2（元素 a 与 B）；B 的长度 2（元素 A 与 ()）。按递归定义展开，A 与 B 互相引用形成**递归表**，深度无定义（不是有限值）。这说明“任何广义表都有深度”是错的——递归表除外。

例 2 `GetHead(GetTail(GetHead(((a,b),(c,d)))))` 的结果是什么？

解 最外层只有一个元素 `(a,b),(c,d)` 构成的子表 → GetHead 得 `(a,b),(c,d)`；其 GetTail 得 `((c,d))`；再 GetHead 得 `(c,d)`。

例 3 为什么广义表的表尾一定带括号，而表头不一定？

解 表尾是“其余元素组成的**表**”，本质上是一个广义表，必须写成括号形式；表头只是第一个**元素**，它可能是原子（如 a）也可能是子表（如 (b,c)）。

易错点与考点提示

- 长度数**最外层元素个数**，不是所有原子个数；`((a,b))` 的长度是 1。
- 空表 `()` 长度为 0、深度为 1——两个值别记反。
- 表尾一定是表；表头可以是原子。
- 递归表（表出现在自己的定义中）没有有限深度。
- 广义表既能用链式（tag 域）表示，也能用“头尾链”表示，读题时注意教材采用哪种结点结构。

习题

1. 求 Length 与 Depth：`((a,b), (c,d))`、`(a, (b, (c, (d))))`、`()`。
2. 设 D = (a, (a, b), D)，写出它的深度是否有限，并说明原因。
3. 用 tag/sublist/link 结构画出 `(a, (b, c))` 的链表。



图 4-4 ★ Depth((a,(b,c),(),d)) = 2。

第 5 章　树和二叉树

5.1　树的定义与存储结构

本节配套动画：第 26 项《树的三种存储结构：双亲 / 孩子 / 孩子兄弟》 · 408 大纲 四(三)1 树的存储结构（三种表示在同一棵树上对照，找爹与找孩子各要摸几格）

> **问题**
> 树中每个结点的孩子个数不定，用“每个结点固定 k 个指针”的存储法要么浪费、要么放不下。三种表示法各自解决一半问题：**双亲表示法找爹快、孩子表示法找娃快、孩子兄弟表示法能表示森林**。

5.1.1　双亲表示法

以一组**连续存储单元**存放结点，每个结点附设 `parent` 域记录其双亲在数组中的下标（根结点的 parent 设为 −1）。

操作	代价	原因
找双亲	**O(1)**	parent 域直接给下标
找孩子	O(n)	必须扫全表找 parent == i 的项
求树高 / 深度	O(n)（每结点向上爬）	沿 parent 链走到根

可增设 `firstChild` 域缓解“找孩子慢”，但会浪费空间（多数结点只有一个孩子）。并查集（5.7 节）正是双亲表示法的直接应用。

5.1.2　孩子表示法

把每个结点的孩子排成一条**单链表**，n 个结点就有 n 条孩子链，再用一个**表头数组**（下标即结点编号）串起来；表头结点还可兼存“该结点的双亲下标”，得到“双亲 + 孩子表”的混合结构。

操作	代价
找孩子	**O(度数)**——只与孩子个数有关
找双亲	O(1)（若表头存了 parent）；否则 O(n)
判断是否为兄弟	O(1)（看是否在同一条链上）

5.1.3　孩子兄弟表示法（二叉链表）

> **结构**
> 每个结点两个指针：`firstchild` 指向**第一个孩子**，`nextsibling` 指向**下一个兄弟**。于是一长串孩子被压成一条兄弟链，任意一棵树都能这样表示成二叉树形态——这就是“左孩子右兄弟”。

实例（8 个结点的树，求结点 B 的相关操作）：

- 找孩子：`firstchild(B) → E`，再沿 `nextsibling` 走到 F，共访问 **3 格**，代价只与孩子个数有关；
- 找双亲：结点里**根本没存爹**，只能从根 A 出发沿 firstchild / nextsibling 逐个点问“你的孩子是不是 B”，最坏要走遍整棵树——本例走过 A → B → C → D 共 **4 / 8** 格。

三种里**只有孩子兄弟表示法能顺手表示森林**（把各棵树的根串成兄弟链即可），代价是每格要存两个指针、且找双亲最麻烦。

5.1.4 三种表示法对照

表示法	找双亲	找孩子	能否表示森林	空间
双亲（顺序）	**O(1)**	O(n)	不便	连续数组，无指针
孩子（链式）	O(n)（或 O(1) 加 parent）	**O(度数)**	不便	n 条链 + 表头数组
孩子兄弟	O(n)	O(度数)	**可以**	每结点 2 个指针

典型例题

例 1 某树用双亲表示法存储，结点 7 的 parent 域依次为 5、3、3、−1、7、7、3（下标 0~6）。画出这棵树并求树高。

解 下标 3 的 parent = −1 是根；0 → 7? 修正：parent[0]=5、parent[1]=3、parent[2]=3、parent[3]=−1、parent[4]=7 越界，说明该数据非法——**双亲表示法的第一个自检就是 parent 必须在 [−1, n−1] 内**。合法化后（把 7 改为 3）树高为 3。

例 2 为什么“树的孩子兄弟表示法”和“树与二叉树的转换”得到的是同一棵树？

解 因为转换口诀“每个结点的第一个孩子作左孩子、下一个兄弟作右孩子”正是孩子兄弟表示法的定义。所以树转二叉树不需要移动任何结点，只是给两个指针换了解释。

例 3 n 个结点的树，用孩子表示法存储，一共有多少条孩子链、多少个链结点？

解 n 条链（每个结点一条，叶子链为空）；链结点总数 = **n − 1**（除根外每个结点恰好是某个结点的孩子，在链中出现一次）。

易错点与考点提示

- 双亲表示法**不能唯一确定**一棵树吗？——在给定“结点编号 + parent 数组”时可以；但孩子兄弟表示法同样一一对应，别混淆两者的适用场景。
- 孩子兄弟链表中，结点的**右链**是兄弟、**左链**是第一个孩子，看反了遍历序全错。
- 树的度 = 各结点孩子数的最大值；森林中树棵数 = 根结点个数 = 对应二叉树中“最左链往右走”的结点数。
- 求树的深度时对每个结点向上爬 parent 链，最坏 $O(n^2)$；一次 DFS 可降到 O(n)。

习题

1. 一棵有 n 个结点的树，用双亲表示法存储，求“找叶子结点”需要扫描多少个结点、为什么不能少于 n。

2. 用孩子兄弟表示法画出树 A(B(E,F,G), C, D(H))，并写出各结点的 firstchild / nextsibling。

3. 为什么“森林转二叉树”后，二叉树的右链长度对应森林里的树棵数？

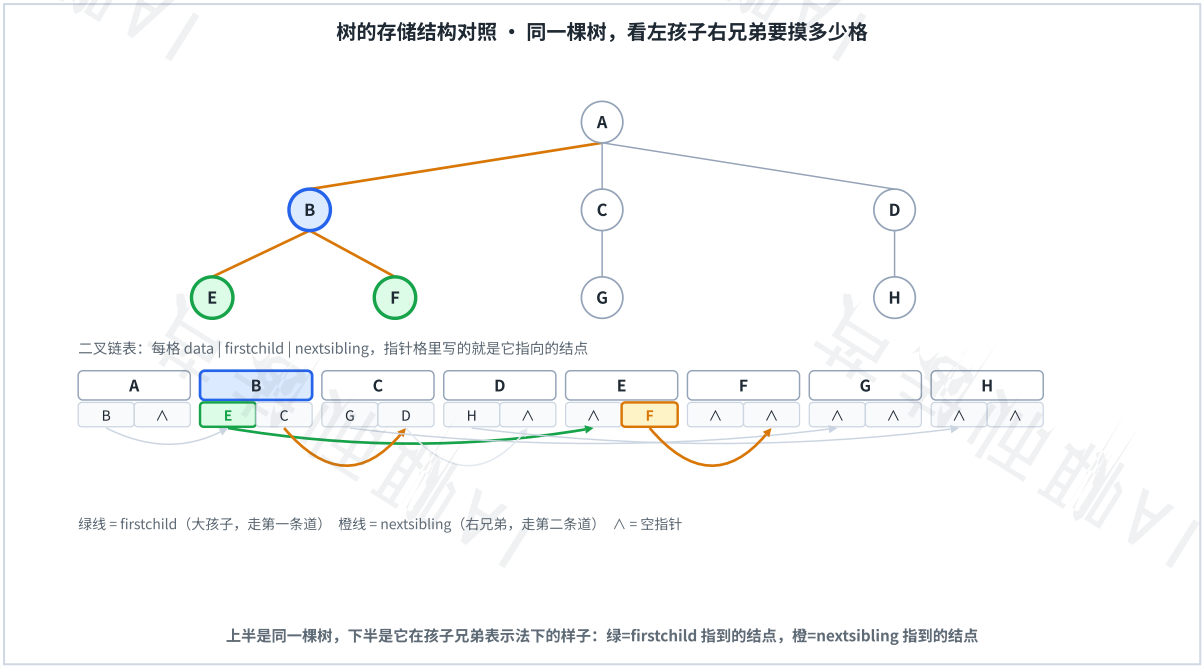


图 5-1 ★ 找孩子：B → E、F，代价只跟孩子个数有关，跟树的规模无关。

5.2 二叉树的定义、性质与顺序存储

本节配套动画：第 27 项《二叉树的顺序存储与主要特性》 · 408 大纲 四(二)2 顺序存储 + 四(一)3 二叉树性质（i ↔ 2i/2i+1、$n_0=n_2+1$、$\lfloor \log_2 n \rfloor+1$）

> **定义**
> 二叉树是每个结点至多两棵子树（**有左右之分，次序不能颠倒**）的树形结构。常用形态：满二叉树（每层都满）、完全二叉树（只有最下层可以不满且靠左连续）。

5.2.1 五条必须记住的性质

编号	性质
①	第 i 层上至多有 **$2^{(i-1)}$** 个结点（$i \geq 1$）
②	深度为 k 的二叉树至多有 **$2^k - 1$** 个结点
③	对任何二叉树，**叶子数 n_0 = 度为 2 的结点数 n_2 + 1**
④	具有 n 个结点的完全二叉树深度为 **$\lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n+1) \rceil$**
⑤	含 n 个结度的 m 叉树，其空域数为 $m(n+1) - n\cdot(m-1)$…（按分支数守恒推）

性质 ③ 的证明用两条守恒：分支数 = $n_1 + 2n_2$ = 结点数 − 1 = $n_0 + n_1 + n_2 - 1$，消去 n_1 即得 $n_0 = n_2 + 1$。

5.2.2 顺序存储：靠下标算父子

把结点按**完全二叉树的层序编号**存入一维数组（下标从 1 起），则

> **映射**
> 结点 i 的双亲 = `⌊i/2⌋`（i = 1 时为根）；左孩子 = `2i`；右孩子 = `2i + 1`；i 的左兄弟 = `i - 1`（i 为偶数时）。

三个关系都是**一次乘除法**，不需要任何指针。

代价：数组必须按“可能的最大编号”开。一棵 4 个结点的**右斜链**（每个结点只有右孩子），最大编号是 $2^3 = 8$……实际要开到 **2^h − 1 = 15** 格才放得下 4 个结点，**浪费 73%**。因此：

> **结论**
> 顺序存储**仅适用于完全二叉树或接近完全的树**。一般二叉树用链式：n 个结点、n + 1 个空链域。

5.2.3 链式存储与空链域

二叉链表每结点含 `data / lchild / rchild`。n 个结点共有 2n 个指针域，其中被用到的只有 n − 1 条边（除根外每个结点被一个指针指到），故

> **空链域**
> 空指针域数 = 2n − (n − 1) = **n + 1**。

这 n + 1 个空格正是线索二叉树要利用的对象（见 5.3）。要存双亲则用三叉链表（多一个 parent 域）。

典型例题

例 1 某二叉树有 10 个度为 2 的结点、5 个度为 1 的结点，求叶子数与总结点数。

解 $n_0 = n_2 + 1 =$ **11**；总数 n = 11 + 5 + 10 = **26**。

例 2 深度为 6 的完全二叉树共有 20 个结点，问有多少个叶子。

解 前 5 层满，共 $2^5 - 1 = 31 > 20$，说明深度不足 6——题设矛盾。改为“深度 5、20 个结点”：前 4 层 15 个，第 5 层 5 个叶子，第 4 层有 8 个结点、其中 ⌈5/2⌉ = 3 个有孩子（第 5 层的 5 个结点挂在最左 3 个父结点下，其中第一个父结点有两个孩子、第二三个各一个）→ 叶子 = (8 − 3) + 5 = **10**。

例 3 对含 n 个结点的完全二叉树用顺序存储，数组至少要多大？

解 最大编号就是 n，故开 **n + 1**（下标 0 空着）即可，无浪费——这正是“完全”的意义。

例 4 二叉树中结点 i（i > 1）的左兄弟存在吗？

解 i 为偶数时它是左孩子，没有左兄弟；i 为奇数时左兄弟是 `i - 1`。

易错点与考点提示

- 二叉树**不是**“度为 2 的树”：二叉树可以是每个结点至多 2 个孩子且**左右有序**，树则无序。

- 满二叉树一定是完全二叉树，反之不成立。
- $n_0 = n_2 + 1$ 对**任何**二叉树成立，不限于完全二叉树。
- 编号约定（从 0 还是从 1 起）会改变公式：从 0 起时双亲 = $\lfloor (i-1)/2 \rfloor$、左孩子 = $2i+1$。
- 顺序存储一般二叉树时公式"仍然对"，但会返回空格子——这是"浪费"而非"错误"。
- 空链域 $n+1$ 这个数在线索化、三叉链表等题目里反复出现，值得直接记住。

习题

1. 一棵有 257 个结点的二叉树，最大深度与最小深度各是多少？
2. 完全二叉树共 1000 个结点，求叶子数。
3. 用数组存下标从 0 起的完全二叉树，写出结点 i 的双亲、左右孩子公式。

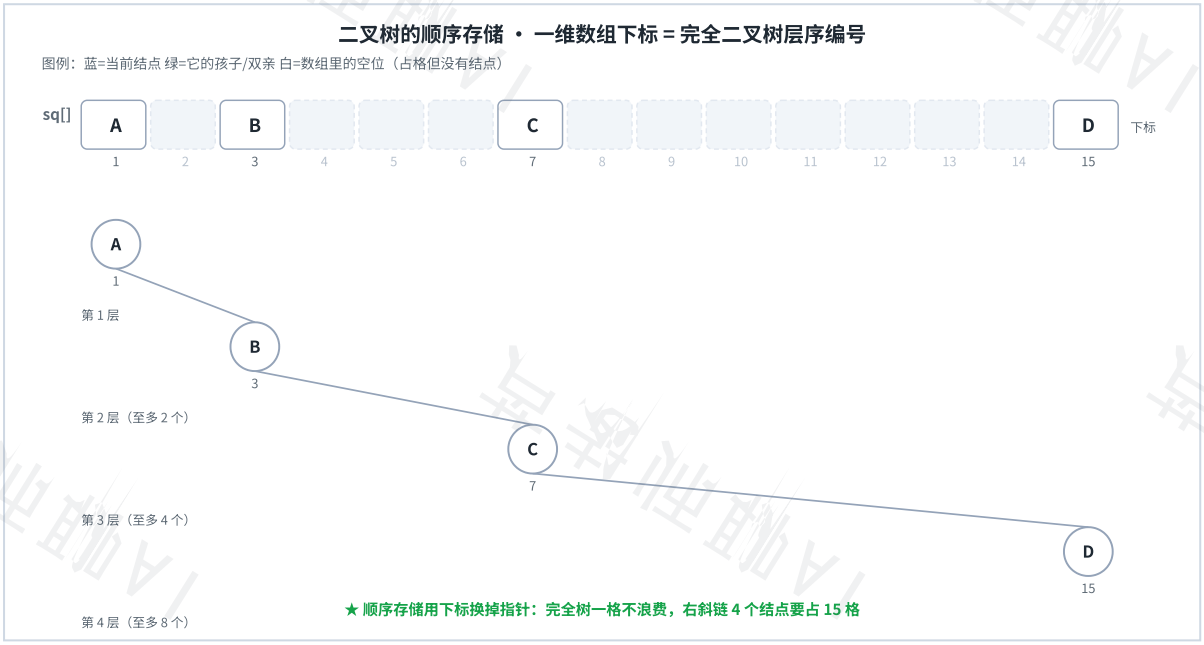


图 5-2 ★ 小结：顺序存储用"下标算关系"换掉了指针，代价是必须按完全二叉树编号。

5.3 二叉树的遍历与线索化

本节配套动画：第 28 项《二叉树的四种遍历》、第 29 项《中序线索二叉树》 · 教材 5.5 遍历二叉树（访问时机与递归栈）

> **核心**
> 三种遍历经过每个结点的路径**完全相同**，只是"访问"发生在第几次经过：先序在第 1 次、中序在第 2 次、后序在第 3 次。把这一点想清楚，递归栈的图就自然画得对。

5.3.1 四种遍历与实例

设二叉树由先序序列 `GDA##FE###MH##Z##`（# 表示空）唯一确定，其结构为：G 的左子树 D(A, F(E, ·))、右子树 M(·, H(·, Z))。四种遍历结果：

遍历	访问时机	序列
先序（根→左→右）	第 1 次经过	**G D A F E M H Z**
中序（左→根→右）	第 2 次经过	**A D E F G H M Z**
后序（左→右→根）	第 3 次经过	**A E F D H Z M G**
层序	逐层从左到右	**G D M A F H Z E**

先序逐步（递归栈变化）：访问 G（栈 G）→ 进 D（栈 G D）→ 进 A（栈 G D A）→ A 无子返回 → 进 F（栈 G D F）→ 进 E → 返回 → 进 M → 进 H → 进 Z，共 8 个结点，时间 O(n)、空间 O(树高)。

5.3.2 由两棵序列唯一确定二叉树

> **判据**
> 能唯一确定的组合：**先序 + 中序**、**后序 + 中序**、**层序 + 中序**。不能唯一确定：**先序 + 后序**、**先序 + 层序**（缺左右划分信息）。

方法：先序（或后序、层序）定**根**，中序用根把序列切成**左子树**与**右子树**两段，递归进行。

例：先序 GDAFEMHZ、中序 ADEFGHMZ → G 是根；中序里 G 左边 ADEF 是左子树、右边 HMZ 是右子树；先序中紧随 G 的 D 是左子树根，中序里 D 左边 A、右边 EF → 逐层还原，得 5.3.1 的那棵树。

5.3.3 线索二叉树：不用栈也能走中序

n 个结点有 **n + 1** 个空链域（本例 8 个结点 → 9 个空域）。把空链域利用起来：

> **线索化**
> 无左孩子 → `lchild` 改存**前驱**（ltag = 1）；无右孩子 → `rchild` 改存**后继**（rtag = 1）。tag = 0 表示仍是指向孩子的指针。

按中序线索化后，沿线索走一遍得到的序列 **A D E F G H M Z** 与递归中序完全一致，而**空间 O(1)**（不设栈）。

三个必记要点：

1. 线索化必须**按中序遍历的顺序**进行，且要记住“上一个访问的结点 pre”；
2. 求某结点的中序后继：若 rtag = 1，直接取 rchild；否则取其**右子树中最左**的结点；
3. 线索化是**破坏性**的——原来的空指针被占用后，若再改变树的形态，线索即失效，需要重新线索化。

典型例题

例 1 先序与中序相同的二叉树是什么形状？

解 每个结点都**没有左子树**（只有右链，含空树）。因为先序是“根左右”、中序是“左根右”，两者相同 ⟺ 左子树恒空。

例 2　设某二叉树有 20 个结点，其中度为 1 的结点 6 个，问线索化后有多少个线索（即被利用的空链域）？

解　空链域 = n + 1 = **21** 个。与度分布无关——这是恒等式，不必先求 n_0。

例 3　由中序与后序能否唯一确定二叉树？由中序与“层序”呢？

解　都能。后序/层序的第一个（或最后一个）结点可定根，中序据此分左右，递归即可。先序 + 后序不行：只有一个孩子时无法判断它是左还是右。

易错点与考点提示

- “遍历”的“访”指访问根结点，三种遍历的区别只在**时机**，不在路径。
- 线索化后 lchild / rchild 有两种语义，必须靠 tag 区分，判空条件也变了（不能再用 `p->lchild == NULL` 判“无左孩子”）。
- 中序线索树里，**整棵树的中序第一个结点**没有前驱线索（ltag = 1 且 lchild 为空），最后一个同理。
- 层序遍历用**队列**（不是栈），这是它与其他三种在实现上的根本区别。
- 由“先序 + 中序”求树时，若两序列里有重复元素则不能唯一确定——考题默认关键字互异。

习题

1. 已知中序 `DBEAFC`、先序 `ABDECF`，画出二叉树并写出后序。
2. 一棵 7 个结点的完全二叉树按层序编号 1~7，写出它的中序序列。
3. 说明为什么“后序 + 先序”不能唯一确定二叉树，举一个反例。

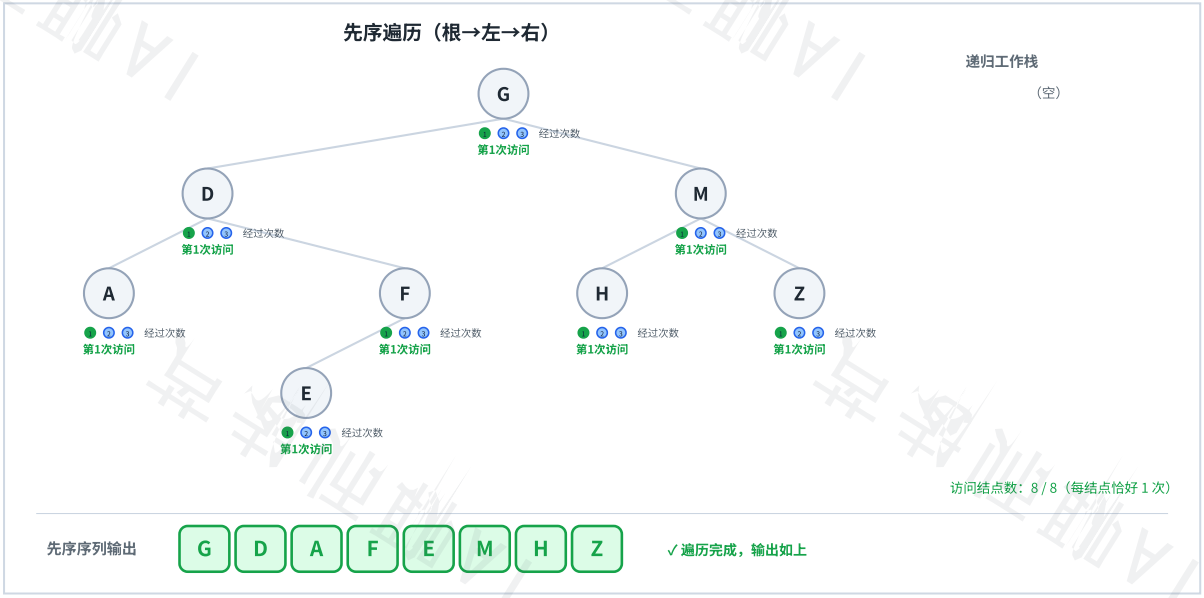


图 5-3　先序遍历（根→左→右）完成：G D A F E M H Z。

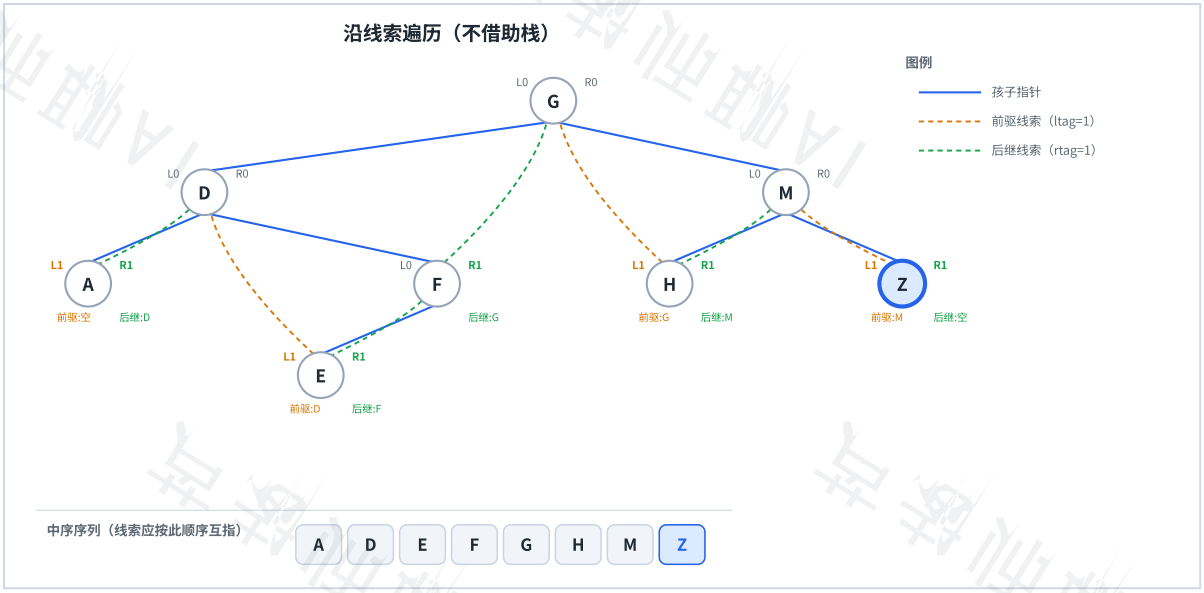


图 5-4 沿线索遍历完成：A D E F G H M Z——与递归中序结果一致，但空间 O(1)（不设栈/队列）。

5.4 树、森林与二叉树的转换

本节配套动画：第 30 项《树 / 森林与二叉树转换》 · 教材 5.6 树与森林（孩子兄弟表示、左孩子右兄弟）

> **口诀**
> 树转二叉树：**加线 → 抹线 → 旋转**，本质就是“左孩子右兄弟”——每个结点的第一个孩子变成它的左孩子，它的下一个兄弟变成它的右孩子。

5.4.1 三步转换

设树 T：A 有三个孩子 B、C、D；B 有 E、F；C 有 G；D 有 H。

1. **加线**：所有**相邻兄弟**之间加一条线（B—C、C—D、E—F）；
2. **抹线**：抹掉每个结点与其**第二个及以后**孩子之间的连线（只保留到第一个孩子的线）；
3. **旋转**：以树根为轴心，把图顺时针旋转约 45°，使“孩子线”成为左链、“兄弟线”成为右链。

得到的二叉树结构：A 的左孩子是 B；B 的右链 B → C → D；B 的左孩子 E，E 的右链 F；C 的左孩子 G；D 的左孩子 H。

5.4.2 两条必须记住的对应关系

> **验证**
> **二叉树的先序 == 树的先根遍历；二叉树的中序 == 树的后根遍历**。

本例中：二叉树先序 = **A B E F C G D H**（等于树 T 的先根序列）；二叉树中序 = **E F B G C H D A**（等于树 T 的后根序列）。

森林转二叉树：把每棵树各自转成二叉树，再把**各根的右链**串起来（第 2 棵、第 3 棵…依次成为第一棵根的右子树）。

5.4.3 由二叉树反推树 / 森林

逆规则：**根无右链**时对应一棵树；根有右链时对应一片森林，右链上每个结点是一棵新树的根。对每个结点，“沿左链的第一个结点”是它的孩子，“该孩子沿右链往下”都是它的孩子。

典型例题

例 1 已知某森林对应的二叉树有 12 个结点，其根的右链上有 3 个结点（含根共 4 个沿右链），问森林里有几棵树？

解 右链上每个结点对应一棵树的根，故 **4 棵**。

例 2 树 T 有 20 个结点、最大度为 4。写出 T 转二叉树后，“只有右孩子、没有左孩子”的结点有什么特征。

解 这样的结点在原树中**没有第一个孩子但有兄弟**——即它是某结点的第 2 个及以后的孩子、且自身不是任何结点的父亲（叶子兄弟）。

例 3 设二叉树先序为 A B D E C F，中序为 D B E A F C，还原二叉树并求它对应的树 / 森林。

解 根 A；左子树 {B, D, E}、右子树 {C, F}；B 有左 D、右 E；C 有右 F（中序里 F 在 C 之后）。由于 A 无右兄弟链延伸（A 是根且其右子树存在）——按森林规则，A 的右孩子 C 是**第二棵树的根**，故对应**含 2 棵树的森林**。

易错点与考点提示

- “左孩子右兄弟”不能记反：左 = 第一个孩子，右 = 下一个兄弟。
- 树的**先根 == 二叉树先序**、树的**后根 == 二叉树中序**；**没有**“树的中根”这种遍历，别去凑“树的中序 == 二叉树中序”。
- 森林中树的棵数 = 对应二叉树**根结点右链上的结点数**。
- 度为 m 的树转成二叉树后，结点的右链长度不超过 m − 1。
- 转换不改变结点个数与“父子 + 兄弟”信息，因此是**一一对应**（可逆）。

习题

1. 画出树 A(B(C,D,E), F(G), H) 对应的二叉树，并写出它的先序与中序。
2. 森林由三棵树 T1(3 结点)、T2(4 结点)、T3(5 结点) 组成，转成二叉树后根结点的右链上有几个结点？
3. 说明为什么“二叉树的中序”能充当“树的后根遍历”。

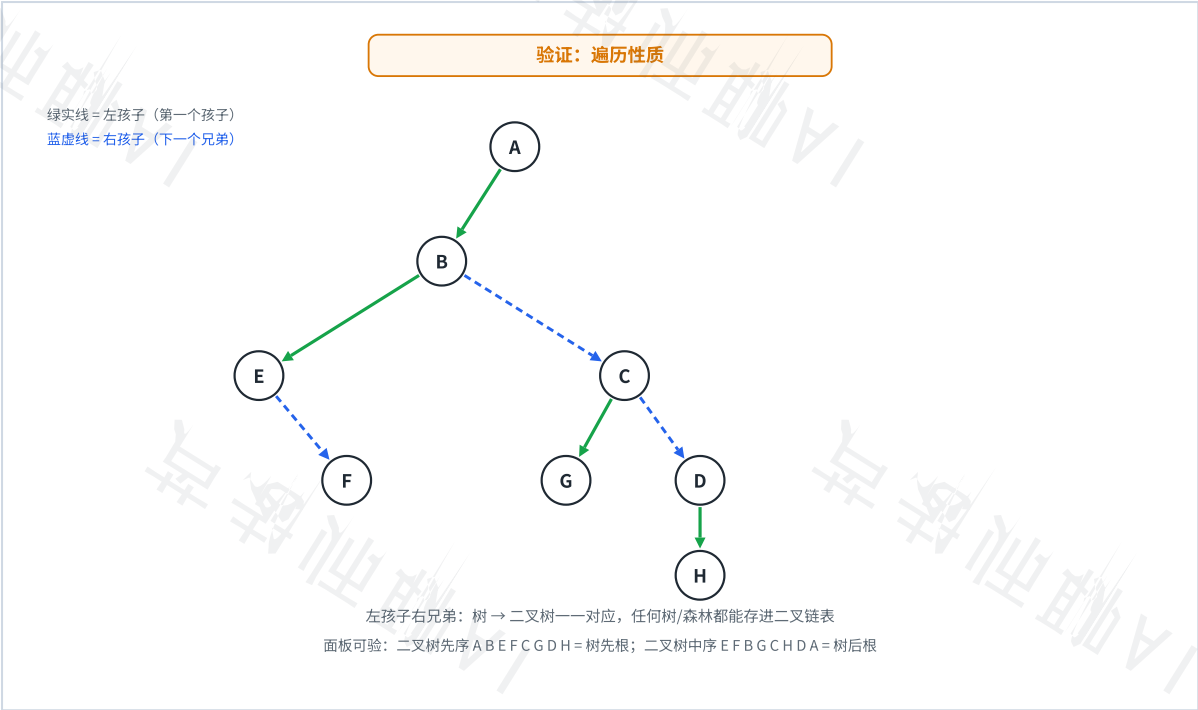


图 5-5 ★ 验证：二叉树先序 A B E F C G D H == 树的先根遍历；

5.5 哈夫曼树及其应用

本节配套动画：第 31 项《哈夫曼树构造与编码/译码》 · 教材 5.7 哈夫曼树及其应用（WPL、前缀编码）

> **定义**
> 结点的**路径长度**是从根到该结点所经分支（边）的条数；**树的路径长度**是根到每一结点的路径长度之和。给定 n 个权值 $\{w_1, \ldots, w_n\}$ 作为 n 个叶子，构造出的**带权路径长度 WPL** 最小的二叉树称为**哈夫曼树（最优二叉树）**。

5.5.1 构造算法

1. 把 n 个权值各视为一棵只有根结点的树，构成森林；
2. 选出根上权值**最小**的**两**棵树，合并成一棵新树，新根权值 = 两者之和；
3. 重复第 2 步 n − 1 次，直到只剩一棵树。

实例：w = {70, 50, 20, 40}（对应字符 A, B, C, D）。

次	Select 出的两棵	新根权	森林剩余权值
1	C(20)、D(40)	60	70, 50, 60
2	B(50)、60	110	70, 110
3	70、110	180	180（根）

WPL = 70 × 1 + 50 × 2 + 20 × 3 + 40 × 3 = 70 + 100 + 60 + 120 = 350。

> **快速算法**
> WPL 也等于**所有非叶结点权值之和**：60 + 110 + 180 = 350。两种算法互相校验，考场上用后者更快。

5.5.2 哈夫曼树的四条性质

1. **没有度为 1 的结点**（每次合并都产生两个孩子）；
2. n 个叶子构造出的哈夫曼树**总共 2n − 1 个结点**（本例 4 叶 → 7 个结点）；
3. 合并过程产生 n − 1 个新根（本例 3 次合并）；
4. 任一结点的权值不小于其父结点？——不成立；正确表述是**权值越大的叶子离根越近**，且**哈夫曼树不唯一，但 WPL 唯一**。

5.5.3 哈夫曼编码：前缀编码保证无歧义

约定左分支记 0、右分支记 1，则每个字符的编码就是从根到它的路径。因所有字符都在**叶子**上，任一字符的编码都不是另一字符编码的前缀——这就是**前缀编码**，译码时不必回看。

按 5.5.1 的树，从根到各叶子的分支串即为编码；**左支记 0、右支记 1** 与反过来记，会得到两套不同的编码表，但 WPL 与译码唯一性都不变。

译码实例：报文 `100110111` 从根开始逐位下走，走到叶子就输出该字符并回到根——依次译出 **B A C D**，恰好读完、无剩余位。前缀性质保证每一步“走到叶子”就是唯一的切分点，**不需要回看**。

> **性能**
> 用哈夫曼编码后，总码长恰为 WPL = 350（若用等长编码需 2 位 × (70+50+20+40) = 360 单位），节省约 2.8%——权值分布越不均匀，节省越多。

典型例题

例 1 给定权值 {1, 3, 5, 7, 11}，构造哈夫曼树并求 WPL。

解 合并 1+3 = 4；再 4+5 = 9；再 9+7 = 16；再 16+11 = 27。WPL = 4 + 9 + 16 + 27 = **56**。（叶深度：11 在 1 层，7 在 2 层，5 在 3 层，3 与 1 各在 4 层 → 11 + 14 + 15 + 12 + 4 = 56 √）

例 2 6 个字符的哈夫曼树共有多少个结点、多少次合并？

解 总结点 2n − 1 = **11**，合并 n − 1 = **5** 次；其中非叶结点 5 个、全部度为 2。

例 3 为什么哈夫曼编码必须把字符放在**叶子**上？

解 若某字符在内部结点，它的编码就是其后代字符编码的前缀，译码时会产生歧义。放在叶子即保证“任何编码互不为前缀”。

易错点与考点提示

- 每次必须选**两棵**权值最小的树，且新树的权值要**放回**森林参与下一轮比较——漏掉放回是最常见的构造错误。
- 权值相同的结点选谁没有唯一答案，**WPL 却唯一**；题目问“哈夫曼树是否唯一”答不唯一。
- 结点数 2n − 1、合并次数 n − 1、无度为 1 的结点，这三条要能秒答。
- 编码长度 = 叶子深度；WPL = Σ(权值 × 深度) = Σ(非叶结点权值)。

- 哈夫曼树**不是**二叉排序树，不能用于查找。
- 若题目要求“构造哈夫曼树并求带权路径长度”，注意 WPL 与“树的路径长度”（不计权）的区别。

习题

1. 权值 {2, 4, 5, 7, 8}，构造哈夫曼树、求 WPL 与编码长度分布。
2. 一棵哈夫曼树有 15 个结点，问有几个叶子、合并了几次。
3. 说明为什么“哈夫曼编码是最优前缀码”，并指出它对字符频率的要求。



图 5-6　译码完成：100110111 → BACD。

5.6　堆与优先队列

本节配套动画：第 32 项《堆与优先队列：建堆、出入队与 Top-K》 · 408 大纲 四(四)3 堆及其应用（筛选建堆 O(n)、优先队列 O(log n)、Top-K 用容量 K 的小顶堆）

> **定义**
> **堆**是一棵完全二叉树，且任一结点的关键字不小于（或不大于）其孩子——前者称**小顶堆**、后者称**大顶堆**。用数组存放，父子关系靠 `i ↔ 2i / 2i+1` 算出来，不需要指针。

5.6.1　建堆：自下而上筛

堆序只要求“爹 ≥ 儿”，兄弟之间不排序。建堆从**最后一个非叶结点** `⌊n/2⌋` 开始，倒着往前对每个结点做一次“筛选”（sift down）：把它与两个孩子中较大者比较，必要时交换，直到不再违反堆序。

实例：n = 8，初始序列 49 38 65 97 76 13 27 49。起点 i = ⌊8/2⌋ = **4**——下标比 4 大的全是叶子（孩子会超出 n），本来就直接满足堆序，不必动。

依次筛 4 号、3 号、2 号、1 号后得到：**97 76 65 49 49 13 27 38**，每个爹都 ≥ 自己的孩子，大顶堆成立。堆顶 97 就是最大值，但**第二名不知道在哪**——堆只保证父子有序，不保证全局有序。

代价

建堆的总代价是 **O(n)**，不是 O(n log n)。因为大部分结点在底层、筛选路径很短：Σ(第 i 层结点数 × 可下沉深度) = O(n)。

5.6.2 出入队与堆排序

操作	做法	复杂度
取顶	直接读 `H[1]`	O(1)
删除堆顶	用**最后一个**元素顶替堆顶，再做一次筛选	O(log n)
插入	追加到末尾，向上交换（sift up）	O(log n)
建堆	自 ⌊n/2⌋ 往前筛	**O(n)**

堆排序 = 建堆 + n − 1 次“换堆顶到末尾、堆缩小一层再筛”，故总 O(n log n)、空间 O(1)、**不稳定**。

5.6.3 优先队列与 Top-K

优先队列的四种实现对照：

实现	入队	出队（取最值）

无序数组 | O(1) | O(n) |

有序数组 | O(n) | O(1) |

堆 | O(log n) | O(log n) |

二叉排序树 | 平均 O(log n) | 平均 O(log n)，最坏 O(n) |

Top-K 问题的标准做法：求 n 个数中最大的 K 个，维护一个**容量为 K 的小顶堆**——新元素比堆顶大才替换堆顶再下沉。空间 O(K)，时间 O(n log K)。用大顶堆是错的：大顶堆堆顶是当前的最大值，无法快速淘汰“最小的那个候选”。

典型例题

例 1 判断序列 97 76 65 49 49 13 27 38 是否为堆。

解 是（大顶堆）。逐格验：97 ≥ 76, 65；76 ≥ 49, 49；65 ≥ 13, 27；49 ≥ 38。全部成立。

例 2 序列 13 27 49 38 21 76 65 是堆吗？

解 不是。逐格验：13 的孩子 27、49 满足；但 27 的孩子是 38 与 **21**，21 < 27 违反小顶堆的“爹 ≤ 儿”。把 21 与 27 交换后（13 21 49 38 27 76 65）再逐格验，全部成立，才是小顶堆。判堆只能逐格验，不能凭“看起来乱”下结论。

例 3 从大顶堆中删除堆顶后，为什么用最后一个元素去顶替、而不是把整堆前移？

解 堆是完全二叉树，删除堆顶后必须保持“层序连续”。用最后一个结点顶到根部，树形仍是完全的，只需一次 O(log n) 筛选；若整体前移会破坏数组与父子下标的对应关系，且代价 O(n)。

易错点与考点提示

- 建堆 **O(n)**、单次筛选 O(log n)——把建堆写成 O(n log n) 是常见扣分点。
- 堆顶是**唯一**确定的最值，第二名不确定；要有序必须做堆排序。
- 堆是**完全二叉树**，不是一般的二叉排序树；中序遍历堆没有任何有序性可言。
- Top-K 求最大 K 个用**小顶堆**；求最小 K 个用大顶堆。
- 堆排序不稳定：相等元素可能在筛选过程中跨越。
- 下标从 0 起时父子公式变为 `2i+1 / 2i+2`，做题先确认约定。

习题

1. 对序列 49 38 65 97 76 13 27 建**小顶堆**，写出逐结点筛选的过程与最终序列。
2. 说明为什么“建堆 O(n)”，用级数求和给出证明思路。
3. 数据流式地求“当前最大的 100 个数”，内存只够放 100 个，给出算法与复杂度。

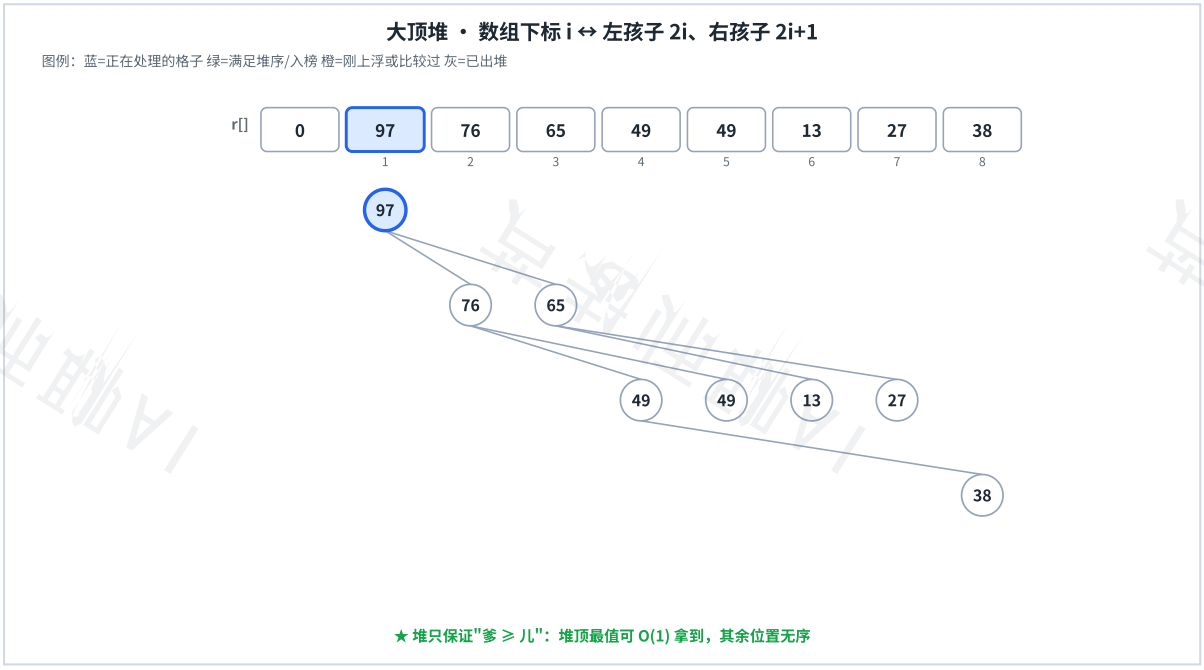


图 5-7 ★ 建堆完成：97 76 65 49 49 13 27 38，每个爹都 ≥ 自己的两个孩子 → 大顶堆成立（已逐格校验）。

5.7 并查集

本节配套动画：第 33 项《并查集：双亲表示与路径压缩》 · 408 大纲 四(四)2 并查集及其应用（双亲数组、按大小合并、路径压缩）

> **定义**
> 并查集是一种处理**不相交集合合并与查询**的数据结构：支持 `Find(x)`（求 x 所在集合的代表元）与 `Union(x, y)`（合并两个集合）。它用**森林的数组表示**——每个数组元素存父结点下标，根的 parent 记为负数。

5.7.1 双亲数组表示

初始化时令 `parent[i] = -1`：既是“我是根”的标志，又（另一种约定下）记录集合大小为 1。于是 8 个结点各自成集合、共 8 棵树、树高 0，**整片森林藏在一个数组里**。

操作	实现	代价
Find(x)	沿 parent 链向上爬，直到 parent < 0	**O(树高)**
Union(x, y)	各自找根，再把一棵树的根挂到另一棵的根下	O(1)（不含 Find）

5.7.2 两个优化：按大小合并 + 路径压缩

① 按大小（或按秩）合并：把**小树**挂到大树下，可保证树高不超过 $\lfloor \log_2 n \rfloor$。若总是大树挂到小树上，链式合并会退化成一条链。

② 路径压缩：Find 途中把经过的结点直接改挂到根上，一次遍历后这条链被“拍平”。

实例：依次合并 1-0、2-1、3-2、4-3、5-4、6-5、7-6（即 8 个结点连成一条链）。

量	值
合并后集合数	1 个（8 个元素）
树高峰值	**7**（退化成链）
Find 累计探测次数	**22** 次
对 v0 做一次带压缩的 Find 后	树高由 7 → **1**，之后再查同一条链只要 1 步

最终 v0 所在集合的根是 v7，其 parent = **−8**（表示该集合有 8 个元素）。

> **复杂度**
> 两种优化合用后，m 次操作的总代价是 $O(m \cdot \alpha(n))$，α 是反阿克曼函数，实际中不超过 4，可视为**近似常数**。

5.7.3 为什么 Kruskal 必须用它

Kruskal 每考察一条边都要问“两端是否已连通”。若每次都用 DFS/BFS 判连通，代价 O(n)，整体退化到 $O(n \cdot e)$；用并查集则每次判连通近似 O(1)，瓶颈只剩排序的 $O(e \log e)$。同理，判断无向图连通分量个数、求迷宫中两点是否同区，都可以用并查集离线处理。

典型例题

例 1 执行序列 Union(0,1)、Union(2,3)、Union(1,3)、Union(4,2) 后，Find(0) 与 Find(4) 是否相同？

解 相同。四次合并把 {0,1,2,3,4} 并成一个集合；只剩 5 独立。故共 **2 个**集合。

例 2 用 parent 数组（根存负的大小值）写出 5 个结点依次 Union(0,1)、Union(1,2) 后的数组。

解 初始 [−1,−1,−1,−1,−1]。Union(0,1)：按大小合并，令 parent[1] = 0，parent[0] = −2 → [−2, 0, −1, −1, −1]。Union(1,2)：Find(1) = 0、Find(2) = 2，把 2 挂到 0 → [−3, 0, 0, −1, −1]。

例 3 为什么“路径压缩后树高变 1”不会破坏并查集的正确性？

解 压缩只改变**指向**（把链上结点直接挂到根），不改变“谁和谁同属一个集合”这一等价关系，Find 结果不变。

易错点与考点提示

- 根的标记有两种约定：parent[i] = i（自己指自己）或 parent[i] = 负数（存集合大小）。做题先确认，否则“找根”条件写错。
- Union 必须**先 Find 再挂根**；直接把 x 挂到 y 上会把两棵子树错接。
- 不做按大小合并时，链式输入会让树高达到 n − 1，Find 退化 O(n)。
- 路径压缩后树高变 1 是“这一次 Find 之后”的效果，别以为合并完就是 1。
- 并查集**不支持拆分**集合（没有高效的 Delete），需要可回滚时要另设结构。

习题

1. 说明“parent[i] < 0 即为根”这一约定下，如何求集合元素个数。
2. 依次执行 Union(a,b) 形式的链式合并（8 个结点、7 次），不做任何优化时树高是多少？做按大小合并后呢？
3. Kruskal 中若不用并查集而用 visited 数组判环，复杂度会变成什么？



图 5-8 ★ 完成：8 个结点归并为 1 个集合，最大集合 8 个元素；

第 6 章　图

6.1　图的定义与术语

本节配套动画：第 34 项《图的基本概念：度、握手定理与连通分量》 · 408 大纲 五(一) 图的定义与术语（度/入度出度、握手定理、完全图、连通与强连通分量）

> **定义**
> 图 G 记作 G = (V, E)，其中 V 是非空的顶点集合，E 是边的集合。若 E 中的元素是**无序对** (v, w)，G 称为**无向图**；若是**有序对** <v, w>（v 为弧尾、w 为弧头），G 称为**有向图**（网）。边（弧）可以附权值，也可不附。

6.1.1　几个必须分清的基本量

概念	无向图	有向图
完全图	n(n − 1) / 2 条边	n(n − 1) 条弧
顶点的度	TD(v) = 与 v 相关联的边数	TD(v) = **ID(v) + OD(v)**
度之和	ΣTD = 2 | E |	ΣTD = 2 | E |（入度之和 = 出度之和 = | E |）
连通	任意两点间有路径	任意两点间**互有方向**的路径称强连通
极大连通子图	连通分量	强连通分量

两个术语容易混淆：**极大连通子图**（连通分量）中的“极大”指不能再往里加顶点，是**子图个数最少**意义上的大；而生成连通子图要求“包含全部顶点”，二者不是一回事。

6.1.2　握手定理及其推论

> **定理**
> 任意图中，所有顶点的度之和等于边数的两倍：Σ TD(v) = 2 \| E \|。

因为每条边同时给它的**两个**端点各记一个度。有向图中另有：Σ ID(v) = Σ OD(v) = \| E \|。

> **推论**
> 度为奇数的顶点必有**偶数个**。

证明思路：若奇度顶点有奇数个，则度之和为奇数，与 2 \| E \| 为偶数矛盾。

6.1.3　手工演算示范

设无向图 G 有 7 个顶点、8 条边：(0,1)、(0,2)、(1,2)、(1,3)、(2,3)、(4,5)、(5,6)、(4,6)。

各顶点的度依次为：TD(0) = 2，TD(1) = 3，TD(2) = 3，TD(3) = 2，TD(4) = 2，TD(5) = 2，TD(6) = 2。

- 度之和 = 2 + 3 + 3 + 2 + 2 + 2 + 2 = **16** = 2 × 8 = 2|E|，握手定理成立；
- 奇度顶点是 v1、v2，共 **2 个**（偶数个），推论成立；
- 该图有**两个连通分量**：{v0, v1, v2, v3} 与 {v4, v5, v6}，故 G 本身不连通。

对照完全图：7 个顶点的无向完全图 K7 应有 $n(n-1)/2 = \mathbf{21}$ 条边，每个顶点的度都是 $n-1=6$；本图只有 8 条，差 13 条。

典型例题

例 1 已知无向图有 8 条边，其中 4 个顶点的度都是 3，其余顶点的度都是 2，问共有几个顶点。

解 设其余顶点 x 个。由握手定理：$4\times3+2x=2\times8=16$，得 $x=2$，共 **6 个顶点**。

例 2 一个有向图有 6 个顶点、10 条弧，能否确定各顶点入度之和与出度之和？

解 两者都等于弧数，即 $\Sigma ID=\Sigma OD=\mathbf{10}$。注意不能据此确定每个顶点的入度，只能确定总和。

例 3 度序列 (5, 4, 3, 2, 2, 1) 能否构成无向简单图？

解 不能。度之和 = 5 + 4 + 3 + 2 + 2 + 1 = 17 为奇数，违反握手定理（无向图的度之和必为偶数）。

例 4 上节那张 7 顶点的图，若改成有向图并仍取“完全”，需要多少条弧？

解 有向完全图的弧数为 $n(n-1)=7\times6=\mathbf{42}$ 条，是无向情形的两倍——因为每对顶点之间要各存一条方向的弧。

易错点与考点提示

- 无向完全图的边数是 $n(n-1)/2$，有向完全图的弧数是 $n(n-1)$，两者差一倍，审题先看有向还是无向。
- 度为奇数的顶点必是偶数个——这是判断题的常设陷阱，也可用来快速排除不可能的度序列。
- 连通分量针对**无向图**，强连通分量针对**有向图**；有向图上只说“连通分量”通常指其**忽略方向后的底图**连通分量，答题时要写明。
- “任意两点间有路径”的有向图叫**强连通**，仅“底图连通”不算。
- 弱连通（底图连通）与强连通的区分在有向图上必考，别把两者混用。

习题

1. 无向简单图 G 有 15 条边，度数分别为 2、4、6 的顶点各 2 个，其余顶点度数均为 3，求 G 的顶点数。
2. 写出上节示例图中 v1 与 v2 的邻接点，并说明为什么把 (1,2) 这条边数两次是错的。
3. 证明：在任何 n（$n\ge2$）个顶点的无向简单图中，必有两个顶点的度相同。
4. 一个有向图各顶点的入度序列为 (0, 1, 2, 2, 1)，出度序列为 (2, 2, 1, 0, 1)，它有多少条弧？是否可能有自环？
5. n 个顶点的无向连通图至少有多少条边？n 个顶点的强连通有向图至少有多少条弧？

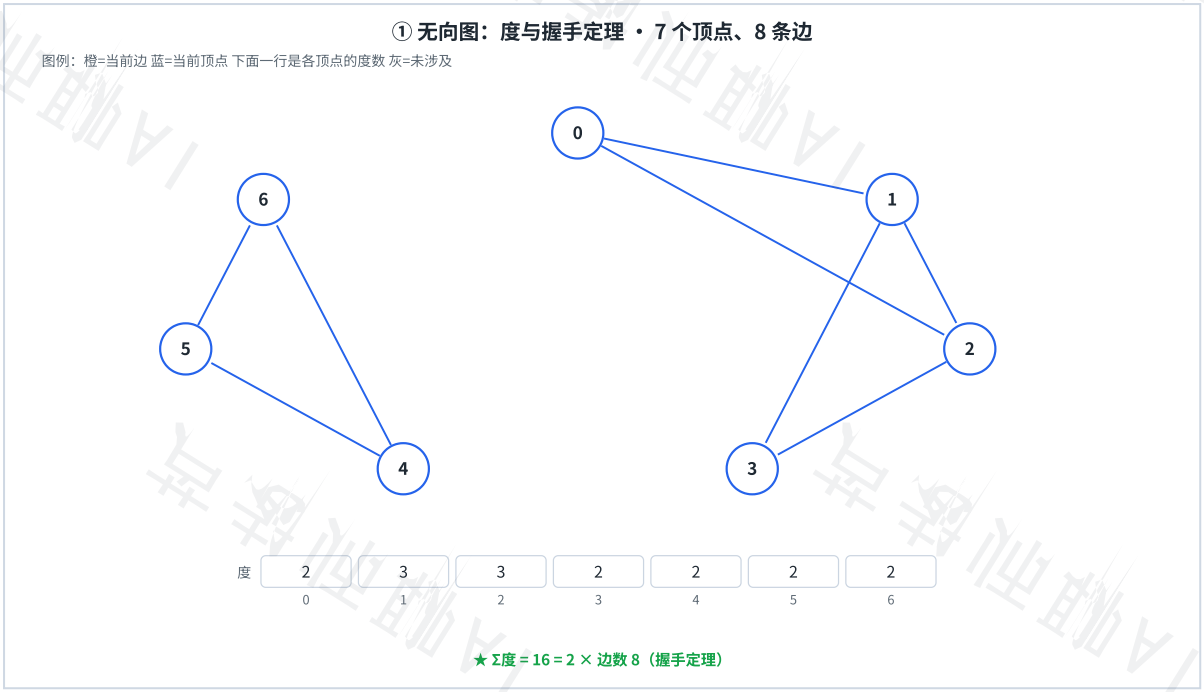


图 6-1　握手定理的逐点核对：7 个顶点的度之和 16 = 2 × 8 条边，奇度顶点 2 个

6.2　图的存储结构

本节配套动画：第 35 项《邻接多重表与十字链表》 · 408 大纲 五(二)3 邻接多重表、十字链表（一条边/弧只存一个结点，被两条链共享）

> **要点**
> 图的存储要同时表达两件事：**顶点信息**和**边（弧）信息**。由于顶点之间的关系是多对多的，图没有“顺序存放就能自然表达关系”的好事，必须显式记录邻接关系——这就是邻接矩阵与邻接表两条路线的由来。

6.2.1　邻接矩阵

用两个数组表示图：`vexs[n]` 存顶点信息，`arc[n][n]` 存边信息。

项目	无向图	有向图	带权图（网）
元素取值	有边记 1，否则 0	<i, j> 存在记 1	有边记权值，无边记 ∞ 或 0
矩阵形状	**对称**（可压缩存下三角）	一般不对称	一般不对称
求 TD(i)	第 i 行（或列）中 1 的个数	行 = 出度，列 = 入度	同左
判边存在	O(1)	O(1)	O(1)
空间	$O(n^2)$	$O(n^2)$	$O(n^2)$
遍历复杂度	$O(n^2)$	$O(n^2)$	$O(n^2)$

邻接矩阵的优点是判断任意两点是否相邻是常数时间、直观；缺点是**空间与边数无关**，稀疏图上浪费严重，且统计边数需要扫全矩阵。

6.2.2　邻接表与逆邻接表

为每个顶点建一条单链表，链上挂该顶点的**邻接边结点**，表头指针存在顶点数组里。

- **邻接表**（有向图存**出边**）：第 i 条链的长度即 OD(i)；求 ID(i) 必须遍历整个表结构，代价 O(n + e)。
- **逆邻接表**（存**入边**）：与邻接表正好相反，求入度 O(1)、求出度 O(n + e)。
- 空间：有向图 O(n + e)，无向图 O(n + 2e)。
- 同一张图的邻接表**不唯一**——它依赖每条链上邻接点的接入次序。头插法会让链上次序与输入次序相反，这一点直接影响遍历序列（见 6.3）。

6.2.3 十字链表与邻接多重表

邻接表在有向图上的痛点是：求出边方便、求入边要扫全图。两种改进分别针对有向图与无向图。

> **十字链表**
> 适用于**有向图**。每个顶点一个表头结点，含 `firstout`（第一条出弧）与 `firstin`（第一条入弧）两个指针；每条弧只建**一个**弧结点，含 `tailvex`、`headvex`、`hlink`、`tlink` 四个域。同一个弧结点同时挂在弧尾顶点的出边链和弧头顶点的入边链上——**一条弧、两个链共享一个结点**。

> **邻接多重表**
> 适用于**无向图**。每条边一个结点，含 `ivex`、`jvex` 及 `ilink`、`jlink` 两个链指针，分别挂进两个端点的边链。这样一条边只存一次，删边时不必像邻接表那样去两条链里各删一个结点。

实例演算。设有向图 4 个顶点、6 条弧：<0,1>、<0,2>、<1,2>、<2,0>、<2,3>、<1,3>。

- 弧结点数 = **6**（不随出入度翻倍），指针总数 = 6 × 2 = **12**；
- 看 v2 这一行：出边链为 <2,3> → <2,0>（出度 2），入边链为 <1,2> → <0,2>（入度 2），TD(2) = 4；
- 求 v2 的入边只需沿 `firstin` 走 2 步，即 **O(入度)**，这是邻接表做不到的。

6.2.4 四种存储结构的选型

存储结构	适用	主要优点	主要代价
邻接矩阵	稠密图、需频繁判边	判边 O(1)，实现简单	空间 $O(n^2)$，稀疏图浪费
邻接表	稀疏图	空间 O(n + e)，便于遍历邻接点	判边 O(度)，求入度需扫全图
十字链表	有向图，需频繁查入边与出边	求出入度都是 O(度)	每条弧 2 个指针，结构复杂
邻接多重表	无向图，需频繁增删边	一条边只存一个结点	每边 2 个链指针，实现繁琐

典型例题

例 1 某有向图的邻接表如下（链头在左，按表头插入次序画出）：v0 → v2 → v1，v1 → v2，v2 → v3，v3 → v0。求各顶点的入度与出度。

解 出度即链长：OD(v0) = 2，OD(v1) = 1，OD(v2) = 1，OD(v3) = 1。入度需统计各链中出现次数：v0 被 v3 指、ID = 1；v1 被 v0 指、ID = 1；v2 被 v0、v1 指、ID = 2；v3 被 v2 指、ID = 1。合计 ΣOD =

ΣID = 5 条弧，可作为自检。

例 2 用 n 个顶点、e 条弧的有向图，证明“邻接表中所有弧结点数 = e，而邻接多重表中边结点数 = e（无向图）”，并说明无向图若用邻接表为何是 2e 个结点。

解 有向图每条弧只属于一个起点，邻接表中出现一次，故共 e 个结点。无向图的边 (v, w) 既是 v 的邻接点也是 w 的，邻接表要在两条链上**各存一个**结点，故共 2e 个；邻接多重表让一条边只建一个结点、用两个链指针同时挂进两条链，所以是 e 个——这正是它存在的理由。

例 3 上节 6 条弧的例子中，若改用邻接多重表存储无向图（把弧看成边），需要多少个边结点、多少个指针？

解 6 条边、每边 2 个链指针，共 6 个边结点、12 个指针，数量与十字链表相同，但**语义不同**：十字链表的两个指针分别表示“尾点的出链下一弧”与“头点的入链下一弧”，邻接多重表的两个指针则各自属于两个端点的边链，无方向之分。

易错点与考点提示

- 邻接矩阵下无向图第 i 行的 1 个数是 TD(i)；**有向图**行是出度、列是入度，别记反。
- 无向图的邻接矩阵对称，可用三角压缩；但压缩之后“判边”与“求度”都要重新推下标，代价换空间。
- 邻接表**不唯一**（与链上次序有关），所以由邻接表写出的遍历序列也不唯一——考题若要求写出序列，必须同时说明邻接点的访问次序。
- 十字链表用于有向图、邻接多重表用于无向图，这是选择题高频考点；判据是“结点被几条链共享、有没有方向”。
- 若图中同时存在 <a,b> 与 <b,a>，在十字链表里是**两个**弧结点，不能当成一条边。
- 判断“哪种存储更省空间”要看图的稠密程度：e 接近 n^2 时邻接矩阵反而不吃亏（无指针开销）。

习题

1. 画出 6.2.3 例中那 6 条弧的十字链表（标出 v2 的出边链与入边链）。
2. 一个有向图用邻接矩阵存储，第 3 行有 4 个 1、第 3 列有 2 个 1，问 v3 的出度、入度与度。
3. n 个顶点的无向图用邻接表存储，共有 2e 个边结点。若采用邻接多重表，边结点数与指针总数各是多少？
4. 说明为什么“求任意两点之间是否有边”这一操作上，邻接矩阵优于邻接表，并给出邻接表的对应复杂度。
5. 已知某图的存储结构占空间 O(n + 2e)，且无法在 O(1) 时间内判断 (v, w) 是否为边，它最可能是什么结构？



图 6-2 十字链表里 v2 这一行：出边链 <2,3> → <2,0>、入边链 <1,2> → <0,2>，两个数直接读链长

6.3 图的遍历

本节配套动画：第 36 项《图的 DFS 与 BFS 遍历》 · 教材 6.5 图的遍历（DFS/BFS、存储结构的影响）

> **问题**
> 从某个顶点出发，按某种顺序**每个顶点恰好访问一次**。与树类似，图也需要一套系统化的搜索方法；但图中有环，因此必须记录“已访问”，否则会绕回去无限循环。

6.3.1 两个必须准备的数据结构

- `visited[n]`：初值全 false，访问过即置 true。这是图遍历区别于树遍历的关键——**树没有环，图有**。
- 辅助结构：DFS 用**递归栈**（或显式栈），BFS 用**队列**。

若一次遍历后仍有顶点 visited 为 false，说明图不连通，须从未访问顶点再出发。因此：

> **结论**
> 对整个图完成遍历所需的**启动次数 = 连通分量（有向图为强连通分量视题意而定）的个数**。

6.3.2 深度优先搜索 DFS

从 v 出发：访问 v 并置 visited；然后依次检查 v 的**未访问**邻接点 w，对每个 w 递归调用 DFS(w)；v 的邻接点全部检查完后**回溯**到上一层。

它等价于树的**前序**遍历，走到的顶点与所经过的边构成**深度优先生成树**（对连通图而言）。

6.3.3 广度优先搜索 BFS

从 v 出发：访问 v 并入队；每次出队一个顶点，把它**所有未访问**的邻接点依次访问并入队。

它等价于树的**层序**遍历，构成**广度优先生成树**。因顶点是按层出队的，首次到达某顶点的路径必是**边数最少**的路径，所以 BFS 可直接用于**无权图的最短路径**。

6.3.4 遍历序列不唯一：一个完整的对照

设无向图有 6 个顶点，边集为 (1,2)、(1,3)、(2,5)、(3,5)、(4,5)、(4,6)，从 **v2** 出发。同一张图、四种情形给出四种序列：

存储结构与算法	访问序列	说明
DFS + 邻接矩阵	2 → 1 → 3 → 5 → 4 → 6	邻接点按编号**升序**扫描
BFS + 邻接矩阵	2 → 1 → 5 → 3 → 4 → 6	先访问 v2 的两个邻接点 1、5，再逐层展开
DFS + 邻接表	2 → 5 → 4 → 6 → 3 → 1	表头插入使链上次序与输入次序相反
BFS + 邻接表	2 → 5 → 1 → 4 → 3 → 6	同上，出队次序随之改变

> **要点**
> 邻接矩阵把邻接点次序固定为编号升序，因此**遍历序列与生成树都唯一**；邻接表因链上次序依赖建表次序，序列**不唯一**，但**生成树的边数恒为 n − 1**（连通图），且“访问到的顶点集合”恒相同。

DFS 从 v2 出发（邻接矩阵）的递归栈变化，是考试爱考的中间过程：

步	动作	栈（底 → 顶）	已访问
1	访问 v2	DFS(v2)	2
2	v2 的邻接点 v1 未访问 → 递归	DFS(v2) → DFS(v1)	2, 1
3	v1 的邻接点 v2 已访问，跳过；v3 未访问 → 递归	… → DFS(v3)	2, 1, 3
4	v3 的 v1 已访问；v5 未访问 → 递归	… → DFS(v5)	2, 1, 3, 5
5	v5 的 v2、v3 已访问；v4 未访问 → 递归	… → DFS(v4)	2, 1, 3, 5, 4
6	v4 的 v5 已访问；v6 未访问 → 递归	… → DFS(v6)	2, 1, 3, 5, 4, 6
7	v6 无未访问邻接点 → 逐层回溯，栈空	（空）	全部 6 个

生成的深度优先生成树边集：(2,1)、(1,3)、(3,5)、(5,4)、(4,6)。

6.3.5 复杂度

存储结构	DFS	BFS
邻接矩阵	$O(n^2)$	$O(n^2)$
邻接表	O(n + e)	O(n + e)

原因：邻接矩阵下每个顶点都要扫完一整行（n 列）；邻接表下每条边至多被检查两次（无向图）。空间上，DFS 的递归栈最深 O(n)，BFS 的队列最宽 O(n)。

典型例题

例 1 用上图的邻接矩阵分别写出从 v4 出发的 DFS 与 BFS 序列。

解 v4 的邻接点为 v5、v6。DFS：4 → 5 → 2 → 1 → 3 → 6；BFS：4 → 5 → 6 → 2 → 3 → 1。（DFS 沿 v5 走到 v2、再 v1、再 v3；回溯后才轮到 v6。）

例 2 某无向图有 6 个顶点，对其执行 DFS 时启动了 3 次，能得出什么结论？

解 该图有 **3 个连通分量**。启动次数等于连通分量个数。

例 3 证明：BFS 得到的生成树中，根到任一顶点的路径长度（边数）在原图中最短。

解 反证。设 BFS 树中根到 v 的路径长为 d，而原图存在更短的、边数 < d 的路径 P。P 上必存在第一个不属于前 d − 1 层的顶点 u，它有一个邻接点在第 d − 2 层或更浅处，故 u 会在不晚于第 d − 1 层时被访问，与 u 的深度矛盾。

例 4 DFS 与 BFS 都能求无权图的最短路径吗？

解 不能。**只有 BFS** 保证首次到达即边数最少；DFS 可能先沿一条很长的链走到终点。带权图两者都不能直接用，须用 Dijkstra（见 6.5）。

易错点与考点提示

- 遍历必须配 `visited[]`；漏掉它，在有环图上会死循环——这是写算法时最常见的错。
- 邻接矩阵下序列唯一（约定按编号升序），邻接表下序列不唯一；考题若未说明建表次序，答案应注明“取决于邻接点次序”。
- 遍历次数 = 连通分量个数，反过来也成立，可用来数连通分量。
- DFS 适合判连通性、找环、拓扑排序与回溯搜索；BFS 适合无权最短路、最少步数类问题。
- 有向图的遍历仍用同样两个算法，但“从一个顶点出发访问不到的顶点”意味着不可达，不意味着图不连通（可能是底图连通而非强连通）。
- 空间复杂度别写成 O(n + e)：邻接矩阵下 DFS 递归栈最坏 O(n)，与存储无关。

习题

1. 对 6.3.4 的图，用邻接矩阵写出从 v1 出发的 DFS 与 BFS 序列，并各画出其生成树边集。
2. 上图中把边 (4,6) 删去，从 v2 出发需要启动几次遍历？图有几个连通分量？
3. 说明为什么邻接表存储下 BFS 的复杂度是 O(n + e)，而邻接矩阵下是 $O(n^2)$。
4. 有向图 <1,2>、<2,3>、<3,1>、<1,4>，从 v1 出发做 DFS（邻接矩阵、编号升序），写出序列；该图有几个强连通分量？
5. 判断并说明理由：任何连通图，其深度优先生成树与广度优先生成树一定不同。

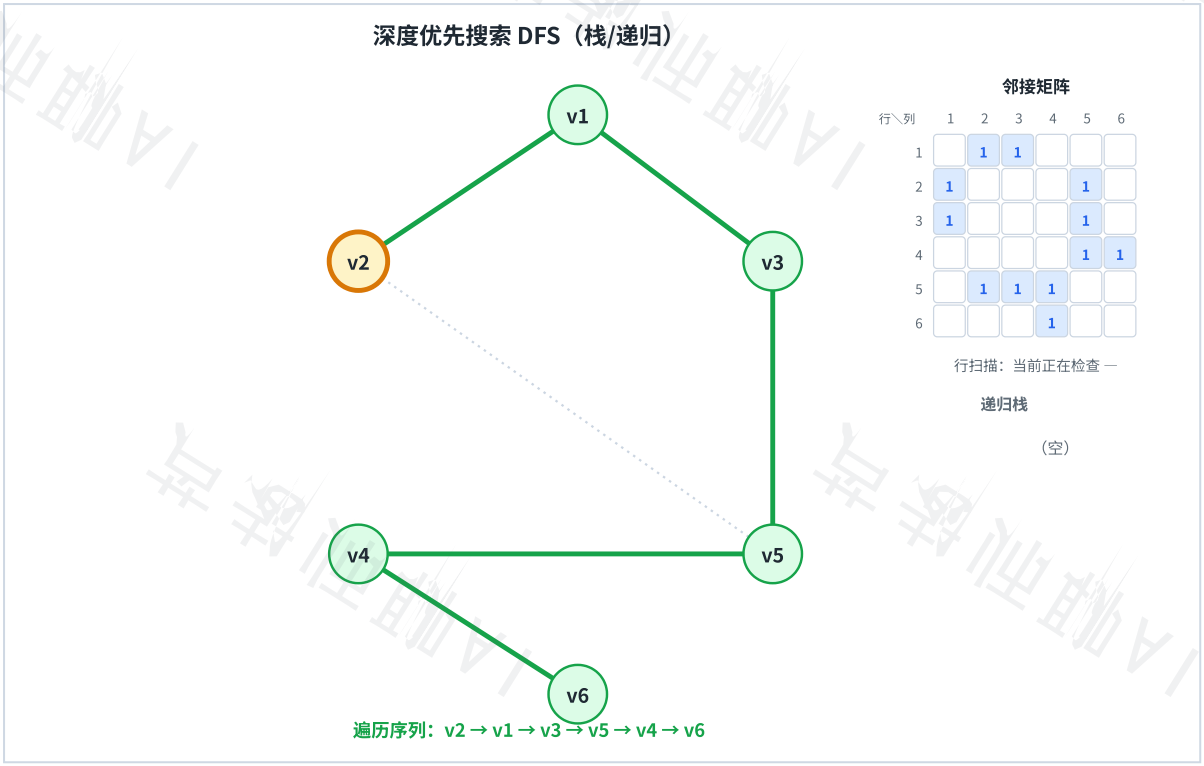


图 6-3　从 v2 出发的 DFS 完成态：序列 2 → 1 → 3 → 5 → 4 → 6，递归栈已逐层清空

6.4　最小生成树

本节配套动画：第 37 项《最小生成树：Prim / Kruskal》　·　教材 6.6 图的应用（最小生成树：Prim / Kruskal）

定义

设连通无向图 G = (V, E) 有 n 个顶点。G 的**生成树**是包含全部 n 个顶点、恰有 n − 1 条边的极小连通子图。**最小生成树**（MST）是使各边**权值之和**最小的那棵生成树，其权值之和称为**最小代价生成树**的代价。

注意两个约束缺一不可：必须**含全部顶点**（否则就是森林），必须**连通且无环**（n − 1 条边）。非连通图没有生成树，只有**最小生成森林**——对每个连通分量分别求 MST。

6.4.1　MST 的性质（两种贪心算法的共同依据）

切割性质

把 V 分成两个非空集合 U 与 V − U，在所有横跨两集合的边中，若边 e 是权值最小的那条，则存在一棵 MST 包含 e。

Prim 算法正是这条性质的逐轮使用：每轮把“离 U 最近的点”拉进 U。

环路性质

若 C 是图中任一回路，e 是 C 中权值**严格最大**的边，则任何 MST 都不含 e。

Kruskal 算法是这条性质的使用：按权升序考察，一旦某边会与已选边成环，它就在该环上是最大者，故舍弃。

由此可推出一个常被误用的结论：

> **定理**
> 一棵 MST 的**权值之和唯一**，但边集**未必唯一**。当图中存在等权边时，可能有多棵不同的 MST，代价相同。

6.4.2 Prim 算法：逐点并入

从任一顶点开始，维护候选边表 `lowcost[j]`：**V − U 中的顶点 j 到 U 的最短边的权值**，并记该边来自 U 中哪个点。每轮取 lowcost 最小者并入 U，再用新并入的点更新其余候选值。

实例演算（教材图 6.19：6 个顶点 10 条边，从 v1 出发）。下表逐轮给出选定顶点、所选边及其权值：

轮	并入的顶点	所选边	权	本轮对候选表的修改
0	v1	—	—	以 v1 为起点建候选表
1	v3	v1—v3	1	v2 → 5（经 v3）、v5 → 6（经 v3）、v6 → 4（经 v3）；v4 经 v3 为 5，不优于原值 5，**不更新**
2	v6	v3—v6	4	v4 → 2（经 v6）；v5 经 v6 为 6，不优于原值 6，不更新
3	v4	v6—v4	2	无改进
4	v2	v3—v2	5	v5 → 3（经 v2）
5	v5	v2—v5	3	结束

生成树边集：v1—v3(1)、v3—v6(4)、v6—v4(2)、v3—v2(5)、v2—v5(3)，**总权值 = 1 + 4 + 2 + 5 + 3 = 15**。

> **易错**
> 第 1 轮里 v4 的候选值经 v3 也是 5，与原来的 5 相等，Prim 用的是严格比较，**不更新**。“相等就不换”是这类贪心的通用写法，否则前驱会来回摆动。

Prim 的复杂度：邻接矩阵下每轮扫全部候选值 O(n)、共 n − 1 轮，合计 **$O(n^2)$**，与边数无关，故适合**稠密图**。

6.4.3 Kruskal 算法：逐边选取

把所有边按权值升序排序，依次考察：若该边连接的两个顶点当前**属于不同连通分量**，则选入生成树（合并两个分量）；否则舍弃（否则成环）。选够 n − 1 条即停。

判“是否同分量”用**并查集**：find 判连通、union 合并，近似常数。故 Kruskal 的瓶颈在排序，复杂度 **$O(e \log e)$**，与 n 关系不大，适合**稀疏图**。

实例演算（4 个顶点 5 条边的并列权图：v1—v2 = 1，v3—v4 = 1，另有三条权为 2 的边彼此并列）。

算法	选出的边	总权值
Kruskal	v1—v2(1)、v3—v4(1)、v2—v3(2)	4
Prim（从 v1 起）	v1—v2(1)、v1—v3(2)、v3—v4(1)	4

同一张图上两种算法**选出的边集不同、总权值相同**——这就是“最小生成树是否唯一”这道判断题的标准答案：**不唯一，但代价唯一**。

6.4.4 两种算法对比

项目	Prim	Kruskal
贪心对象	每次选**离树最近的点**	每次选**不成环的最短边**
中间结果	始终是一棵树（逐步长大）	是一片森林，最后才合成一棵树
辅助结构	lowcost 表 + 前驱	边排序 + **并查集**
复杂度	$O(n^2)$	O(e log e)
适合	稠密图	稀疏图
起点依赖	结果可能因起点不同而不同（等权时）	与起点无关，但取决于排序稳定性（等权时）

典型例题

例 1 教材图 6.19 中有三条权 5 的边、三条权 6 的边，为什么它的 MST 仍然是唯一的？

解 “有等权边”只是“可能不唯一”的**必要非充分**条件。把该图的全部生成树枚举出来共 **121 棵**，其中权值和达到最小值 15 的**只有 1 棵**——等权的边并没有同时进入同一棵最小树的可能。判唯一性要看“最小代价的树有几棵”，而不是“有没有等权边”。

例 2 设图有 6 个顶点，各边权值互不相同，问 MST 是否唯一。

解 唯一。若存在两棵不同的 MST，取其中一棵的一条不在另一棵中的边 e，按环路性质与“权值互异”可推出可以替换出更小代价，矛盾。更直接地说：Kruskal 在权值互异时考察次序完全确定，故结果唯一。

例 3 某图的一条权值最小的边，是否一定在所有 MST 中？

解 若这条边**权值严格最小**，则它在任何 MST 中（否则可用它替换回路中的最大边得到更小树）；若有多条并列最小，则每次只能选其中若干条，**不必然全在**。

例 4 把 6.4.2 表中第 3 轮的边 v6—v4 换成 v1—v4（权 5），总代价如何变化？

解 原选法 v6—v4 权 2；若强制改用 v1—v4(5)，则代价增加 5 − 2 = 3，变为 18。这说明贪心选择的价值：在候选表里“离 U 最近”这一条，正是全局最优的一部分。

易错点与考点提示

- 生成树必须含**全部顶点**；非连通图没有生成树，只有生成森林。
- 生成树有 **n − 1** 条边，这是 Kruskal 的终止条件（“凑够 n − 1 条即停，剩余边不必再看”）。

- MST **代价唯一、边集未必唯一**：判断题问“最小生成树是否唯一”，答“不一定”，问“最小权值和是否唯一”，答“唯一”。
- 有等权边**推不出** MST 不唯一（例 1 就是反例）。
- Prim 的 lowcost 是“**到集合 U 的最近边权**”，不是“到某个固定源点的累计距离”——后者是 Dijkstra 的 D（见 6.5），两者形式极像、语义不同，是最经典的混淆点。
- Kruskal 判环必须用并查集；若每次都用 DFS 判连通，复杂度会退化到 O(e·n)。
- 求“连接所有顶点的最小成本”就是 MST；求“从一点到各点的最短路线”是最短路径，别用错模型。

习题

1. 用 Prim（从 v1 出发）重做 6.4.2 的图，把每轮的 lowcost 向量完整写出来。
2. 用 Kruskal 处理 6.4.3 的并列权图，说明为什么第三条权 2 的边必须舍弃。
3. 一个连通无向图有 n 个顶点、e 条边，其中权值最小的边恰有 2 条且并列。问这 2 条边能否同时出现在一棵 MST 中，什么情况下不能？
4. 说明“Prim 算法中间过程始终是一棵树”而“Kruskal 中间过程是森林”，并各举一步说明。
5. 设图 G 中每条边权值均为 1，问 MST 的代价与 G 的生成树个数之间有什么关系。

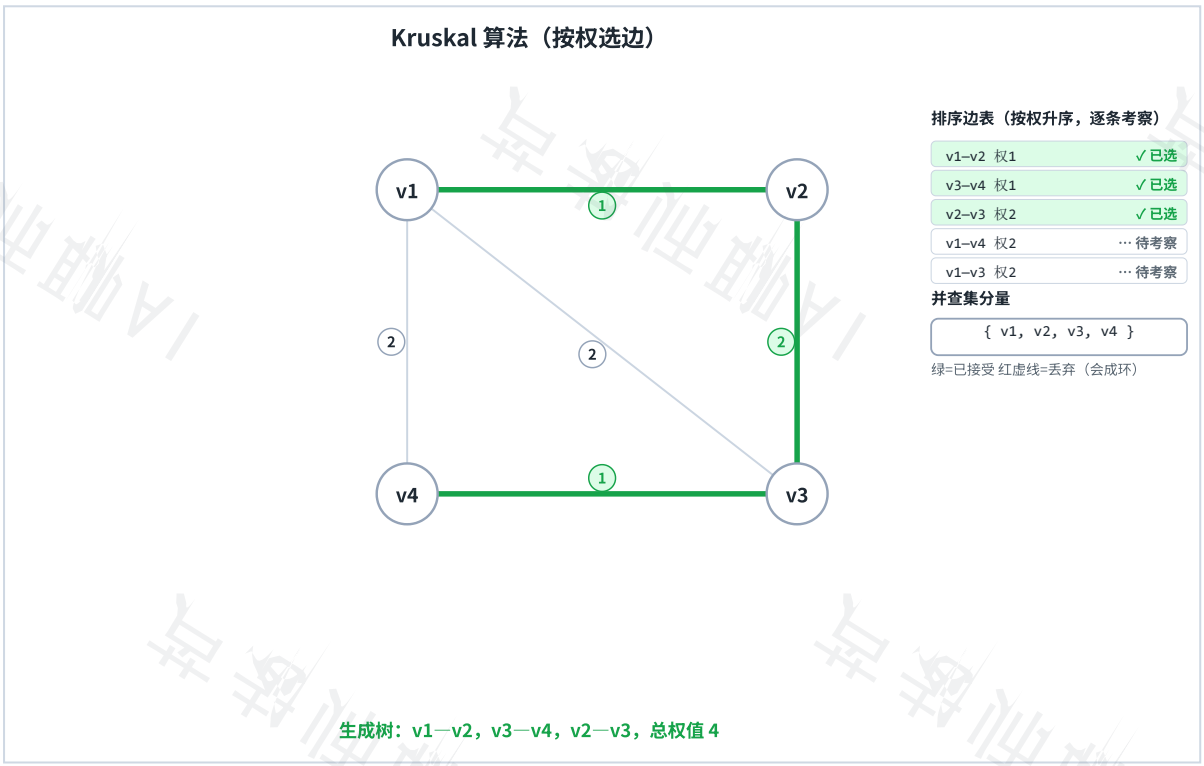


图 6-4 并列权的图：Kruskal 收 v1—v2(1)、v3—v4(1)、v2—v3(2)，总权值 4；同一张图 Prim 收 v1—v2、v1—v3、v3—v4——边集不同、权值相同，最小生成树不唯一

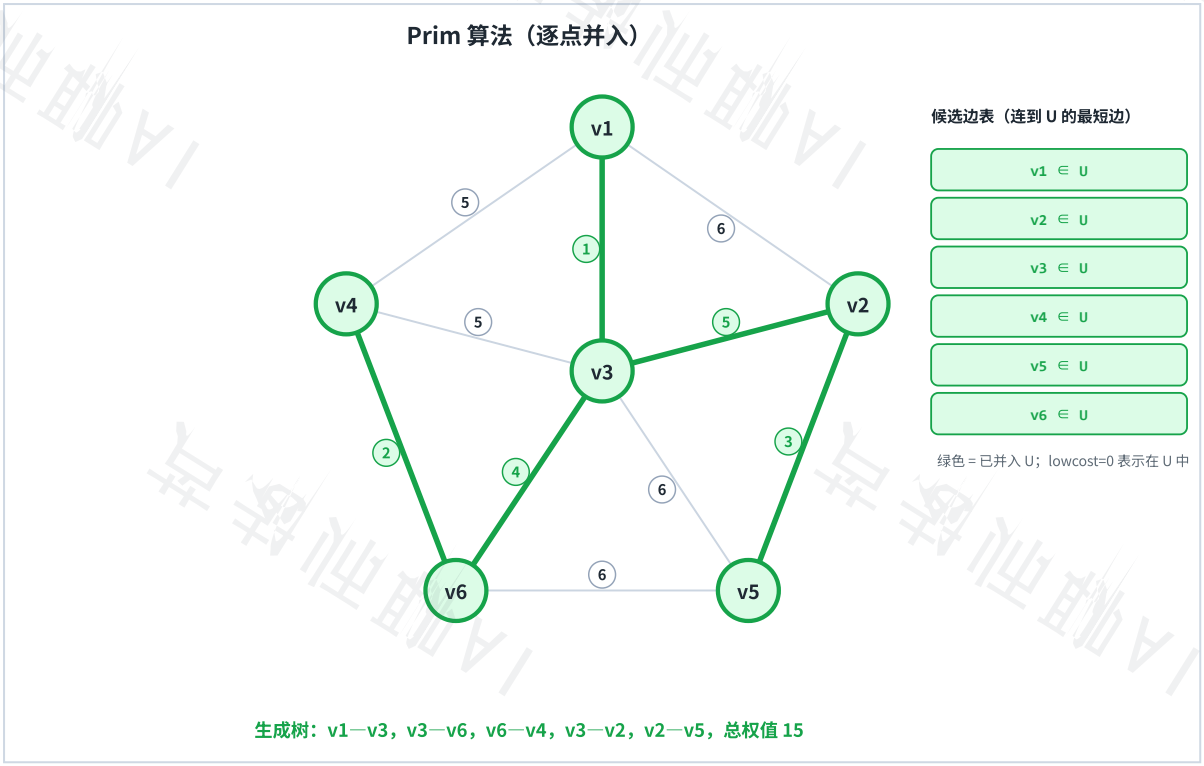


图 6-5　教材图 6.19 上 Prim 的完成态：v1—v3、v3—v6、v6—v4、v3—v2、v2—v5，总权值 15

6.5　最短路径

本节配套动画：第 38 项《最短路径：Dijkstra》、第 39 项《Floyd：各顶点间最短路径》 · 教材 6.6 图的应用（单源最短路径）

定义

设图 G 中顶点 v 到 w 的路径为 P，P 上各边（弧）的**权值之和**称为该路径的**路径长度**；从 v 到 w 的**路径长度最小**的那条路径称为**最短路径**，其长度记作 d(v, w)。无权图中路径长度即边数。

最短路径不唯一时其长度唯一；本章所求的“最短路径”通常同时给出长度与其中一条路径。

6.5.1　问题分类与算法适用条件

单源最短路径按图的性质分为三种情形，选择算法的依据即在于此：

图的性质	应采用的算法	时间复杂度
无权图（或各边权值相等）	BFS	O(n + e)
带权图，**所有边权非负**	Dijkstra	O(n²)（邻接矩阵）；O((n+e) log n)（邻接表 + 堆）
带权图，**存在负权边**	Bellman-Ford 或 SPFA	O(n·e)
有向无环图（DAG），可含负权	拓扑排序 + 松弛	O(n + e)
求任意两顶点之间（多源）	Floyd	O(n³)

前提

Dijkstra 算法要求**所有边权为非负**。这是算法正确性的前提而非效率考虑：出现负权边时算法可能给出错误结果（见例 2）。

6.5.2 Dijkstra 算法的贪心思想

算法设置两个互斥的顶点集合：

- **S**：已确定最短路径的顶点；
- **U = V − S**：尚未确定的顶点。

并设两个向量：

- **D[i]**：源点 v0 到 vi 的**当前**已知最短路径长度。初值为 v0 到 vi 的直连边权，无边则记 ∞；
- **Path[i]**：最短路径上 vi 的**直接前驱**，用于最后回溯路径。

每轮执行三个操作：

1. 在 U 中选取 D 值最小的顶点 k，将 k 并入 S，此时 D[k] 即为最终结果；
2. 以 k 为中转点，对其每条出边松弛：若 D[k] + w(k, j) < D[j]，则令 D[j] = D[k] + w(k, j)，Path[j] = k；
3. 若 U 为空，或 U 中所有顶点的 D 值均为 ∞（源点不可达剩余顶点），算法结束。

正确性证明要点。设第 t 轮并入 S 的顶点为 k，k 是 U 中 D 值最小者。反设 v0 到 k 还存在长度更短的路径 P，则 P 必先从 S 内某顶点出发、经 U 中某个顶点 x 才能到达 k。因边权非负，P 的长度不小于 D[x]，而 D[x] ≥ D[k]，与假设矛盾。故 D[k] 已是最终值。

上述推导中“边权非负”用于两处：一是 P 的长度不小于 D[x]，二是绕行后续路径不会使总长减小。任一条边为负，推理即不成立。

6.5.3 手工演算的标准写法

考试中按轮列表：每行写出本轮选定的顶点、选定之后**整个 D 向量**，以及本轮发生哪些更新。演算时必须同步维护 Path——D 值相同但被改写过前驱的情形很常见，只填 D 会丢路径分。

设带权有向图含 6 个顶点 8 条弧：v0→v2 = 10，v0→v4 = 30，v0→v5 = 100，v1→v2 = 5，v2→v3 = 50，v3→v5 = 10，v4→v3 = 20，v4→v5 = 60。求源点 v0 到其余顶点的最短路径。

轮	选定 u	D[v1]	D[v2]	D[v3]	D[v4]	D[v5]	本轮的更新
0	v0	∞	10	∞	30	100	初值取 v0 的出边权
1	v2	∞	**10**	60	30	100	v2→v3：10 + 50 = 60 < ∞
2	v4	∞	10	**50**	**30**	90	v4→v3：30 + 20 = 50 < 60；v4→v5：30 + 60 = 90 < 100
3	v3	∞	10	50	30	**60**	v3→v5：50 + 10 = 60 < 90
4	v5	∞	10	50	30	60	v5 无出边
5	结束	∞	—	—	—	—	U 中仅剩 v1 且 D = ∞

结果 D = [0, ∞, 10, 50, 30, 60]。回溯 Path 得各最短路径：

终点	最短路径	长度
v2	v0 → v2	10
v4	v0 → v4	30
v3	v0 → v4 → v3	50
v5	v0 → v4 → v3 → v5	60
v1	不可达	∞

注意 v3 的路径是 v0 → v4 → v3 而非 v0 → v2 → v3：后者长度 60，在第 2 轮被改小。这说明**最短路径树中一个顶点的前驱，未必是第一个把它松弛成功的顶点**。

6.5.4 最短路径树及其与生成树的区别

源点到各可达顶点的最短路径合起来构成一棵以源点为根的树，称为**最短路径树**。使用时须分清两件事：

- 最短路径树是**生成树**当且仅当源点可到达全部顶点；存在不可达顶点时，它只覆盖可达的那一部分；
- 最短路径树**不是最小生成树**：前者的判据是“到源点的累计长度”最小，后者是“这条边自身的权值”最小，二者结果一般不同。

6.5.5 多源最短路径：Floyd 算法

Floyd 求解任意两顶点间的最短路径。设 D(i, j, k) 表示“从 vi 到 vj、只允许经过 {v1, …, vk} 作为中间顶点”时的最短路径长度，则

> **递推**
> D(i, j, k) = min(D(i, j, k−1)，D(i, k, k−1) + D(k, j, k−1))

即：路径要么不经过 vk，要么恰好由 vi ⇝ vk 与 vk ⇝ vj 两段拼接。k 从 1 取到 n，最终 D 矩阵即为答案。因第 k 轮只用第 k−1 轮的数据，可就地累加，故实现上是三重循环、k 在**最外层**：

要点	说明
循环次序	必须是 k-i-j，k 在最外层；把 k 挪到最内层是常见错误写法，一般图上也算不出正确结果
时间 / 空间	$O(n^3)$ / $O(n^2)$，与边数无关，故适合**稠密图**
负权	可以处理负权边
负环	图中不得存在负环。若存在，算法结束后**对角线出现负数**，可据此判定

负环的意义：若某点能沿负环绕回自身，则该环上及一切能绕上此环的点对，最短路径没有定义（再绕一圈总长更短）；与该环无关的点对仍为正确答案。因此“对角线为负”不是算法失效，而是算法把负环报告了出来。

“适合稠密图”这一说法是相对“对每个源点跑一次 Dijkstra”而言：邻接矩阵下后者也是 $O(n^3)$，而 Floyd 编程简单、一次得到全部点对，还能顺带检出负环。该比较只在边权非负时成立——有负权时 Dijkstra 根本不可用。

典型例题

例 1 在 6.5.3 的图中，若把弧 v4→v3 的权由 20 改为 45，D[v3]、D[v5] 与相应路径如何变化？

解 第 2 轮经 v4 得 30 + 45 = 75，不小于 v3 当时的 60，故**不更新**。于是 D[v3] = 60、路径为 v0 → v2 → v3；第 3 轮 v3 定死后松弛 v3→v5 得 60 + 10 = 70 < 90，故 D[v5] = 70、路径 v0 → v2 → v3 → v5。一条边权的改动会同时改变多个终点的路径。

例 2 在上图中增加弧 v5→v2 = −60，Dijkstra 的输出是否正确？

解 不正确。第 1 轮 v2 即被并入 S，D[v2] = 10；第 4 轮 v5 定死为 60，此时 60 + (−60) = 0 < 10 本可改进 v2，但 v2 已不在 U 中、不再被检查，算法最终输出 D[v2] = 10，而真实值为 0。注意回路 v2→v3→v5→v2 的权值和为 50 + 10 − 60 = 0，**图中并无负环，真实最短路径存在**——错误完全来自算法前提被破坏。此图应改用 Bellman-Ford 或 SPFA。

例 3 设 v0 到 v4 有三条路径：经 v1 为 2 + 4，经 v2 为 3 + 3，经 v3 为 5 + 1。问 D[v4] 与算法给出的路径。

解 三条长度均为 6，故 D[v4] = 6 唯一；但松弛判据是严格小于，“等于”时不更新，因此 Path[v4] 只保留最先松弛成功的那条前驱。结论：最短路径长度唯一，最短路径可以不止一条，而算法只输出其中一条。

例 4 某图执行 Floyd 后 D 矩阵的对角线元素为 [0, 0, −45, −45, 0, −90]，能得出什么结论？

解 v2、v3、v5 满足 d(i, i) < 0，即“从自身出发绕一圈回来长度反而减小”，说明图中存在负环，且这三个顶点都能绕上该环；涉及它们的点对最短路径无定义。对角线上数值的大小不能用来核对环的权值——同一格可能被更新多轮（−90 表示相当于绕了两圈），**判负环只看符号**。

易错点与考点提示

- 使用 Dijkstra 前必须确认边权非负；题干出现负权时，正确回答是“不能用 Dijkstra”，而不是“结果一样所以能用”。
- 与 Prim 算法的区分：两者都维护“到已定集合的距离向量、每轮取最小”，但 Prim 的 lowcost 是**到生成树的最近一条边的权**，Dijkstra 的 D 是**到源点的累计长度**，语义与结果均不同。
- 松弛采用**严格小于**，因此并列最短路径只记录一条。
- 有向图中“忽略弧的方向后连通”与“从源点可达全部顶点”是两件事：后者不成立时应表述为非强连通或源点不可达，不能称“图不连通”。
- 无权图求最短路应直接用 BFS，复杂度 O(n + e)。
- Floyd 的三重循环次序是 k-i-j；k 不在最外层即错。
- Floyd 可处理负权边，但要求图中无负环；Dijkstra 则要求边权非负。

习题

1. 用 6.5.3 的图，把源点改为 v2，手工演算 D 向量与各最短路径。
2. 上图中若删去弧 v4→v3，D[v3] 与 D[v5] 分别是多少？
3. 说明为什么 Dijkstra 不能处理负权边，并构造一个只含 3 个顶点、3 条弧的反例。

4. 对 6 个顶点的稠密图求所有点对间最短路径，比较“6 次 Dijkstra（邻接矩阵）”与“Floyd”的运行次数级别，并说明还应考虑什么因素。
5. Floyd 结束后 D[i][i] = 0 对所有 i 成立，能否据此断定图中无环？

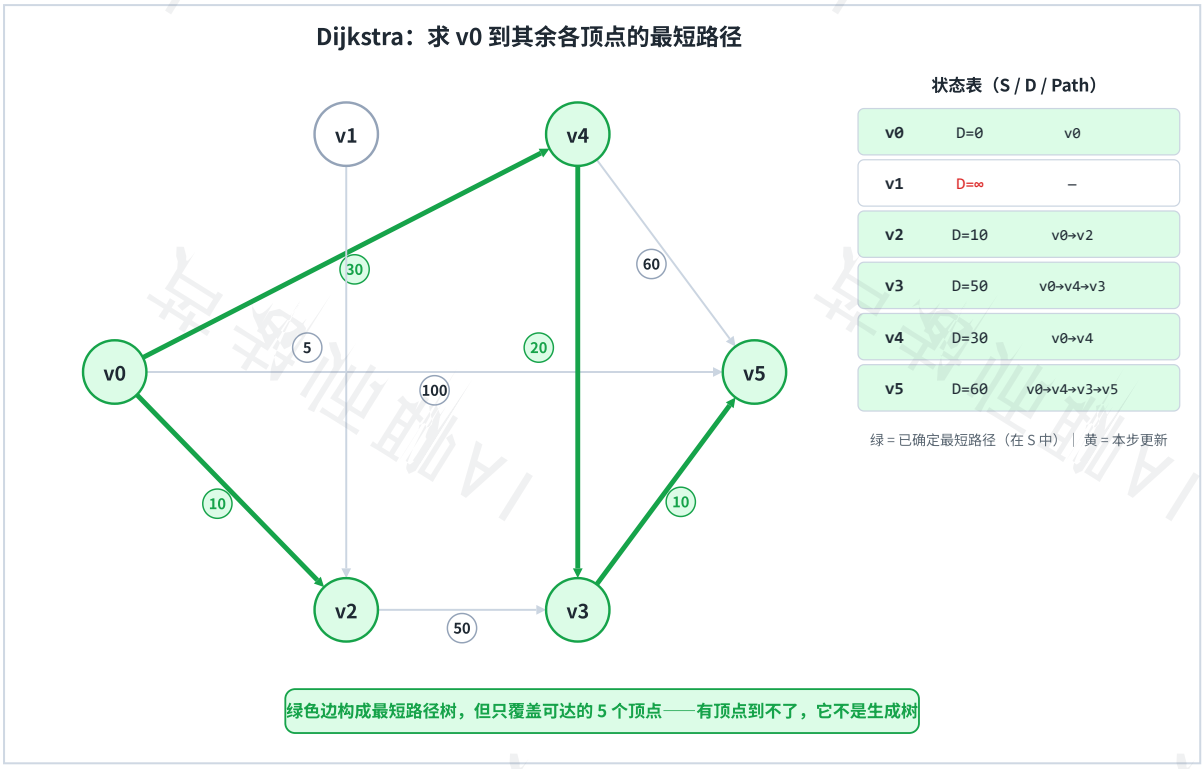


图 6-6　Dijkstra 跑完后的 D 表与最短路径树

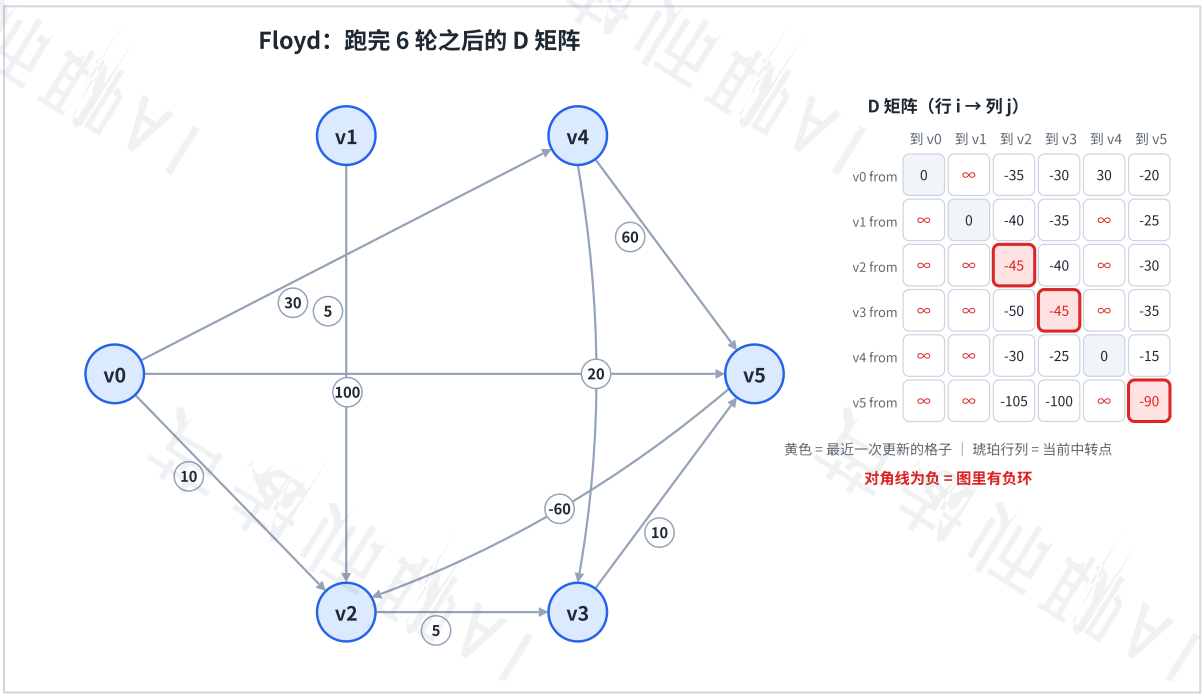


图 6-7　Floyd 跑完的 D 矩阵：对角线出现 −45 / −45 / −90，即图中有负环

6.6　拓扑排序与关键路径

本节配套动画：第 40 项《拓扑排序：AOV 网与回路检测》、第 41 项《关键路径：AOE 网与关键活动》 · 教材 6.6 图的应用（AOV 网、拓扑排序、回路检测）· 活动名 C1…C6 是教材编号，伪码里的 inDegree[0] 就是 C1

两类网络

用顶点表示活动、弧表示先后关系的有向无环图称 **AOV 网**；用弧表示活动、顶点表示事件、弧上权表示活动持续时间的有向无环图称 **AOE 网**。前者用于**拓扑排序**（排工序、判回路），后者用于**关键路径**（算工期、找瓶颈）。两者都要求图无环。

6.6.1 拓扑排序

定义

拓扑序列是把 AOV 网全部顶点排成的一个线性序列，使得对每条弧 <i, j>，i 在序列中必先于 j 出现。

算法：

1. 求所有顶点的入度，把入度为 0 的顶点全部进栈；
2. 弹栈输出一个顶点 k，并把 k 的**所有出弧删除**——即其后继顶点入度减 1，减到 0 者进栈；
3. 重复第 2 步，直到栈空。

实例演算（教材的 AOV 网，顶点表示课程 C1…C6，6 条弧）：

顶点	C1	C2	C3	C4	C5	C6
初值入度	0	0	1	2	1	2

弧集为 <C1,C3>、<C1,C4>、<C2,C4>、<C2,C5>、<C3,C6>、<C5,C6>。以“入栈次序按编号升序、弹栈取栈顶”实现：

步	动作	栈（底→顶）	已输出	入度变化
1	C1、C2 入度为 0，先后进栈	C1, C2	—	—
2	弹 C2 输出	C1	C2	C4: 2→1；C5: 1→0 进栈
3	弹 C5 输出	C1	C2, C5	C6: 2→1
4	弹 C1 输出	空	C2, C5, C1	C3: 1→0 进栈；C4: 1→0 进栈
5	C3、C4 进栈后弹 C4	C3	C2, C5, C1, C4	无
6	弹 C3 输出	空	C2, C5, C1, C4, C3	C6: 1→0 进栈
7	弹 C6 输出	空	C2, C5, C1, C4, C3, C6	结束

得到拓扑序列 **C2 → C5 → C1 → C4 → C3 → C6**。

序列不唯一

只要某一步栈中同时有两个以上入度为 0 的顶点，就有多种选择。把上例的全部约束展开枚举，合法拓扑序列共有 **22 个**——栈这一版给出的恰好是其中字典序最大的一个。

判据：若拓扑排序结束时输出的顶点数少于 n，说明剩下的顶点都在环上（它们的入度永远降不到 0），故图**有环**。这就是“拓扑排序当回路检测用”的原理。

6.6.2 拓扑序列唯一的判据

拓扑序列唯一 ⇔ 该唯一序列中**每对相邻顶点之间都有一条弧** ⇔ 图中存在一条**穿过全部顶点的路径**。等价地表述：每一步栈里都恰好只有一个入度为 0 的顶点。

6.6.3 关键路径与四个量

工程由若干活动组成，AOE 网中源点 v0 表示工程开始、汇点表示工程结束。四个量按下列顺序求：

量	含义	求法	方向
ve(k)	事件 k 的**最早**发生时间	ve(j) = max{ ve(k) + w(<k, j>) }，ve(源) = 0	按**拓扑序**正向
vl(j)	事件 j 的**最迟**发生时间	vl(k) = min{ vl(j) − w(<k, j>) }，vl(汇) = ve(汇)	按**逆拓扑序**逆向
e(a)	活动 a = <k, j> 的**最早**开始时间	e(a) = ve(k)	—
l(a)	活动 a 的**最迟**开始时间	l(a) = vl(j) − w(<k, j>)	—

> **关键活动**
> l(a) − e(a) = 0 的活动没有一天富余，称为关键活动；从源点到汇点全部由关键活动构成的路径称为**关键路径**，其长度就是**工程的最短工期**。

实例演算（AOE 网：6 个事件、8 项活动，权为所需天数）：

活动	弧	权 w	e(a) = ve(k)	l(a) = vl(j) − w	l − e	关键?
a1	<v0, v1>	3	0	4 − 3 = 1	1	
a2	<v0, v2>	2	0	2 − 2 = 0	**0**	√
a3	<v1, v3>	2	3	6 − 2 = 4	1	
a4	<v1, v4>	3	3	7 − 3 = 4	1	
a5	<v2, v3>	4	2	6 − 4 = 2	**0**	√
a6	<v2, v5>	3	2	8 − 3 = 5	3	
a7	<v3, v5>	2	6	8 − 2 = 6	**0**	√
a8	<v4, v5>	1	6	8 − 1 = 7	1	

事件时间：ve = [0, 3, 2, 6, 6, 8]，vl = [0, 4, 2, 6, 7, 8]。

关键活动为 a2、a5、a7，**关键路径 v0 → v2 → v3 → v5，长度 2 + 4 + 2 = 8 天**，即工程最短工期 8 天。

求 ve 的逐格过程（正向、按拓扑序 v0→v1→v2→v3→v4→v5）：

- ve(v1) = ve(v0) + 3 = 3；ve(v2) = ve(v0) + 2 = 2；
- ve(v3) = max{ ve(v1) + 2 = 5, ve(v2) + 4 = 6 } = **6**；
- ve(v4) = ve(v1) + 3 = 6；
- ve(v5) = max{ ve(v2) + 3 = 5, ve(v3) + 2 = 8, ve(v4) + 1 = 7 } = **8**。

典型例题

例 1 上例中若把活动 a6 的持续时间由 3 改为 7，工期是否改变？

解 a6 = <v2, v5>。经 a6 到汇点的长度为 ve(v2) + 7 = 2 + 7 = 9 > 8，故 ve(v5) 变为 9，工期**由 8 天延长为 9 天**，且关键路径变为 v0 → v2 → v5（a2、a6）。这说明：**非关键活动的富余被吃光之后，它就会变成关键活动**。

例 2 压缩关键活动 a5 的时间 1 天，工期一定缩短 1 天吗？

解 在只有一条关键路径时是的：a5 缩短 1 天使 ve(v3) 由 6 变为 5，进而 ve(v5) = max{5, 5 + 2 = 7, 7} = 7，工期缩短 1 天。但若存在**多条**关键路径，压缩其中一条上的活动不会缩短工期——另一条同样长的路径立刻成为新的关键路径。

例 3 某 AOV 网拓扑排序结束时只输出了 4 个顶点（共 6 个），能断定什么？

解 图中**有环**，且未输出的 2 个顶点都在环上或环的下游。注意这个判据成立的前提是“删弧”这步实现正确：若只弹顶点不删弧，无环的图也会输出不满。

例 4 上例的 AOV 网中，若要求拓扑序列唯一，需要怎样添加弧？

解 只需让序列中相邻项之间都有弧。例如对序列 C2, C5, C1, C4, C3, C6 补上 C5→C1、C1→C4 已有、C4→C3、C3→C6 已有，则缺 C2→C5（已有）与 C4→C3，补一条 <C4, C3> 即可使每对相邻都有弧，从而序列唯一。

易错点与考点提示

- 拓扑序列**不唯一**，但“能否完成全部 n 个顶点”是唯一的——与算法实现无关。
- 判回路的依据是“输出顶点数 < n”，其**前提**是每弹出一个顶点就删掉它的全部出弧；漏掉删弧，无环图也会排不完。
- 拓扑排序的本质是**入度归零才允许输出**，与队列还是栈无关；用栈得到的是某一类序列，用队列得到另一类，两者都合法。
- AOE 网中**顶点是事件、弧才是活动**；求 ve 用 max（要等所有前驱完成），求 vl 用 min（不能耽误任何后继），方向反了就全错。
- 关键路径**可能不止一条**；缩短工期的必要条件是所有关键路径都被压缩。
- 关键活动上任何延误都会推迟工期，但**非关键活动的延误不一定影响工期**——只要不超过它的富余 l − e。
- 事件的最早发生时间 ve 与活动的最早开始时间 e 是两回事：e(a) = ve(k)，a 的弧尾事件。

习题

1. 对 6.6.1 的 AOV 网，把第 4 步的栈改为“入度为 0 时按编号**降序**进栈”，写出拓扑序列。
2. 上例 AOV 网共有 22 个合法拓扑序列。若再添加弧 <C4, C5>，合法序列个数变为多少？
3. 对 6.6.3 的 AOE 网，把 a3 的权由 2 改为 5，重新计算 ve、vl 与关键路径。
4. 说明为什么求 ve 必须按拓扑序、求 vl 必须按逆拓扑序。

5. 一个 AOE 网有两条关键路径，长度都是 10。若只压缩第一条上某活动 2 天，工期变成多少？

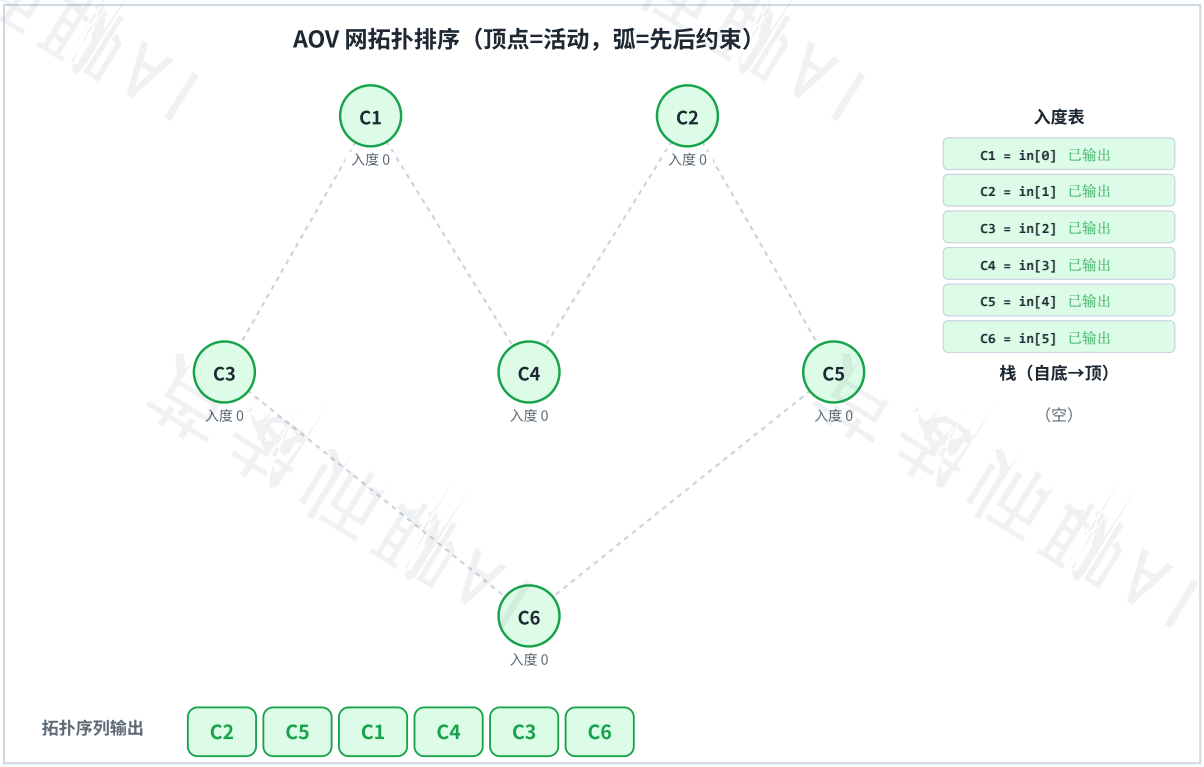


图 6-8　拓扑排序完成：C2 → C5 → C1 → C4 → C3 → C6，而该图的合法序列共有 22 个

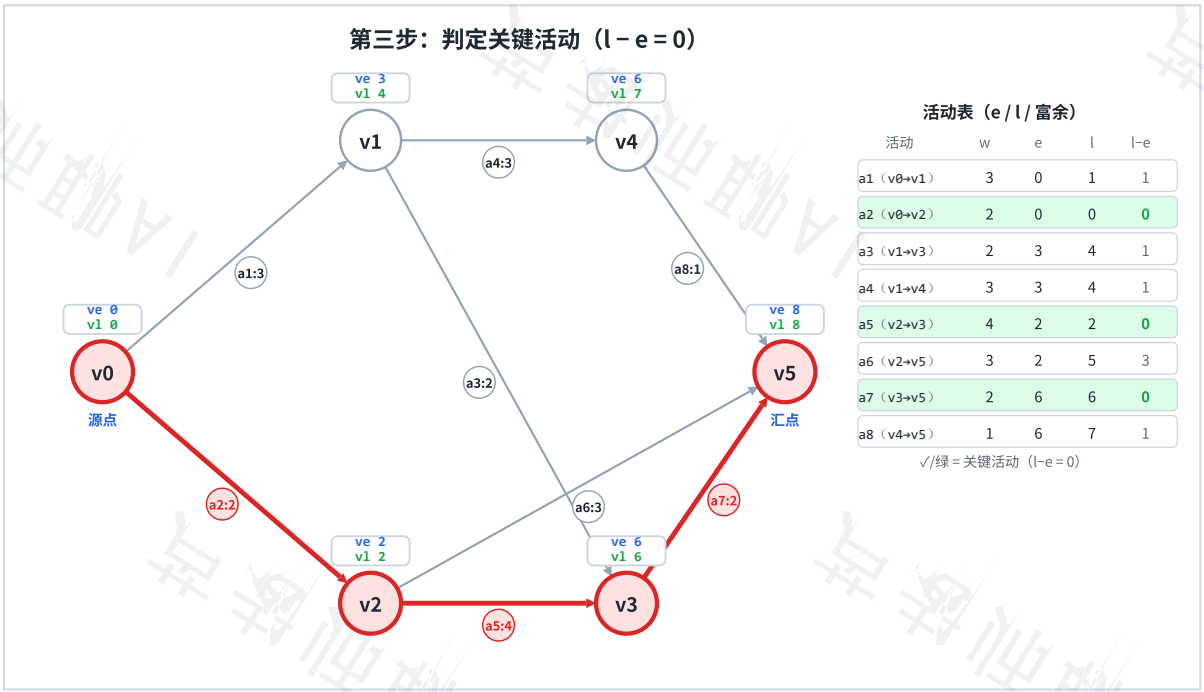


图 6-9　AOE 网的关键路径 v0 → v2 → v3 → v5，长度 8 天即最短工期

第 7 章　查找

7.1　查找的基本概念与线性表上的查找

本节配套动画：第 42 项《顺序查找与折半查找》、第 43 项《分块查找（索引顺序查找）》 · 教材 7.2 静态查找（哨兵技巧、二分判定树、ASL）

> **定义**
> 查找表是同一类型数据元素的集合。查找的性能用**平均查找长度 ASL**（Average Search Length，即关键字比较次数的期望）衡量。按表是否有序、是否动态可改，分为静态查找结构与动态查找结构。

7.1.1　顺序查找与哨兵技巧

从表的一端起逐个比较。设表长 n、查找成功概率等概率：

> **ASL**
> 成功时 ASL = Σ i·(1/n) = **(n + 1)/2**；失败时（需比到表尾）为 n + 1。

哨兵技巧：把待查关键字复制到 `ST[0]`，则循环 `while(ST[i] != key) i--;` 无需判越界——每轮省一次比较，代码更短且不会漏判。时间仍是 O(n)，但常数更小。

顺序查找的适用面最广：**无序表、链表都只能用它的思想**；若表按概率有序（高频在前），ASL 可降到约 0.55。

7.1.2　折半查找与判定树

前提：**有序 + 顺序存储**（链表不能折半，因为无法随机存取）。

```
low = 1; high = n;
while (low <= high) { mid = (low + high) / 2;
    if (key == ST[mid]) return mid;
    else if (key < ST[mid]) high = mid - 1;
    else low = mid + 1; }
```

实例：表 (5, 13, 19, 21, 37, 56, 64, 75, 80, 88, 92)，n = 11，查 key = 21。

步	low	high	mid	ST[mid]	结果
1	1	11	6	56	56 > 21 → high = 5
2	1	5	3	19	19 < 21 → low = 4
3	4	5	4	**21**	命中，第 4 个

共比较 **3** 次。

判定树

折半过程的比较结构是一棵**二叉排序树**：根是 ⌊(1+n)/2⌋，左右由区间递归生成。它是一棵**平衡**树，故成功查找最多 ⌈log₂(n+1)⌉ 次，ASL ≈ **$\log_2(n+1)-1$**。本例 ≈ $\log_2 12 - 1$ ≈ **2.58**，远优于顺序查找的 (n+1)/2 = 6.0。

失败情形要看**外部结点**：判定树空指针处的层数即“比较次数 + 1”，失败时最多比较 ⌈log₂(n+1)⌉ 次。

7.1.3 分块查找（索引顺序查找）

把表分成 b 块，要求**块间有序**（第 i 块所有元素都小于第 i+1 块），块内可无序。另建索引表，每项记该块的**最大关键字**与起始位置。

查找分两段：先在索引表中定块（可顺序可折半），再在块内顺序查找。

实例：15 个元素分 3 块、每块 5 个，索引上界为 22、44、74，查 key = 38。

- 索引表顺序比较：22 < 38（跳）、44 ≥ 38 → 落在第 2 块，比较 2 次；
- 块内从第 6 个元素起逐个比，到第 9 个命中，比较 4 次；
- 合计 **6 次**。

等概率下 ASL = 索引 ASL + 块内 ASL。若索引与块内都用顺序查找，取 b = ⌈√n⌉、每块 ⌈√n⌉ 个时 ASL ≈ **$\sqrt{n}+1/2$** 最小（本例 3 块每块 5 个：顺序索引 + 顺序块内 = (4/2) + (6/2) = 5）。

三种静态查找对照

方法	前提	ASL（成功）	适用
顺序查找	无	(n+1)/2	无序、链表、任何表
折半查找	有序 + 顺序存储	≈ $\log_2(n+1)-1$	静态表，很少增删
分块查找	块间有序	≈ $\sqrt{n}+1/2$	介于两者之间，“动态索引表”

典型例题

例 1 对长度 12 的有序表做折半查找，成功查找最多比较几次？

解 最多 ⌈log₂(12 + 1)⌉ = **4** 次。

例 2 下列哪种查找不能用于链表？(a) 顺序 (b) 折半 (c) 分块

解 (b)。折半要求随机存取中间位置，链表做不到 O(1) 定位。

例 3 2000 个元素的表，等概率查找。用顺序查找、分块查找（每块 20 个，索引顺序查）、折半查找的 ASL 各约多少？

解 顺序 (n+1)/2 = **1000.5**；分块：索引 100 项顺序查 (100+1)/2 = 50.5，块内 (20+1)/2 = 10.5，合计 **61**；折半 ≈ $\log_2 2001 - 1$ ≈ **10**。

易错点与考点提示

- 折半的 `mid = (low + high) / 2` 在真实代码里有**整型溢出**风险，工程写法是 `low + (high - low) / 2`。
- 折半的循环条件必须是 `low <= high`。写成 `low < high` 时区间缩到最后**一格**就不再比了：漏掉的是每段区间的端点，教材那张 11 元素的表里有 **4 个 key（13、37、75、92）明明在表中却判失败**。
- 上面这条最阴的地方在于它对**当前查的那个 key 常常是对的**——中途命中时两条写法走的路完全一样，勾上错法画面看不出差别。所以要判就得把整张表逐个 key 扫一遍，不能只问“这一次对不对”。
- 判定树的**层数**与**比较次数**差 1：第 k 层的结点成功时要比 k 次，失败要看外部结点。
- 分块查找“块间有序”是硬前提，块内才允许无序。
- 顺序查找是**唯一**能用于无序链表的方法；别答“折半 + 链表”。
- ASL 要区分“成功”与“失败”两种，题目常只问其中一个。

习题

1. 对表 (7, 14, 18, 21, 23, 29, 31, 35, 38) 折半查找 key = 31，写出每步的 low / high / mid 与比较次数。
2. 上表等概率查找失败时，ASL 是多少？
3. 把 27 个元素分块，每块多大能使“索引顺序 + 块内顺序”的 ASL 最小？



图 7-1　小结：折半查找 ASL ≈ $\log_2(n+1) - 1 \approx 2.6$，远优于顺序查找的 $(n+1)/2 = 6.0$。



图 7-2　小结：等概率下 ASL = 索引查找 + 块内查找。

7.2　树形查找：二叉排序树与平衡二叉树

本节配套动画：第 44 项《二叉排序树（插入与删除）》、第 45 项《平衡二叉树（四种旋转）》 · 教材 7.3 动态查找（中序有序、删除三情形）

> **定义**
> **二叉排序树（BST）**或者为空，或者：若左子树非空则其所有结点关键字 < 根；若右子树非空则其所有结点关键字 > 根；左右子树本身又是 BST。它是一棵**动态**查找树——查找、插入、删除都在树上做。

7.2.1　查找、插入与中序有序性

查找从根出发，比根小走左、比根大走右、相等命中；比较路径长度 = 该结点深度 + 1。

> **性质**
> **中序遍历 BST 得到递增序列**。这是判定“是不是 BST”和验证插入删除是否正确的标准方法。

插入永远**发生在叶子层**：沿查找路径走到空位挂上新结点，**不需要调整树形**。

实例：依次插入 45, 12, 24, 37, 53, 90。

步	动作	中序序列
1	45 作为根插入	45
2	12 < 45 → 45 的左孩子	12 ≤ 45
3	24 < 45 → 左；24 > 12 → 12 的右孩子	12 ≤ 24 ≤ 45
4	37 沿 45左→12右→24右 挂上	12 24 37 45
5	53 > 45 → 45 的右孩子	12 24 37 45 53
6	90 > 45 → 右；90 > 53 → 53 的右孩子	**12 ≤ 24 ≤ 37 ≤ 45 ≤ 53 ≤ 90**

7.2.2 删除的三种情形

情形	处理
① 叶子 | 直接删，父结点相应指针置空 |	
② 只有左或右子树	**用子树顶替自己（改父指针）**
③ **左右子树都有**	用**中序前驱**（左子树最大）或**中序后继**（右子树最小）顶替其位置，再删除那个前驱 / 后继（它至多有一个子树，归入情形 ①②）

7.2.3 BST 的退化与平衡二叉树

查找 ASL 与树高同阶：理想平衡时 $O(\log_2 n)$，但按有序序列 (1, 2, 3, …, n) 插入会退化成**一条右链**，ASL 变 O(n)。

> **AVL**
> 平衡二叉树是**每个结点的平衡因子 bf = 左子树高 − 右子树高，满足 |bf| ≤ 1** 的二叉排序树。

失衡只可能发生在插入路径上**最小的失衡子树**的根上，四种情形：

情形	插入位置	旋转
LL	左孩子的左子树	**右旋**一次
RR	右孩子的右子树	**左旋**一次
LR	左孩子的右子树	先对左孩子**左旋**，再对根**右旋**
RL	右孩子的左子树	先对右孩子**右旋**，再对根**左旋**

实例：依次插入 30 → 20 → 10。插 30 为根；插 20 后 30 的 bf = 1（未失衡）；插 10 后 30 的 bf = 2，属 **LL**，对 30 右旋，新子树根 = **20**，中序仍是 10 ≤ 20 ≤ 30，全部 |bf| ≤ 1。

> **代价**
> AVL 把查找稳定在 O(log n)，代价是每次插入 / 删除都要沿路径回溯检查并可能旋转。

典型例题

例 1 由序列 (45, 12, 24, 37, 53, 90) 构造的 BST 与由 (45, 53, 90, 12, 24, 37) 构造的，形状是否相同？

解 相同。两序列的根都是 45，左子集合都是 {12, 24, 37}、右子集合都是 {53, 90}；再看内部：{12, 24, 37} 里 12 总是最先出现，故仍是该子树的根，24 挂其右、37 再挂右；{53, 90} 里 53 先于 90。逐层都保持『祖先先于后代』，所以两棵树形状一致。**会改变形状的是**把某个后代排到它的祖先之前（例如把 24 提到 12 前面）。

例 2 删除 BST 中“左右子树都有”的结点时，为什么不能直接删除它？

解 它有两个子树，父结点的指针无法同时指向两者。用中序前驱 / 后继顶替可以保持“左 < 根 < 右”，且前驱 / 后继至多有一个子树（左子树最大者必无右孩子），删除它属于情形 ① 或 ②。

例 3 对空 AVL 树依次插入 10, 20, 30, 40, 50, 60，画出最终树。

解 10 → 20（RR 倾向）→ 插 30 时 10 失衡 RR，左旋得根 20（左 10 右 30）→ 插 40、50 时 20 失衡 RR，左旋得根 30（左 20(10,·)、右 40(·,50)）→ 插 60 使 40 失衡 RR，对 40 左旋。最终根 30，左子树 20(10)，右子树 50(40, 60)，全部 |bf| ≤ 1。

易错点与考点提示

- BST 的**中序**才有序；先序、后序都不体现大小关系。
- 删除操作可能改变树形（用前驱 / 后继顶替后，结点值被搬动），别以为“只是摘掉一个结点”。
- 同样的结点集合，不同插入顺序得到**不同形状**的 BST；有序输入是最坏情况。
- AVL 只在**最小失衡子树**的根上旋转，一次插入最多两次旋转（LR / RL）；判失衡要沿插入路径**自底向上**找第一个 |bf| > 1 的结点。
- LR / RL 是**两次**旋转，第一次旋转的对象是“孩子的孩子”方向。
- 红黑树（7.4）用“近似平衡”换更少的旋转次数，AVL 更平衡、查找更快但维护代价更高。

习题

1. 依次插入 50, 30, 70, 20, 40, 60, 80 构造 BST，写出中序与树高。
2. 删除题 1 中的 50，画出结果并说明用的是前驱还是后继。
3. 依次插入 10, 15, 5, 8 构造 AVL，指出发生在哪种失衡、如何旋转。

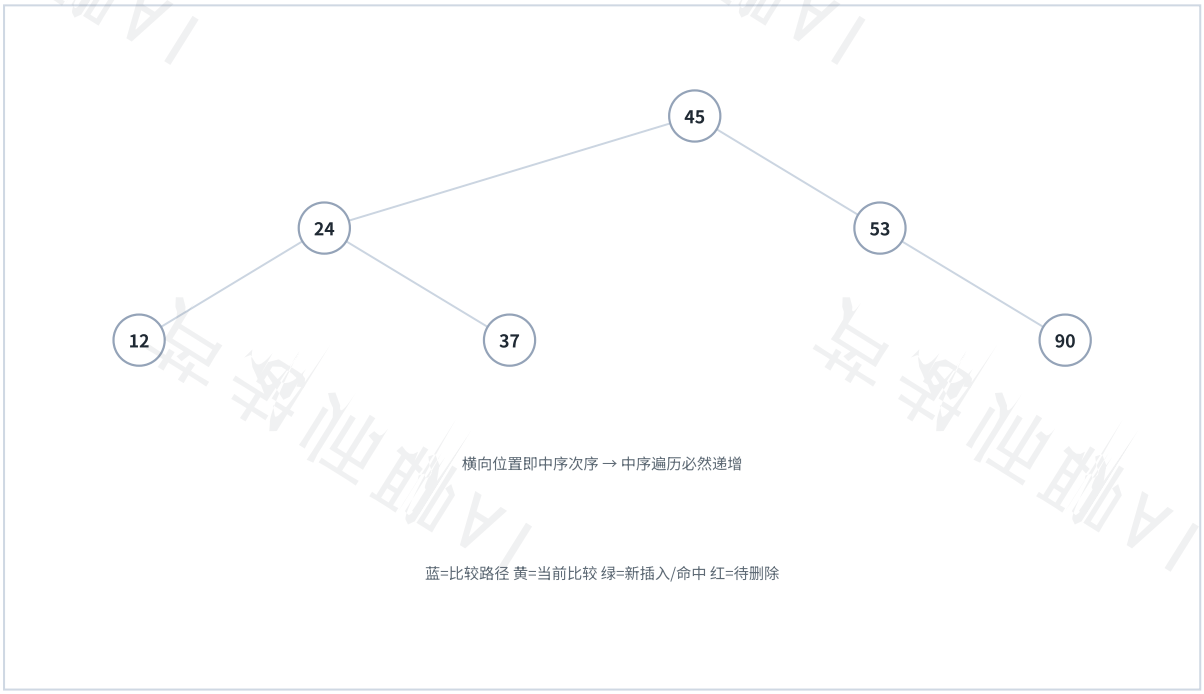


图 7-3 插入完成。中序遍历：12 ≤ 24 ≤ 37 ≤ 45 ≤ 53 ≤ 90 —— 恰好递增！查找就是沿着这条比较路径走，ASL 与结点深度挂钩（树越平衡越快）。

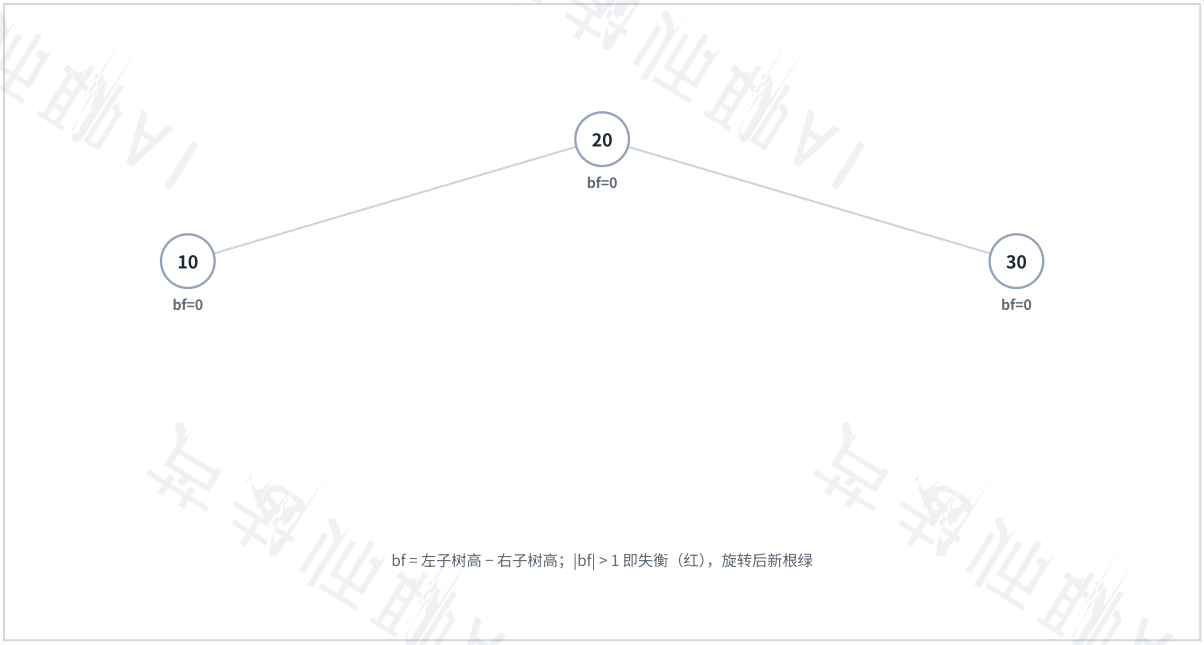


图 7-4　场景完成：依次插入 30 → 20 → 10，最终根为 20。

7.3　多路查找树：B 树与 B+ 树

本节配套动画：第 46 项《B 树与 B+ 树：多路平衡与分裂上移》 · 408 大纲 六(六) B 树及其基本操作、B+ 树的基本概念（插入分裂、查找路径、叶子链表）

> **动机**
> 二叉树查找树高 $[\log_2 n]$，n 大时磁盘就要读十几次。**B 树**把“每个结点只装一个关键字”改成“一个结点装 m − 1 个关键字”，树变矮变胖，一次磁盘页能读进多个关键字，外存读写次数随之下降。

7.3.1　m 阶 B 树的定义

一棵 m 阶 B 树（又称 B 树）满足：

1. 每个结点至多有 **m 棵子树**，即至多 **m − 1** 个关键字；
2. 除根外，每个结点至少有 **⌈m/2⌉** 棵子树，即至少 **⌈m/2⌉ − 1** 个关键字；根若非叶子则至少 2 棵子树（1 个关键字）；
3. 若含 k 个关键字，则含 **k + 1** 棵子树，关键字递增有序：$K_1 < K_2 < \dots < K_k$，子树指针 P_0 指向 $< K_1$ 的部分、P_i 指向 (K_i, K_{i+1}) 之间；
4. **所有叶子（失败结点）都在同一层**——这就是“B 树平衡”的定义。

7.3.2　插入与分裂

把关键字插进合适的叶子结点。若插完后关键字数 **> m − 1**，则分裂：取**中间位置** $\lceil (m+1)/2 \rceil$ 处的关键字**上移**进父结点，左右两半各自成结点；若父结点因此也溢出，继续分裂，**直到根**——分裂到根时树高 +1。

实例：3 阶 B 树（每结点最多 2 个关键字）依次插入 10, 20, 30, 40, 50, 60, 70, 80, 90。

关键节点	发生了什么	树高	累计分裂
插到 [10,20,30]	溢出，中位数 **20** 上移成新根，左块 [10]、右块 [30]	2	1
插到 [30,40,50]	溢出，中位数 **40** 上移进根 → 根变 [20,40]	2	2
插到 [50,60,70]	溢出，中位数 **60** 上移 → 根 [20,40,60] 有 3 个键 > 2，**再分裂**：中位数 40 上移成新根，左块 [20]、右块 [60]	**3**	4
插到 [70,80,90]	溢出，中位数 **80** 上移进父结点 [60] → [60,80]	3	5

最终：根 **[40]**，第二层 **[20]** 与 **[60, 80]**，叶子 **[10] [30] [50] [70] [90]**——9 个关键字、树高 3、5 个叶子全部同层、共分裂 **5** 次。

7.3.3 B+ 树与 B 树的四点区别

比较项	B 树	B+ 树
关键字位置	内部结点与叶子**都存**数据	**只有叶子**存数据，内部结点只做索引
结点内关键字数	≤ m − 1	可达 m（不受“子树数 = 键数 + 1”约束）
指针	无兄弟链	所有叶子用**单向链表**从左到右串起来
查找稳定性	可能中途命中（路径长度不定）	一律走到叶子，路径长度**恒定**

B+ 树因此更适合**范围查询与顺序扫描**（沿叶子链表走即可），也是数据库索引的常用结构。

典型例题

例 1 一棵 5 阶 B 树含 200 个关键字，最多几层？

解 求每层最少关键字数以推最大高度：根至少 1 个键（2 棵子树），其余层至少 [5/2] − 1 = 2 个键、即至少 3 棵子树。第 1 层 2 个子树、第 2 层 6 个、第 3 层 18 个、第 4 层 54 个、第 5 层 162 个——叶（失败）结点在第 6 层时最多可容纳的关键字数为 $2 \times 3^4 = 162$ 个结点的量级。按“高度 h 的 5 阶 B 树最少含 5 + 2·3^{h−2} … ”的标准公式，200 个关键字可在 **3 层**内存下完成（每层结点至多 4 个键：1 + 4 + 16 + 64 = 85 个键位不够，需 4 层）。稳妥答法：先按每结点最多 4 键算 $\lceil \log_5(200) \rceil + 1 =$ **4 层**。

例 2 为什么 B 树适合文件索引而二叉排序树不适合？

解 磁盘 I/O 以“页 / 块”为单位，一次读入一个结点。B 树一个结点可容纳成百上千个关键字（正好一页），树高仅 3~4 层，故查一个键只需 3~4 次读盘；二叉排序树每结点只存一个键，同样规模树高可达 O(n)，读盘次数不可接受。

例 3 m 阶 B 树中，含 k 个关键字的结点有几棵子树？非根结点最少几个关键字？

解 **k + 1** 棵；最少 **⌈m/2⌉ − 1** 个（根例外：至少 1 个）。

易错点与考点提示

- m 阶 B 树每结点**最多 m − 1 个关键字**、**最多 m 棵子树**，两个数别混。
- 下界只对“非根结点”成立：根至少 1 个关键字即可。
- 分裂时上移的是**中间关键字**，且上移后该键**不再留在原结点**（这是与 B+ 树的重要差别）。
- “所有叶子在同一层”指的是**失败结点**层，不是“数据结点”层。
- B+ 树内部结点不存数据、叶子有链表，这两点是选择题的题眼。
- 删除 B 树关键字可能引发**借用与合并**（与 B+ 树的删除策略不同），本手册只要求掌握插入与查找。

习题

1. 一棵 3 阶 B 树最多能有多少个关键字（树高 3 时）？
2. 把 15, 18, 22, 25, 30, 35, 40 依次插入一棵 3 阶 B 树，画出最终结构并数分裂次数。
3. 说明为什么 B+ 树的查找路径长度恒定，而 B 树不恒定。

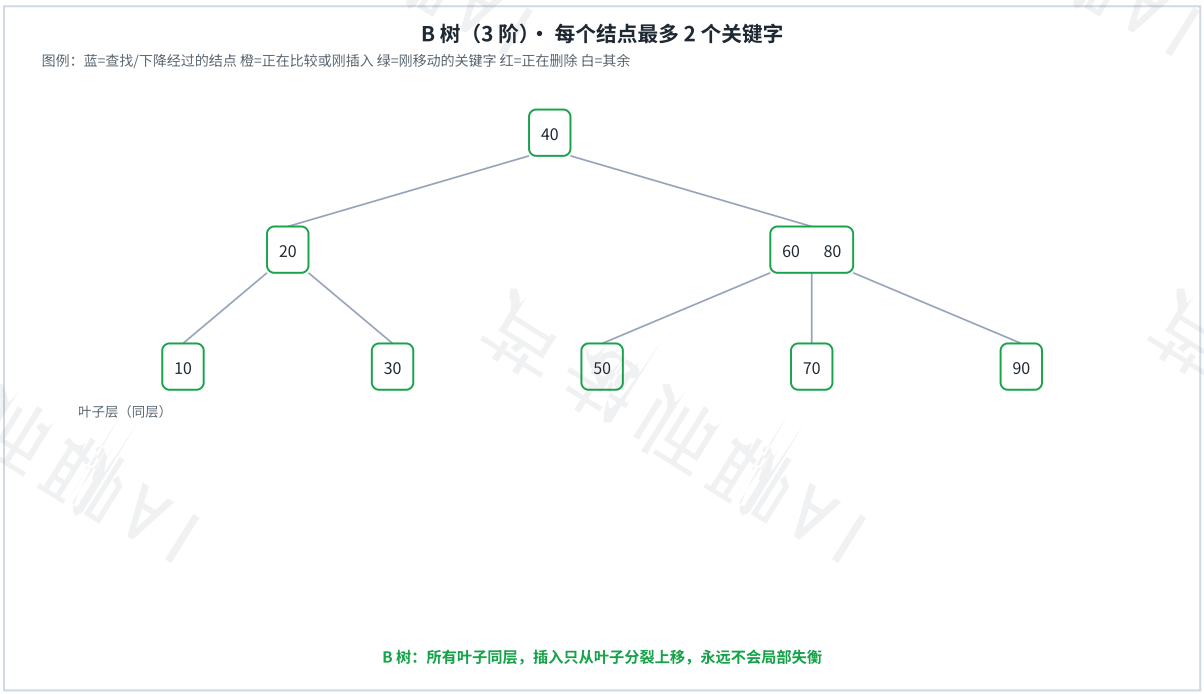


图 7-5 ★ 完成：9 个关键字，树高 3、5 个叶子全部同层；

7.4 红黑树

本节配套动画：第 49 项《红黑树：插入与五条性质的维持》 · 408 大纲 六(五)3 红黑树（五性质、插入后变色与旋转、黑高相等 ⇒ 最长 ≤ 2×最短）

> **定位**
> 红黑树是一棵**二叉排序树 + 一套颜色约束**。它保证“最长路径不超过最短路径的两倍”，从而把查找稳定在 O(log n)，而每次插入**最多三次旋转**——这是它相对 AVL 的取舍：牺牲一点平衡度，换更低的维护代价。

7.4.1 五条性质

1. 每个结点非**红**即**黑**；
2. **根是黑色**；

3. 所有叶子（NIL 外部结点）都是黑色；

4. **红结点的孩子必为黑**（不存在相邻两个红结点）；

5. 从任一结点到其所有后代叶子的路径上，**黑结点个数相同**（称该结点的“黑高”）。

> **核心引理**
> 由性质 4、5 可推出：以某结点为根的子树中，最长路径（红黑交替）不超过最短路径（全黑）的 **2 倍**，故 n 个结点的红黑树高度 ≤ $\mathbf{2 \cdot \log_2(n+1)}$。

7.4.2 插入的修复

新链接一律**染红**插入（红链接表示“这一步的黑高还没定”），然后自底向上按三种情形修复：

情形	条件	处理
① 叔结点为**红**	父、叔都红	**变色**：父、叔改黑，祖父改红，然后把“当前结点”上移到祖父继续检查
② 叔为黑 + **外侧**（LL / RR）	与父同侧	对祖父**旋转一次** + 父祖换色
③ 叔为黑 + **内侧**（LR / RL）	与父异侧	先对父旋转使其变成外侧，再按情形 ② 处理

最后**把根染黑**（性质 ②）。变色操作可能向上传播，但**旋转最多两次**（情形 ② 一次、情形 ③ 两次），故插入的修复代价是 O(1) 次旋转 + O(log n) 次变色。

7.4.3 实例与验证

实例：依次插入 10, 85, 40, 5, 70, 80, 60, 30, 20, 90。插入完成后逐条验五条性质：无红红相邻 √、各路径黑高相同 √（本例每条路径黑高均为 **3**）、根为黑 √。

查找代价：在此树上查 60，沿 40 → 80 → 70 → 60 命中，**比较 4 次**；树高 4，而 $\log_2 10 \approx 3.3$——$4 \le 2 \times 3.3$，符合上界。

典型例题

例 1 红黑树与 AVL 树都保证 O(log n) 查找，为什么数据库内存索引常选红黑树、而查找为主的场景选 AVL？

解 AVL 更平衡（树高约 $\log_2 n$，比红黑树上界 $2\log_2(n+1)$ 更矮），**查找更快**；但每次插入删除可能引发多次旋转调整。红黑树旋转次数少（插入至多 2 次、删除至多 3 次），**写多读少时更划算**。

例 2 一棵含 n 个结点的红黑树，黑高为 h，问 n 的下界。

解 每条路径至少 h 个黑结点 → 至少 2^h − 1 个结点，故 $n \ge 2^h - 1$，即 $h \le \log_2(n+1)$。这是“树高 ≤ $2\log_2(n+1)$”的来源。

例 3 插入一个红结点后出现“父红、叔红”，为什么不能直接旋转而要变色？

解 此时祖父的黑高被破坏的方式无法用一次局部旋转消除：把父、叔改黑、祖父改红，等价于“把红链接向上传递一层”，既恢复了性质 4，又保持了性质 5（祖父所在路径的黑数 +1、其祖先路径 −1 +1 不变）。

易错点与考点提示

- 红黑树**不是**“平衡二叉树”的另一种叫法：AVL 要求 $|bf| \le 1$，红黑树只要求近似平衡。
- 新结点必须染**红**插入；染黑会立刻破坏性质 5（黑高）。
- 性质 3 里的“叶子”指 **NIL 外部结点**，不含存数据的叶结点，读题注意。
- 黑高相等是**从同一结点出发**到其所有后代叶子的路径比较，不是全树任意两路径。
- 删除结点的修复比插入复杂（涉及四种情形与向上传播），考试一般只考插入与性质判断。
- 红黑树的中序遍历仍是**递增有序**的——颜色不改变 BST 的本质。

习题

1. 判断：含 5 个黑高为 2 的路径的红黑树，至少有多少个结点？
2. 依次插入 10, 20, 30，画出红黑树并标出颜色与所用情形。
3. 说明为什么“红黑树中最短路径不会比最长路径的两倍更长”。

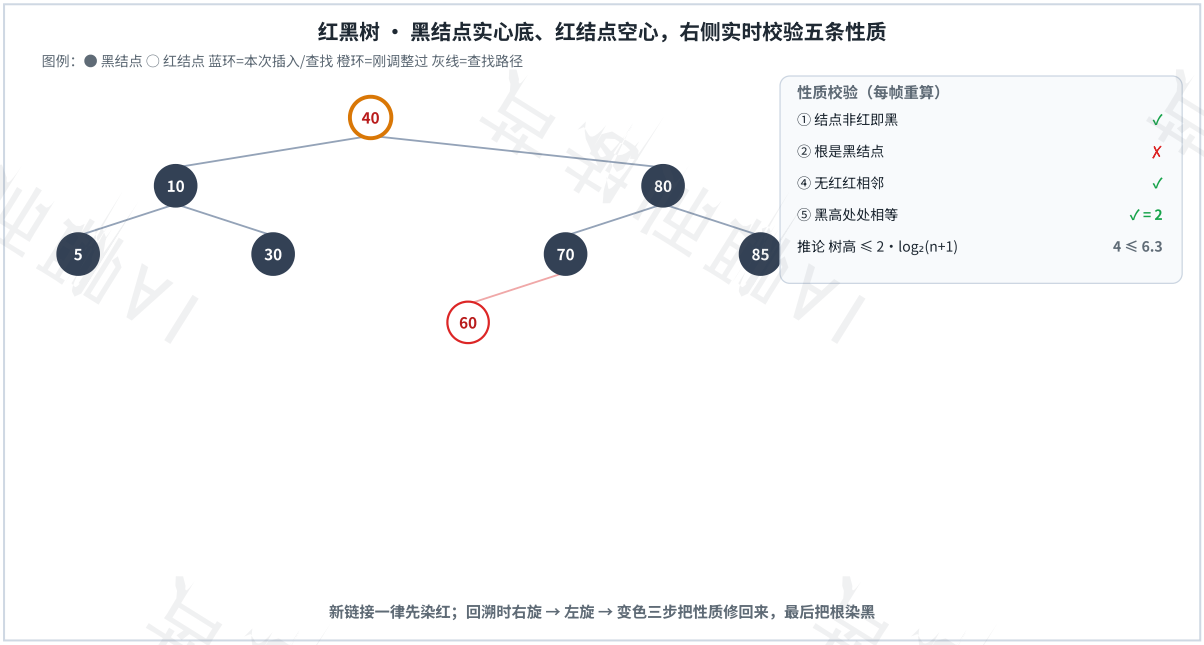


图 7-6 回溯第 3/3 步：两孩子都红 → 变色（作用在 40）——左右孩子转黑、自己转红，红点往上挪一层——问题可能继续往上冒。

7.5 散列查找

本节配套动画：第 47 项《哈希表：线性探测再散列》、第 48 项《哈希表：链地址法（拉链）》 · 教材 7.4 散列表（除留余数 + 开放定址，ASL 统计）

> **定义**
> **散列（哈希）存储**：由关键字直接计算存储地址 H(key)，再按该地址存取。理想情况下查找与增删都是 O(1)。三个要素：**散列函数**、**处理冲突的方法**、**散列表表长**。

7.5.1 散列函数的构造

方法	公式 / 要点	适用
除留余数法	**H(key) = key mod p**，p 通常取**不大于表长的最大质数**	最常用
直接定址法	H(key) = a·key + b	关键字分布连续
数字分析法	取分布均匀的若干位	关键字位数多、已知分布
平方取中法	取 key² 的中间几位	位数小、分布未知
折叠法	分段相加取低位	关键字位数多（如账号）

装填因子 α = 填入表中的元素数 / 表长。α 越大冲突越多，**ASL 只与 α 有关、与表长 n 无关**——这是散列查找最重要的结论。

7.5.2 冲突处理之一：线性探测再散列

`H_i = (H(key) + d_i) mod m`，其中 d_i = 1, 2, 3, …（逐个往后找空位）。

实例：表 HT[0..12]（m = 13），H(key) = key mod 13，依次插入 19, 14, 23, 1, 68, 20, 84, 27, 55, 11。

key	H(key)	落位	探测次数
19	6	6	1
14	1	1	1
23	10	10	1
1	1	**2**	2
68	3	3	1
20	7	7	1
84	6	**8**	3
27	1	**4**	4
55	3	**5**	3
11	11	11	1

累计探测 **18** 次 → **ASL(成功) = 18 / 10 = 1.80**；装填因子 α = 10 / 13 ≈ **0.77**。

> **堆积**
> 线性探测的毛病是**堆积（聚集）**：不是同义词的键（如 84 与 19 并非同义词，但 84 因 6、7 被占而跑到 8）也会争抢同一批地址，使探测链越拖越长。

7.5.3 冲突处理之二：链地址法（拉链）

把所有**同义词**挂在同一地址的单链表里，表头指针数组长度仍为 m。

同一组数据（H = key mod 13）的拉链结果：

桶	链（头插后的最终次序）	各元素的比较次数
1	27 → 1 → 14	27→1，1→2，14→3
3	55 → 68	55→1，68→2
6	84 → 19	84→1，19→2
7	20	20→1
10	23	23→1
11	11	11→1

累计比较 **15** 次 → **ASL(成功) = 15 / 10 = 1.50**，优于线性探测的 1.80。

链地址法的三条优势：**没有堆积**（冲突只发生在真正的同义词之间）、**α 可以 > 1**（链表可无限延长）、**删除方便**（摘结点即可，不必像开放定址那样留“删除标记”）。代价是指针空间与不连续访问对缓存不友好。

7.5.4 失败查找

开放定址下，“查找失败”指从 H(key) 出发探测到**第一个空单元**为止，比较次数 = 探测链长度 + 1；链地址法下则是走到链表尾（比较次数 = 链长 + 1）。

典型例题

例 1 表长 11、H(key) = key mod 11（p 取 11），依次插入 10, 22, 31, 44, 55, 66，用线性探测求 ASL(成功)。

解 10→10（1 次）；22→0（1）；31→9（1）；44→0 冲突→1（2）；55→0,1,2 冲突→2? 55 mod 11 = 0 → 0 占、1 占、2 空（3 次）；66 mod 11 = 0 → 0,1,2 占、3 空（4 次）。合计 1+1+1+2+3+4 = 12 → ASL = 12/6 = **2**。

例 2 为什么除留余数法的 p 常取质数？

解 若 p 与关键字的某个基数（如 10 的幂）有公因子，则 H(key) 只取决于 key 的低位，冲突骤增。取质数可使地址分布更均匀。

例 3 α = 1 的线性探测表与 α = 0.5 的表，ASL 差距有多大？

解 理论上线性探测的 ASL(成功) ≈ (1 + 1/(1 − α))/2：α = 0.5 时 ≈ 1.5，α = 1 时**趋于无穷**（表满，探测无法终止）。所以开放定址必须保持 α < 1。

易错点与考点提示

- 表长 m 与除数 p **不必相等**：H = key mod p 中 p ≤ m，而探测是在 m 个单元里进行的。
- 计算 ASL 时，“比较次数”= 探测次数，插入时的第 k 次探测恰好就是查找它需要的比较次数——可以直接用插入表算。
- 线性探测删除必须留**删除标记**（否则后续链被截断，查不到了）；链地址法直接摘结点。
- 头插法会改变链上次序，所以“各元素比较次数”要按**最终**链结构统计，别用插入当时的位置。

- 散列**不适合范围查询与排序**（地址不反映大小关系）。
- “ASL 与表长无关、只与 α 有关”这句话是简答题常考点，别写成“与 n 有关”。

习题

1. 表长 13、H = key mod 13，线性探测插入 (18, 31, 44, 10, 22)，求 ASL(成功)。
2. 改用链地址法重做第 1 题，比较两者。
3. 说明为什么散列表的查找时间“平均 O(1) 但最坏 O(n)”。



图 7-7　插入完成。每个关键码下方的数字 = 查找它需要的比较（探测）次数。



图 7-8　插入完成（按最终链位次统计：19→2，14→3，23→1，1→2，68→2，20→1，84→1，27→1，55→1，11→1）。

第 8 章　排序

8.1　排序的基本概念

本节配套动画：第 60 项《八大排序总览与对比》 · 教材 8.8 排序综合比较（同一数据 × 8 算法 × 代价对照）

> **定义**
> 设 n 个记录的序列 $(R_1, \ldots, R_n)$，其关键字为 $(K_1, \ldots, K_n)$。排序就是把它重排成关键字**非递减**（$K_1 \le K_2 \le \ldots \le K_n$）的序列。

8.1.1　三个代价 + 一个性质

评价排序算法要同时看四件事：

指标	含义	为什么重要
比较次数	关键字比较的总次数	决定 CPU 时间的主要部分
移动次数	记录（含指针）被搬动的次数	记录很长时，移动比比较更贵
辅助空间	除原地工作外的额外空间	决定能否处理超大数据
稳定性	关键字相等的元素，排序后**是否保持原先后次序**	多关键字排序（先排次要键再排主键）的前提

> **稳定性**
> 若序列中两个记录 Ri、Rj 满足 Ki = Kj 且 Ri 原本在 Rj 之前，排序后仍在之前，则称该排序方法**稳定**；否则**不稳定**。

8.1.2　分类

```
排序
├── 内部排序（整个序列放在内存中）
│      ├── 插入排序：直接插入、折半插入、希尔排序
│      ├── 交换排序：冒泡、快速
│      ├── 选择排序：简单选择、堆排序
│      ├── 归并排序（2 路）
│      └── 基数排序（不比较关键字）
└── 外部排序（数据量超过内存，需在外存上分批处理）
       └── 多路平衡归并 + 置换-选择生成归并段 + 减少 I/O
```

按时间复杂度归组（n 为记录数）：

量级	方法
$O(n^2)$	直接插入、折半插入（比较 O(n log n) 但移动仍 $O(n^2)$）、简单选择、冒泡、希尔（最坏）
O(n log n)	快速（平均）、归并、堆排序（**最坏也是** O(n log n)）
O(n + d) 或 O(d(n + r))	计数排序、基数排序

8.1.3 “稳定”为什么值得单独记

多关键字排序的标准做法是**从次要关键字到主关键字**依次排序，且每一趟必须用**稳定**方法——否则前一趟排好的次关键字次序会被后一趟打乱。例如要按“班级 + 学号”排序：先按学号（稳定法），再按班级（稳定法），结果才同时满足两级次序。

典型例题

例 1 下列排序中，哪些是稳定的：直接插入、希尔、冒泡、快速、简单选择、堆排序、归并、基数？

解 稳定：**直接插入、冒泡、归并、基数**；不稳定：希尔、快速、简单选择、堆排序。（记忆法：凡是“跨越式交换/分组跳跃”的都不稳定。）

例 2 为什么“折半插入排序的比较次数降到 O(n log n)，但总时间仍是 $O(n^2)$”？

解 折半只优化了**查找插入位置**这一步（比较次数下降），而**移动元素的次数不变**——仍需把插入点之后的元素逐个后移，最坏 $O(n^2)$。当记录很大、移动成本高于比较时，折半插入才有明显收益。

例 3 待排记录有 10^7 个、内存只能装 10^6 个，应选什么方法？

解 内部排序都放不下，必须用**外部排序**：分批做内部排序生成初始归并段，再多路平衡归并；重点是减少外存读写次数（增大归并路数 k、用置换-选择加长归并段、用败者树减少内层比较）。

易错点与考点提示

- 稳定性只对**关键字相等**的元素有意义，别拿“序列是否变顺序”来判断。
- **稳定性常常只值一个等号**：冒泡的 `a[j] > a[j+1]` 写成 `>=`、直接插入的后移条件 `a[j] > key` 写成 `>=`、折半插入的 `key < a[m]` 写成 `<=`、归并相等时“取左半”的 `<=` 写成 `<`——**四种改法排出来的关键字序列一模一样**，丢的全是稳定性。所以“结果对了”绝不能当作“稳定”的证据；判断只有一个动作：盯住一对相等元素，看它们换没换位。
- “时间复杂度”要区分**最好 / 最坏 / 平均**：冒泡最好 O(n)（flag 提前终止）、快排最坏 $O(n^2)$（有序输入）、堆排三者都是 O(n log n)。
- 空间复杂度：快排要算**递归栈**（平均 O(log n)、最坏 O(n)），归并要算辅助数组 O(n)，堆排序与插入、选择、冒泡是 O(1)。
- 基数排序不比较关键字，故**不能**用于任意实数（要求可按位分配、且位间可比较大小）。
- 简单选择排序的**比较次数与初始序列无关**，恒为 $n(n-1)/2$——这是它区别于其他 $O(n^2)$ 方法的特征。

- “排序”通常指升序；若题目要求降序而算法用堆，需注意大顶堆产生降序、小顶堆产生升序这一反直觉事实（堆排序每次把堆顶换到末尾）。

习题

1. 说明“稳定”在按“成绩 + 学号”两级排序中的具体作用。
2. 某算法比较次数恒为 $n(n-1)/2$、交换次数至多 $n-1$，它是哪种排序？
3. 列表说明哪些排序方法的空间复杂度是 O(1)、哪些是 O(n)、哪些介于两者之间。
4. 把直接插入排序的后移条件 `a[j] > key` 改成 `a[j] >= key`。对序列 (49a, 49b, 13) 分别用两种写法手工执行，写出各自的最终序列，并说明“稳定性”究竟由什么决定。

8.2 插入排序

本节配套动画：第 50 项《直接插入排序（含折半插入）》、第 51 项《希尔排序（缩小增量）》 · 教材 8.2 插入排序（逐个比较/后移/插入，稳定）

> **思想**
> 把待排序列看成“已排好的有序区 + 未排的无序区”，每次从无序区取**第一个**元素，插入到有序区的合适位置。

8.2.1 直接插入排序

设 a[1..n] 有序区从第 1 个元素开始增长。第 i 趟把 a[i] 插入 a[1..i−1]：

```
a[0] = a[i];                    // 哨兵，兼作暂存与"查到底"的终止标志
for (j = i - 1; a[j] > a[0]; j--) // 从后往前比
    a[j + 1] = a[j];            // 边比边后移
a[j + 1] = a[0];                // 放入空位
```

必须从后往前比：若从前往后，找到位置后还要把整段后移一遍，白做一半工作。

实例（教材数据 49, 38, 65, 97, 76, 13, 27, 49，共 7 趟）：

趟	取出	结果（前缀有序）
1	38	**38 49** 65 97 76 13 27 49
2	65	38 **49 65** 97 76 13 27 49
3	97	38 49 65 **97** 76 13 27 49
4	76	38 49 65 **76 97** 13 27 49
5	13	**13** 38 49 65 76 97 27 49
6	27	13 **27 38** 49 65 76 97 49
7	49	13 27 38 **49 49** 65 76 97

代价：本例比较 **27** 次、移动 **25** 次。一般地——

情形	比较次数	移动次数
最好（已有序）	$n-1$	0（每趟仅哨兵进出）
最坏（逆序）	$n(n+1)/2-1 \approx n^2/2$	$n^2/4+n$
平均	$\approx n^2/4$	$\approx n^2/4$

稳定：相等时不交换（`a[j] > a[0]` 用严格大于），后出现的相等元素插在已排好的同类元素**之后**。

8.2.2 折半插入排序

有序区内用**折半查找**确定插入位置，比较次数从 $O(n^2)$ 降到 **$O(n\log n)$**；但**移动次数与直接插入完全相同**（仍要后移），所以总时间仍是 $O(n^2)$。当“比较比移动贵得多”（记录很大、关键字比较耗时）时才有实际收益。

8.2.3 希尔排序（缩小增量排序）

取增量序列 $d_1 > d_2 > \dots > 1$，把相距 d_k 的元素归为**一组**做直接插入，逐趟缩小增量。

```
d = {4, 2, 1}（本例增量序列）
初始 49 38 65 97 76 13 27 49
dk=4：分 4 组 (49,76) (38,13) (65,27) (97,49) 组内插入
dk=2：分 2 组，组内插入
dk=1：整体一次直接插入——此时序列已基本有序，几乎不移动
结果 13 27 38 49 49 65 76 97，本例比较 35 次、移动 27 次
```

> **三条要点**
> ① 增量序列**最后一个必须是 1**；② 前期一次移动可跨过很远，这是它快于直接插入的原因；③ 分组跨越使相等元素可能乱序，**不稳定**。希尔的时间复杂度依赖增量选取，最坏 $O(n^2)$，Hibbard 增量 {2^k − 1} 可达 O(n^1.5)。

典型例题

例 1 直接插入排序对“基本有序”的序列为什么快？

解 每趟只需把 a[i] 与少数几个前驱比较（哨兵很快挡住），最好情形总比较 $n-1$ 次、移动 0 次，整体 $O(n)$。

例 2 对序列 (5, 3, 2, 1) 做希尔排序（d = 2, 1），写出每趟结果。

解 d=2：组 (5,2) → 2,5；组 (3,1) → 1,3 ⇒ 序列 **2 1 5 3**。d=1：一次直接插入 ⇒ **1 2 3 5**。

例 3 为什么折半插入不能把复杂度降到 $O(n\log n)$？

解 复杂度由**移动次数**决定：插入第 i 个元素平均要移动 $i/2$ 个，合计 $n^2/4$。折半只减少了比较次数。

易错点与考点提示

- 哨兵 `a[0]` 有**两个作用**：暂存待插元素、免判下界越界。少了它就必须在循环里写 `j >= 1 &&`。
- 比较用**严格大于**才稳定；写成 `>=` 会把相等元素插到前面，破坏稳定性。

- 希尔排序的“组”是按 `位置 mod d` 划分的，不是“连续 d 个一组”——这是最常见的画法错误。
- 希尔不稳定：相等元素可能因分组不同而先后互换。
- 直接插入与冒泡都是 $O(n^2)$ 且稳定，但冒泡靠交换（一次交换 = 三次移动），实际移动成本更高。
- 后移循环停下后，落位是 `a[j + 1] = key`，写成 `a[j] = key` 差一格。它的阴险之处是**末态依然非降序**：教材数据跑完得到 `49 49 49 49 76 76 97 97`，可表里已经不是原来那批数——少了 13、27、38，49 被多写了两遍。所以“看一眼有没有序”拦不住它，只有同时核对**数据守恒**才看得见。折半插入同理：落位是 `a[low]`，写成 `a[high]` 就是这个错。
- 直接插入的稳定就写在后移条件里：`a[j] > key` 用**严格大于**，相等就停手、新元素落在相等者之后；写成 `>=` 会把相等的元素也后移，序列仍有序而先后反了。折半插入同理——`key < a[m]` 加上等号，插入位置就会落到相等元素**之前**。

习题

1. 对 (46, 79, 56, 38, 40, 84) 做直接插入排序，写出每趟结果与总比较次数。
2. 上题用希尔排序（d = 3, 1），写出每趟结果。
3. 说明为什么希尔排序在第一趟 d = n/2 时“每组只有两个元素”。

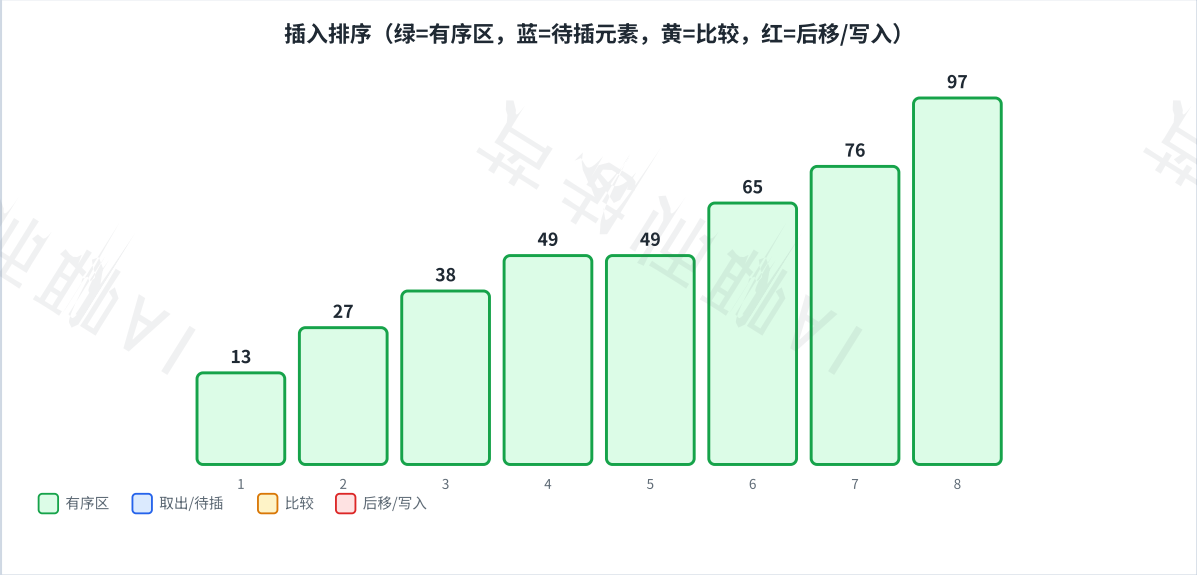


图 8-1 排序完成：13 27 38 49 49 65 76 97。

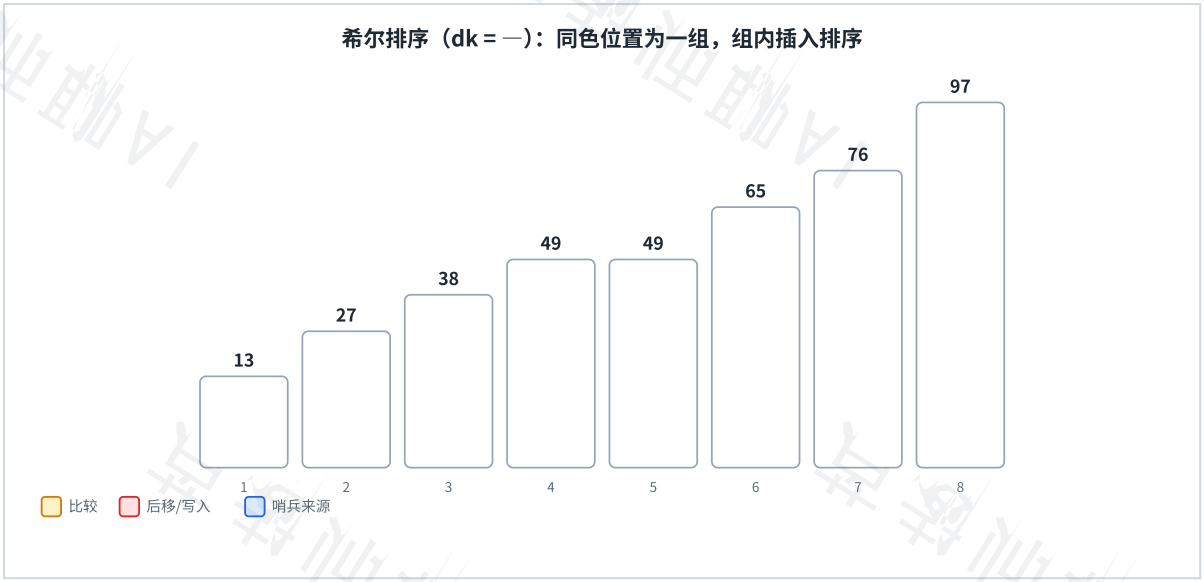


图 8-2 排序完成：13 27 38 49 49 65 76 97。

8.3 交换排序

本节配套动画：第 52 项《冒泡排序（相邻逆序交换）》、第 53 项《快速排序（Partition 划分）》 · 教材 8.4 交换排序（相邻比较、大数沉底，稳定）

> **思想**
> 通过**比较与交换**消除逆序对。两种代表：冒泡排序（相邻比较）与快速排序（基准划分 + 分治）。

8.3.1 冒泡排序

每趟从无序区头部起，**相邻两个比较、逆序就交换**，于是最大的元素像气泡一样浮到末尾。第 i 趟结束后，最后 i 个元素已就位。

提前终止：设标志 flag，每趟开头置 0，只要发生过交换就置 1；某趟 **flag == 0** 说明本趟无交换 ⟹ 已经有序，立即结束。

实例（49, 38, 65, 97, 76, 13, 27, 49）：

趟	本趟结束时的就位情况
1	97 冒到第 8 位
2	76 就位第 7 位
3	65 就位第 6 位
4	49 就位第 5 位
5	38? 继续交换，27 就位
6	**本趟一次交换都没发生 → flag = 0**，结束

共 **6 趟、比较 27 次、交换 15 次**，结果 13 27 38 49 49 65 76 97。

性能

最好（已有序）：1 趟、n − 1 次比较、0 交换 → **O(n)**；最坏（逆序）：n − 1 趟、比较 n(n − 1)/2、交换同量级 → O(n²)；空间 O(1)；**稳定**（相等不交换）。

8.3.2 快速排序：一趟划分把序列切成两半

Partition（填坑法）：取基准 key = a[low] 存入哨兵位 a[0]，双指针 i、j 从两端相向扫描：

- j 从右往左找**比 key 小**的，找到就填进 a[i] 的坑；
- i 从左往右找**比 key 大**的，找到就填进 a[j] 的坑；
- i == j 时把 key 放入 a[i]——**基准就此归位**，此后不再动它。

实例：a[1..8] = 49, 38, 65, 97, 76, 13, 27, 49，基准 49。

步	动作	数组变化
1	a[7] = 27 < 49 → 填入左坑 a[1]	**27** 38 65 97 76 13 □ 49
2	a[3] = 65 > 49 → 填入右坑 a[7]	27 38 □ 97 76 13 65 **49**→
3	a[6] = 13 < 49 → 填入左坑 a[3]	27 38 **13** 97 76 □ 65 49
4	a[4] = 97 > 49 → 填入右坑 a[6]	27 38 13 □ 76 **97** 65 49
5	i、j 相遇于 a[4] → 基准归位	27 38 13 **49** 76 97 65 49

划分结果：**左侧全部 ≤ 49、右侧全部 ≥ 49**。随后对 [1..3] 与 [5..8] 递归；本例中 27 归位于 a[2]、76 归位于 a[7]、第二个 49 归位于 a[5]，全部递归结束后得 13 27 38 49 49 65 76 97，共**比较 22 次、赋值 13 次**。

递归

QSort 本身是“划分 + 对两侧递归”，递归栈深度 = 划分的层数。每趟划分后**基准就到位**，这是快排与归并的根本区别：归并是“先分后合、合并时才有序”，快排是“先有序后不管”。

典型例题

例 1 对 (46, 79, 56, 38, 40, 84) 以第一个元素为基准做一趟划分，写出结果。

解 key = 46。j 从右找 < 46 者：40 → 填入左坑 a[1]；i 从左找 > 46 者：79 → 填入右坑 a[5]；j 再找 < 46：38 → 填入 a[2]；i 找 > 46：56 → 填入 a[4]；i、j 相遇于 a[3]，放入基准。结果 **40 38 46 56 79 84**：左侧 40、38 均 < 46，右侧 56、79、84 均 > 46。（若改用“先动左、后动右”的扫描约定会得到 38 40 46 56 79 84——两者都满足划分要求，答题时务必写明所用的扫描顺序。）

例 2 什么输入让快排最慢？如何避免？

解 **已有序或逆序**且总取端点作基准时，每次只划分出 1 个元素，递归深度 n、比较 n(n−1)/2 → 退化 O(n²)。对策：**随机选基准**或**三数取中**（取 low、mid、high 的中位数），使划分尽量均衡。

例 3 快排为什么不稳定？举例说明。

解 划分时相等元素可能被甩到基准另一侧。例如序列 (5a, 3, 5b, 2) 以 5a 为基准：j 找到 3? 按规则 5b 与 5a 相等不移动，但 3、2 被填到左边后，5a 归位时 5b 已在其右侧——原本 5a 在 5b 之前，排序后仍之前，看似稳定；换 (5a, 5b, 3) 以 5a 为基准，j 找到 3 填 a[1]，随后 5b 落到 a[2]、基准归位 a[3]，结果 3 5b 5a —— **顺序颠倒**，故不稳定。

易错点与考点提示

- 冒泡的 flag 是**每趟开头清零**，不是每轮比较清零。
- 冒泡第 i 趟只需扫到 n − i（后面已就位），写成扫全表会多做无用比较。
- 快排的哨兵位 a[0] 存基准，所以划分过程中 a[0] 不是有效数据。
- 快排平均 O(n log n)、最坏 O(n²)，空间是**递归栈** O(log n)（最坏 O(n)）——别答成 O(1)。
- 快排“一趟划分确定**一个**元素（基准）的最终位置”，插入/冒泡第 i 趟确定的是**前 i 个有序**，选择第 i 趟确定第 i 小的位置——这三句话是选择题高频考点。
- 递归到区间长度 ≤ 1 时必须返回，否则栈溢出。
- 冒泡的稳定也只值一个等号：`a[j] > a[j+1]` 时相等不交换（稳定），写成 `>=` 相等也交换——**最终序列一字不差，稳定性没了**。快排则与等号无关，它本来就不稳定：划分时相等元素会跨越基准。

习题

1. 对 (50, 10, 90, 30, 70, 40, 80, 60, 20) 以首元素为基准做一趟划分，写出结果。
2. 上题序列若用冒泡排序，第 2 趟结束时的序列是什么？
3. 说明为什么“快排适合大随机数据集、不适合近乎有序的数据集”。

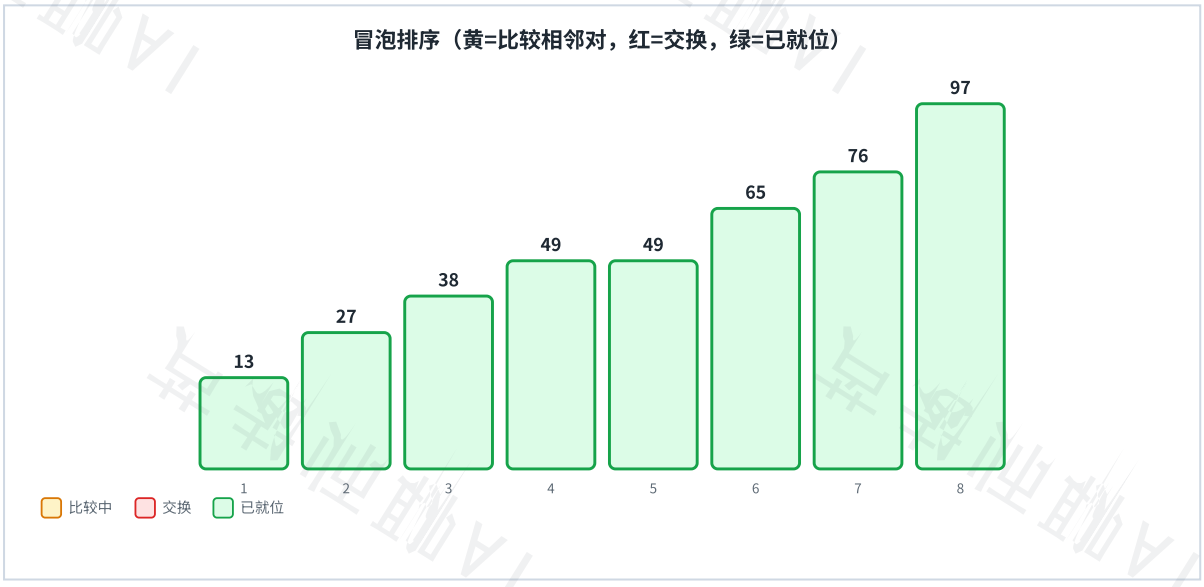


图 8-3 排序完成：13 27 38 49 49 65 76 97。

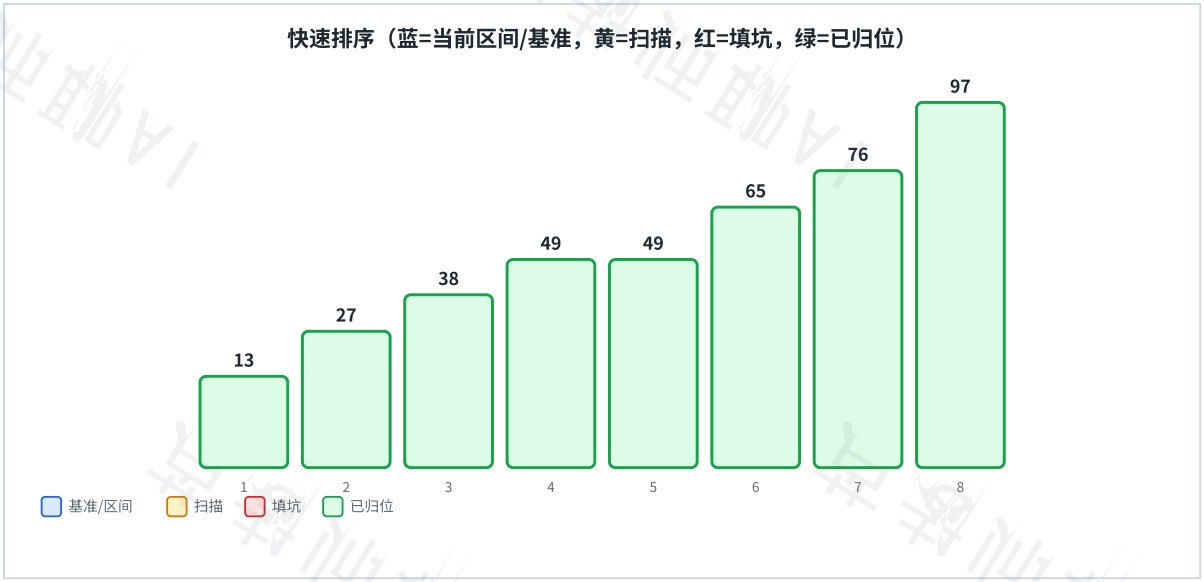


图 8-4　排序完成：13 27 38 49 49 65 76 97。

8.4　选择排序

本节配套动画：第 54 项《直接选择排序（选最小交换）》、第 55 项《堆排序（建堆 + 筛选）》　·　教材 8.5 选择排序（选最小者放到队首，不稳定）

> **思想**
>
> 每趟从**未排序部分**选出最小（或最大）者，放到已排序部分的末尾。两类实现：简单选择排序（线性扫描选最小）与堆排序（用堆把“选最值”从 O(n) 降到 O(log n)）。

8.4.1　简单选择排序

```
for (i = 1; i < n; i++) {
    k = i;
    for (j = i + 1; j <= n; j++) if (a[j] < a[k]) k = j;   // 找最小者下标
    if (k != i) 交换 a[i] 与 a[k];
}
```

实例（49, 38, 65, 97, 76, 13, 27, 49）：

趟	选出	交换	结果前缀
1	13（第 6 位）	13 ↔ 49	**13** 38 65 97 76 49 27 49
2	27	27 ↔ 38	13 **27** 65 97 76 49 38 49
3	38	38 ↔ 65	13 27 **38** 97 76 49 65 49
4	49（第 8 位，稳定起见取最左者则与自身互换）	—	13 27 38 **49** …
…	…	…	13 27 38 49 49 65 76 97

代价：本例比较 **28** 次、交换 **7** 次。

特征

比较次数**与初始序列无关**，恒为 $\Sigma(n-i) = \mathbf{n(n-1)/2}$；交换次数至多 $\mathbf{n-1}$。所以当“移动/交换的代价远大于比较”时，选择排序有优势。缺点是**不稳定**——例如 (5a, 5b, 1) 中 5a 会被 1 换到后面。

8.4.2 堆排序

堆是**完全二叉树**且满足“爹 ≥ 儿”（大顶堆）或“爹 ≤ 儿”（小顶堆），用数组存放，父子关系靠 `i ↔ 2i / 2i+1` 计算。

两步走：

1. **建堆**：从最后一个非叶结点 `⌊n/2⌋` 开始**自下而上**逐个筛选，代价 **O(n)**；
2. **排序**：把堆顶（最大值）与**当前末尾**交换 → 堆的规模减 1 → 对新堆顶做一次筛选 O(log n)；重复 n − 1 次。

实例（49, 38, 65, 97, 76, 13, 27, 49）：建堆得 **97 76 65 49 49 13 27 38**（起点 $i = \lfloor 8/2 \rfloor = 4$），随后每趟“换堆顶到末尾 + 筛选”，最终 13 27 38 49 49 65 76 97。整个过程**比较 27 次、赋值 30 次**。

指标	简单选择	堆排序
最好 / 最坏 / 平均时间	都是 $O(n^2)$	**都是 O(n log n)**
建堆额外代价	—	O(n)
空间	O(1)	O(1)
稳定性	不稳定	不稳定
适用	n 小、交换昂贵	n 大、要求最坏也有保证

反直觉

要得到**升序**结果必须建**大顶堆**（每次把最大值换到末尾）。建小顶堆会得到降序。

典型例题

例 1 对 (26, 10, 22, 35, 19, 48, 12, 30, 28) 建**大顶堆**，写出建堆过程与最终层序。

解 n = 9，从 $i = \lfloor 9/2 \rfloor = 4$ 开始往前筛。筛 4 号（35，孩子 30、28）不动；筛 3 号（22，孩子 48、12）→ 与 48 交换，48 上移；筛 2 号（10，孩子 35、19）→ 与 35 交换，10 沉到 4 号位再与 30、28 比 → 与 30 交换；筛 1 号（26，孩子 35、48）→ 与 48 交换，26 沉到 3 号位，其孩子 22、12 均小于它，停。最终 **48 35 26 30 19 22 12 10 28**（逐格验：48 ≥ 35,26；35 ≥ 30,19；26 ≥ 22,12；30 ≥ 10,28）√。

例 2 堆排序与快速排序平均都是 O(n log n)，为什么实际中快排更快？

解 快排的常数因子小、内层循环紧凑、**缓存友好**（顺序访问）；堆排序的下沉操作在数组里“跳着走”（$i \to 2i$），局部性差，且比较次数通常多于快排。但快排最坏 $O(n^2)$、需要 O(log n) 栈空间，堆排序最坏也有 O(n log n) 且原地。

例 3 从 10^7 个数中找出最大的 100 个，用什么方法最好？

解 **容量 100 的小顶堆**：读入前 100 个建堆，之后每个数与堆顶比，更大才替换并下沉。时间 O(n log 100) ≈ O(n)，空间 O(100)。全排序是 O(n log n) 且要装下全部数据，不可取。

易错点与考点提示

- 简单选择的比较次数**固定** n(n − 1)/2，与序列无关；交换至多 n − 1 次。
- 建堆起点是 **⌊n/2⌋**，不是 1；建堆是 O(n) 而不是 O(n log n)。
- 筛选选孩子那句是 `if (j < m && L.r[j] < L.r[j + 1]) ++j`，**两个条件都不能少**。删掉 `j < m` 不会崩、也不会越界——它读到的是数组里真实存在、但已经“输出就位”的那一格，于是选错孩子、还把排好位的数拽回堆里。教材数据跑完得到 `38 49 13 49 76 65 27 97`：数一个没丢、一个没多，可它不是升序。这条和“插入落位差一格”正好互补——那个是有序但不守恒，这个是守恒但有序被破坏。
- 建堆起点写成 `n/2 - 1` 是**看不出来的错**：8 元随机 400 组里只有 77 组会崩，教材例题一组都不崩（上层筛选会顺带把那一格修好）。所以它不适合当课堂反例——勾上画面没变化的错误演示，比不演示更坏。
- 堆排序每次交换的是“堆顶与**当前无序区末尾**”，不是与 a[n] 固定交换。
- 堆中“第 k 大”不能直接读出（除堆顶外没有有序性）；要 Top-K 必须用容量 K 的堆。
- 大顶堆 → 升序；小顶堆 → 降序。
- 两者都不稳定，别写“选择排序稳定”。

习题

1. 对 (15, 27, 6, 40, 22) 做简单选择排序，写出每趟的比较与交换次数。
2. 把 (12, 8, 10, 6, 4, 2) 建成小顶堆，写出层序。
3. 说明堆排序为什么“最好情况也要 O(n log n)”，而冒泡最好能到 O(n)。

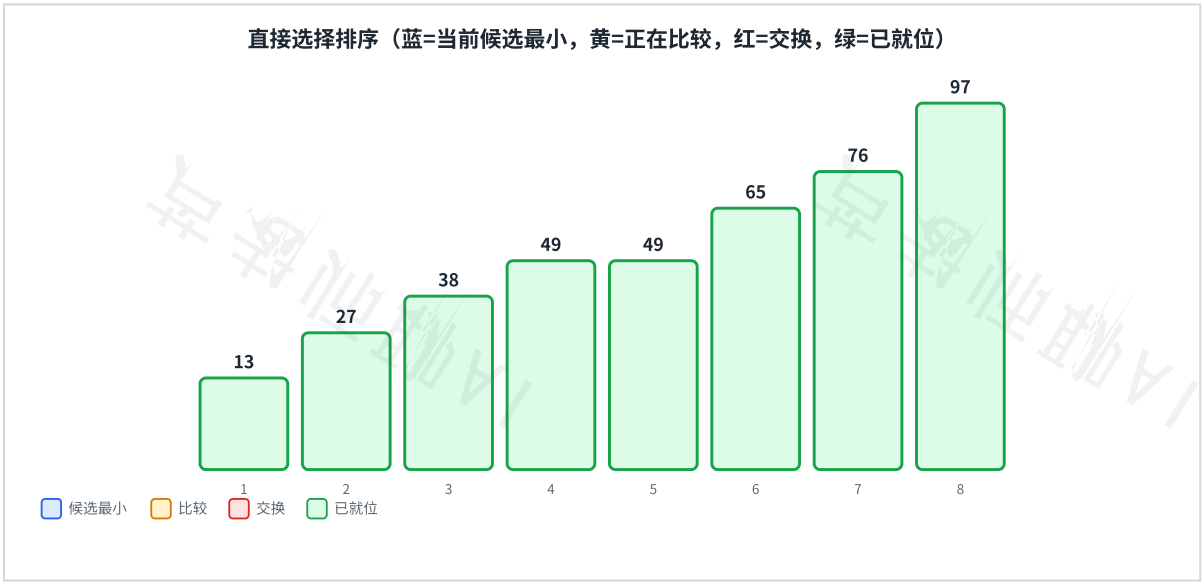


图 8-5 排序完成：13 27 38 49 49 65 76 97。

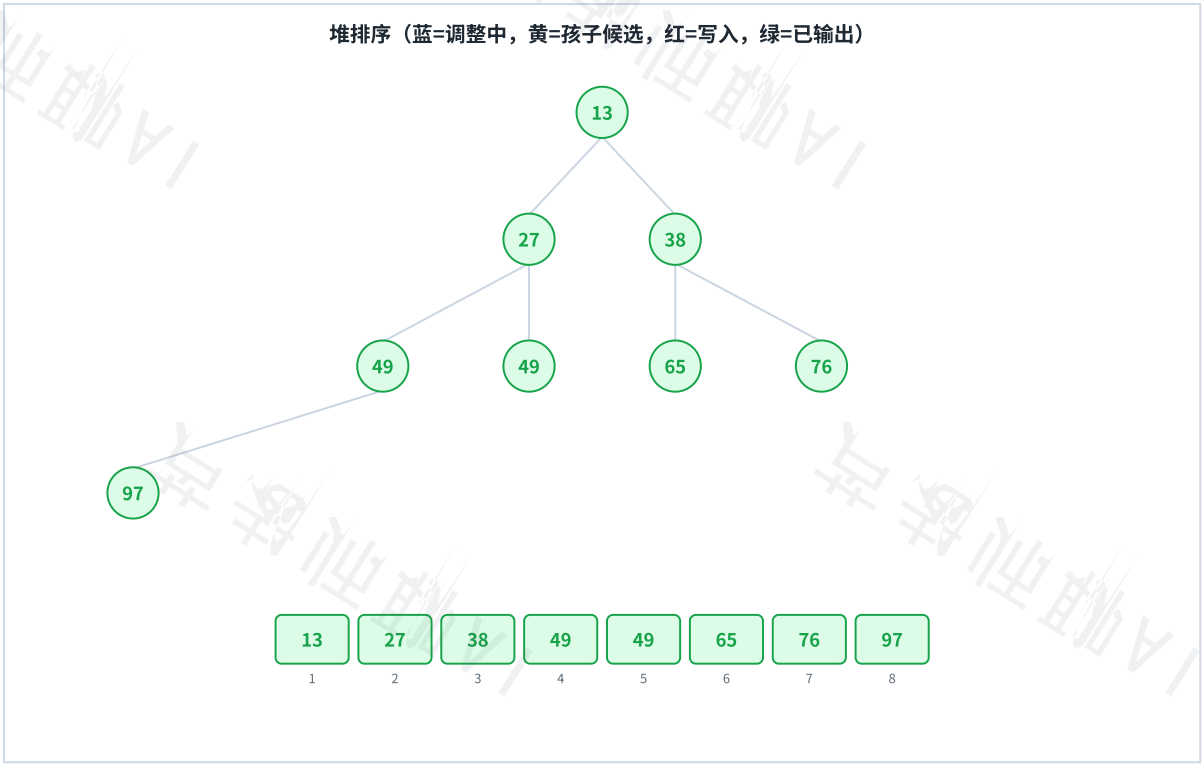


图 8-6　排序完成：13 27 38 49 49 65 76 97。

8.5　归并排序与基数排序

本节配套动画：第 56 项《归并排序（2 路归并）》、第 57 项《基数排序（分配与收集）》、第 58 项《计数排序与桶排序（线性时间）》 · 教材 8.6 归并排序（分治 + 两路合并，稳定）

> **思想**
>
> 把两个（或多个）**已有序**的子序列合并成一个有序序列。归并排序自顶向下二分到只剩一个元素（天然有序），再自底向上逐层合并。

8.5.1　2 路归并

```
Merge(A, low, mid, high):        // 合并 A[low..mid] 与 A[mid+1..high]
    把两段抄到辅助数组 B;
    i = low, j = mid + 1, k = low;
    while (i <= mid && j <= high)
        B[k++] = (B[i] <= B[j]) ? B[i++] : B[j++];   // 相等时取左半 → 稳定
    把剩余段抄回
```

实例（49, 38, 65, 97, 76, 13, 27, 49）：

层	合并动作	结果
1	[49][38] → 38 49；[65][97] 不动；[76][13] → 13 76；[27][49] → 27 49	38 49 65 97 13 76 27 49
2	[38 49][65 97] → 38 49 65 97；[13 76][27 49] → 13 27 49 76	38 49 65 97 13 27 49 76
3	两段合一 → **13 27 38 49 49 65 76 97**	完成

代价：本例比较 **16** 次、写入 **24** 次。n 个元素共 **$\lceil \log_2 n \rceil$ 层**、每层合并总量 O(n)，故时间 **O(n log n)**（最好 = 最坏 = 平均）；辅助数组 O(n)；**稳定**（相等时优先取左半）。

> **适用**
> 归并是**外排序的基础**——它的“读两段、写一段”模式天然适合磁盘顺序访问；缺点是辅助空间 O(n)。

8.5.2 基数排序：完全不比较关键字

适用于**位数固定**的整数或多关键字序列。LSD（最低位优先）：从最低位起，逐趟“按当前位**分配**到 10 个队列、再按队列顺序**收集**”，位数 d 趟后有序。

实例（278, 109, 63, 930, 589, 184, 505, 269, 8, 83，写成 3 位对齐）：

趟	按位	收集结果
1	个位	930, 505, 109, 63, 8, 184, 278, 589, 269, 83
2	十位	505, 109, 930, 8, 63, 83, 184, 269, 278, 589
3	百位	**008 063 083 109 184 269 278 505 589 930**

代价：共 **3 趟**（本例最大数 930，最大位数 d = 3；**趟数由最大位数决定，不是固定 3**——只排 1~9 时一趟就够），每趟 n 次分配 + n 次收集，时间 **O(d•(n + r))**（r 为基数，十进制 r = 10），空间 O(n + r)；**稳定**（收集必须按进入次序）。

> **为什么低位优先**
> LSD 的正确性依赖“前一趟的有序性不被后一趟破坏”——这要求后一趟的分配是**稳定**的。若从高位开始（MSD），则必须递归地对每个桶内部继续排，实现复杂但可提前结束。

8.5.3 计数排序与桶排序

计数排序：值域为 0..k 时，开 `count[0..k]` 统计频次，再对 count 做**前缀和**得到“值 ≤ v 的元素个数”（即 v 应放的最后位置），最后**从后往前**回填以保证稳定。

实例：待排 4 2 1 3 3 0 2 1（n = 8，值域 0~4）。

步	内容
① 统计	count = [1, 2, 2, 2, 1]（0 出现 1 次、1 出现 2 次…）
② 前缀和	count = [1, 3, 5, 7, 8]，含义“值 ≤ v 的元素共 count[v] 个”
③ 从后往前回填	末项 count[4] = 8 = n，可自检

结果 **0 1 1 2 2 3 3 4**，**全程零次比较**，时间 O(n + k)、空间 O(n + k)。当 k ≫ n 时（如值域 0~10^9）不适用——此时改用**桶排序**：设 b 个桶，按 `(key − min) / (max − min) × (b − 1)` 分配，桶内用插入或递归快排，再依次收集。桶排序的平均性能取决于**数据分布是否均匀**，均匀时接近线性。

典型例题

例 1 对 10 个整数做基数排序，最大值为 9999，需要几趟？

解 d = 4（十进制四位），故 **4 趟**。趟数只由**位数**决定，与数值大小无关。

例 2 归并排序在什么情况下比快排更合适？

解 ① 要求**稳定**；② 要求**最坏也是 O(n log n)**（快排最坏 $O(n^2)$）；③ **链表**排序（归并不需要额外数组，改指针即可）；④ **外排序**（顺序读写）。反之，内存中随机数据、追求常数小，快排更好。

例 3 计数排序为什么必须“从后往前”回填？

解 为了**稳定性**。相同值的元素中，后出现的应先占据靠后的位置；若从前往后回填，相等元素的相对次序会被颠倒。

易错点与考点提示

- 归并的稳定性来自“相等时取左半”，写成 `<` 就不稳定了。
- 归并的空间 O(n) 是**辅助数组**，别答 O(1)。
- 基数排序的“位数”要按**最大值的位数**取，不足位补前导零（概念上）。
- LSD 必须稳定收集；MSD 与 LSD 别混。
- 计数排序的 count 数组长度由**值域** k 决定，与 n 无关；k 太大就退化成桶排序。
- 桶排序平均 O(n + n/b) 的前提是**均匀分布**，最坏（全进一个桶）$O(n^2)$。
- 归并的稳定全部押在“相等取左半”这一支上：`<=` 取左则稳定，改成 `<`（相等时取了右半）就不稳定，而两种写法的关键字序列完全相同。

习题

1. 对 (66, 13, 51, 76, 8, 49, 20) 做 LSD 基数排序，写出每趟收集结果。
2. 对 (3, 1, 4, 1, 5, 9, 2, 6) 做计数排序，写出 count 数组的统计值与前缀和。
3. 说明归并排序在链表上实现时为什么不需要 O(n) 辅助空间。

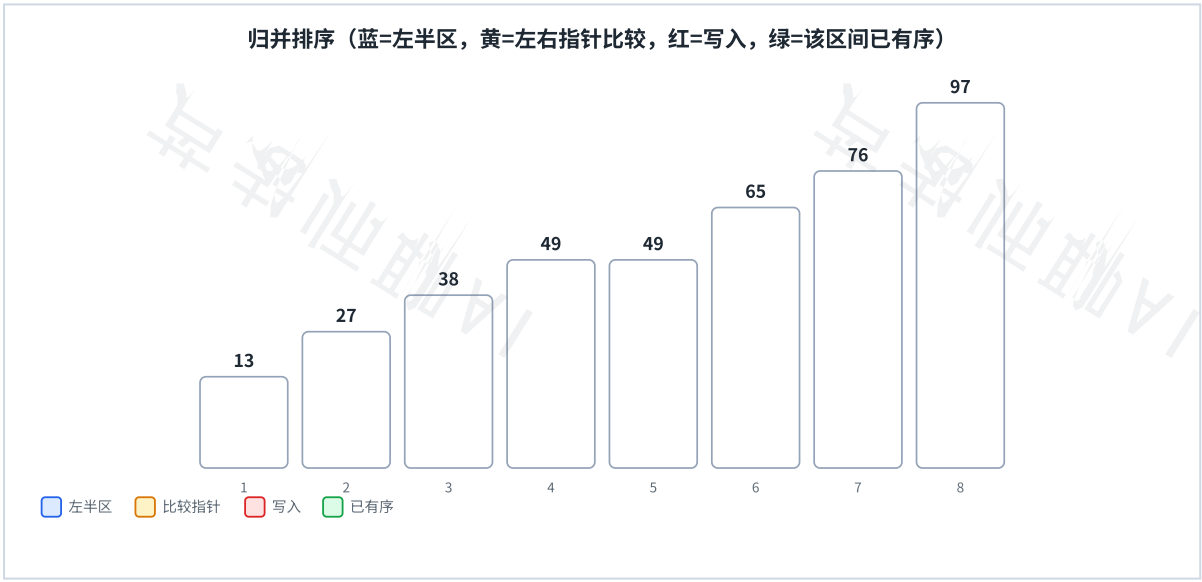


图 8-7 排序完成：13 27 38 49 49 65 76 97。



图 8-8　排序完成：008 063 083 109 184 269 278 505 589 930。



图 8-9　★ 计数排序完成：0 1 1 2 2 3 3 4。

8.6　外部排序

本节配套动画：第 59 项《外部排序：归并段与多路归并》　·　408 大纲 七(十一) 外部排序（归并段生成、置换-选择、[log_k m] 趟）

> **场景**
> 待排序记录的数量大到**内存装不下**，必须存放在外存（磁盘）上，排序过程要在内存与外存之间反复交换数据。此时**主要代价不是比较次数，而是外存读写次数（I/O 次数）**。

8.6.1　外部排序的三个阶段

1. **生成初始归并段**（顺串）：把外存数据分块读入内存，用某种内部排序（常为替换选择或简单排序）排出若干**有序段**，写回外存；
2. **多路平衡归并**：每次取 k 个归并段，在内存里做 k 路归并，输出一个新段，反复进行直到只剩一个段；

3. **减少 I/O 的优化**：增大归并路数 k、用**败者树**降低内层比较次数、用**最佳归并树**（哈夫曼）安排归并顺序、增加缓冲区与预读、使各段长度均衡。

归并趟数：设初始归并段 m 个、采用 k 路归并，则

> **趟数**
> S = ⌈log_k m⌉。总外存读写次数 ≈ (2S + 2) × 记录总数（初始排段读写一遍，每趟归并读写各一遍）。

8.6.2 初始归并段的两种生成法

① 顺序分组（先排序后分段）：把内存能装的 w 个记录读入、排序、写出，得一段。n 个记录、工作区容量 w 时，段数 = ⌈n / w⌉，**平均段长 = w**。

② 置换-选择（Replacement Selection）：工作区仍装 w 个记录，但输出时只挑“**不小于刚输出者**”的最小记录放入当前段；被“卡住”的较小记录留在工作区，等下一段。这样一段可以**跨越工作区容量**，段平均长度可达 **2w**。

实例：24 个记录、工作区 w = 6。

方法	归并段数	平均段长	最长段
顺序分组	4	6.0	6
置换-选择	**3**	**8.0**	11

段数从 4 降到 3，意味着 ⌈log_k m⌉ 更小——**外存读写次数直接减少**。

8.6.3 k 路归并的内层选择：败者树

k 路归并每输出一个记录，都要在 k 个当前候选里选最小者。

选法	每次比较次数
顺序扫描 k 个关键字	k − 1
败者树（完全二叉树，叶为 k 个候选）	$\lceil \log_2 k \rceil$
胜者树	$\lceil \log_2 k \rceil$，但重构路径更长

败者树只保存“败者”，冠军在树顶；某叶更新后只需沿到根的一条路径重比，代价 O(log k)。因此增大 k 会**同时**减少归并趟数与每趟的比较代价——但 k 增大会挤占内存中的缓冲区，需折中。

8.6.4 最佳归并树

若各初始归并段**长度不等**，两两归并（k = 2）时应让**短的段早归并、长的段晚归并**，使总读写量最小——这就是**哈夫曼树**的应用：以段长为权构造（严格）哈夫曼树，归并顺序即树的合并顺序。

> **补虚段**
> m 叉哈夫曼树要求 (m − 1) | (n − 1)，不满足时需补长度为 0 的虚段。

典型例题

例 1 1000 个初始归并段，用 10 路归并需要几趟？用 4 路呢？

解 $\lceil \log_{10} 1000 \rceil = \mathbf{3}$ 趟；$\lceil \log_{4} 1000 \rceil = \lceil 4.98 \rceil = \mathbf{5}$ 趟。

例 2 为什么外排序不直接用快排一次排完？

解 数据总量超过内存，无法一次装入；即使能装下，快排的随机读写在磁盘上代价极高（每次比较可能触发一次寻道）。外排序的核心是**把随机访问变成顺序访问**：分块读入、顺序写出、多路顺序归并。

例 3 四个初始归并段长度为 (9, 5, 3, 2)，用 2 路归并给出总记录读写量最小的归并顺序。

解 构造哈夫曼树：合并 2 + 3 = 5；再 5 + 5 = 10；再 10 + 9 = 19。带权路径长度（各趟输出量之和）= 5 + 10 + 19 = **34**，即读写 34 个记录量。若按 (9+5) + (3+2) 再合并，则 14 + 5 + 19 = 38 更大。故应**先并最短的两段**。

易错点与考点提示

- 外排序的性能指标是**外存读写次数**，不是比较次数。
- 归并趟数公式是 $\lceil \log_k m \rceil$，m 是**初始段数**、k 是**归并路数**，别把 n 代进去。
- 置换-选择的平均段长是 **2w**（工作区容量的一倍），不是 w。
- 增大 k 会减少趟数，但每个缓冲区都要占内存，k 过大反而因缓冲区变小而增加 I/O。
- 败者树每叶更新只需 $\lceil \log_2 k \rceil$ 次比较，不是 $k-1$ 次。
- “最佳归并树”只在各段长度不等时才有意义；等长时任意归并顺序相同。

习题

1. 有 12 个初始归并段，要求 3 趟完成归并，至少需要几路归并？
2. 5 个归并段长度为 (4, 6, 2, 8, 10)，用 2 路归并求最小总读写量与归并顺序。
3. 说明为什么“增加缓冲区块数”能减少外存启动次数。



图 8-10 ★ 阶段一完成（置换-选择）：24 个记录 → 3 个归并段，平均段长 8.0（顺序分组是 6.0、4 段）。

8.7 排序方法的综合比较

本节配套动画：第 60 项《八大排序总览与对比》 · 教材 8.8 排序综合比较（同一数据 × 8 算法 × 代价对照）

> **目的**
> 把前六节的算法放进同一张表里比较，回答工程与考试共同的问题：**给定场景该用哪一个**。

8.7.1 总表

方法	平均时间	最坏时间	最好时间	空间	稳定性
直接插入	$O(n^2)$	$O(n^2)$	**O(n)**	O(1)	稳定
折半插入	$O(n^2)$（移动未减）	$O(n^2)$	O(n log n)? 比较少、移动仍多	O(1)	稳定
希尔排序	依赖增量，约 O(n^1.3~1.5)	$O(n^2)$	—	O(1)	**不稳定**
冒泡排序	$O(n^2)$	$O(n^2)$	**O(n)**（flag）	O(1)	稳定
快速排序	**O(n log n)**	**$O(n^2)$**（有序输入）	O(n log n)	栈 O(log n)~O(n)	不稳定
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$（比较固定）	O(1)	不稳定
堆排序	O(n log n)	**O(n log n)**	O(n log n)	**O(1)**	不稳定
2 路归并	O(n log n)	**O(n log n)**	O(n log n)	**O(n)**	稳定
基数排序	**O(d(n + r))**	O(d(n + r))	同左	O(n + r)	稳定
计数排序	O(n + k)	O(n + k)	O(n + k)	O(n + k)	稳定

8.7.2 同一组数据上的实测

对教材序列 **49, 38, 65, 97, 76, 13, 27, 49**（n = 8），各算法实际付出的代价（口径：比较含“哨兵挡住”的那一次判定；移动含 a[i]→a[0] 的暂存与最后落位）：

方法	比较次数	移动 / 交换 / 赋值
直接插入	27	25
希尔排序	35	27
冒泡排序	27	交换 15（共 6 趟）
快速排序	22	赋值 13
简单选择	**28**（固定 n(n−1)/2）	交换 7
堆排序	27	赋值 30
2 路归并	**16**	写入 24
基数排序	**0**	分配 / 收集 48

口径提醒

同一算法在不同统计口径下数字会不同，差别只在两处：**比较**是否把“让扫描停下的那一次判定”（含哨兵挡住自己）算进去；**移动**是否把 `a[i] → a[0]` 的取出算作一次搬动。上表按“两种都算”逐条计数，与课件里每一帧点亮的操作一一对得上。若换成“只数通过判定、且不数取出（但每趟仍算一次落位，哪怕原地没动）”，直接插入是 22 次比较 / 22 次移动，希尔是 26 / 26。做习题时应先写明自己的口径再算数——两个数都对，**不写口径的数才错**。

>

说明

顺带一个真实教训：希尔排序那一页曾长期写“比较 26 次”，而它自己的口径说明要求把停下的那次算进去——漏数被当成口径分歧挂了一年，直到总览表与单页对账才抓出来（35 才是本口径下的数）。

从这张表能读出三件事：**归并的比较最少**（16 次），**基数一次都不比**（靠位分配），**简单选择的比较与输入无关**（28 = 8×7/2，恒成立）。

8.7.3 选择策略

场景	首选	理由
n 很小（≤ 20）	直接插入	常数小、代码简单
序列**基本有序**	直接插入 / 冒泡	最好情形 O(n)
一般内存排序、追求平均最快	**快速排序**	O(n log n)、常数小、缓存友好
要求**最坏也有保证**且省空间	**堆排序**	最坏 O(n log n) + O(1) 空间
要求**稳定**且要 O(n log n)	**归并排序**	稳定；链表版不需辅助数组
关键字是**位数固定的整数** / 多关键字	**基数排序**	接近线性，且不比较
值域很小（如成绩 0~100）	**计数排序**	O(n + k)，零比较
数据超过内存	**外部排序**（多路归并 + 置换-选择）	目标是减少 I/O

一句话经验法则

基本有序 → 插入 / 冒泡；要稳定又要 O(n log n) → 归并；空间苛刻又要平均快 → 快排；最坏也要 O(n log n) 且 O(1) 空间 → 堆排；位数固定的整数或多关键字 → 基数。

典型例题

例 1 某要求：“必须稳定、必须保证 O(n log n)、内存充足”。选哪种？

解 **归并排序**。堆排与快排都不稳定；快排最坏 $O(n^2)$；只有归并同时满足稳定与最坏 O(n log n)，代价是 O(n) 辅助空间（题设内存充足）。

例 2 对 10^6 个**近乎有序**的记录排序，选什么最好？

解 **直接插入**（或带 flag 的冒泡）——近乎有序时比较次数接近 n，实际最快；快排此时反而可能退化。若担心“近乎”程度不够，可用**TimSort 思想**（按自然有序段归并）。

例 3 按“年份 + 月份 + 日期”给日期排序，如何用基数排序一次完成？

解 多关键字 LSD：**从最低位优先（次要关键字）开始**——先按日期稳定排、再按月份稳定排、最后按年份稳定排。每一趟必须用**稳定**方法（如计数排序），否则前面的成果会被打乱。

易错点与考点提示

- “O(n log n) 且稳定”只有归并；“O(n log n) 且 O(1) 空间”只有堆排——两者不可兼得，这是常考的取舍题。
- 快排的**平均** O(n log n) 依赖划分的均衡性，取端点基准 + 有序输入是最坏情形。
- 简单选择比较次数固定，但交换次数少，适合“记录很大、移动昂贵”。
- 基数排序的稳定性是**正确性前提**，不是附加优点。
- 排序方法的“时间复杂度”若不注明，指**平均**；说“最坏”必须点明。
- 内部排序方法中，只有**不归并、不递归**的那几类才是 O(1) 空间：插入、选择、冒泡、希尔、堆排。

习题

1. 为下列需求各选一种方法并说明理由：(a) 10^8 个 32 位整数，内存 512 MB；(b) 学生成绩表，要求总分相同者按学号次序不变；(c) 只需知道前 10 名的名次。
2. 证明：任何基于比较的排序，最坏情况下界是 Ω(n log n)。
3. 说明为什么“快速排序最坏 $O(n^2)$”在实际中仍被广泛采用。

八大排序 · 同一数据 · 代价对照表

算法	比较	移动/交换	稳定	最好	最坏	空间
直接插入	27 次	25 次	✓是	O(n)	$O(n^2)$	O(1)
希尔排序	35 次	27 次	✗否	≈O(n)	$O(n^2)$	O(1)
冒泡排序	27 次	15 次	✓是	O(n)	$O(n^2)$	O(1)
快速排序	22 次	13 次	✗否	O(n log n)	$O(n^2)$	O(log n)栈
直接选择	28 次	7 次	✗否	$O(n^2)$	$O(n^2)$	O(1)
堆排序	27 次	30 次	✗否	O(n log n)	O(n log n)	O(1)
归并排序	16 次	24 次	✓是	O(n log n)	O(n log n)	O(n)
基数排序	0 次	48 次	✓是	O(d(n+r))	O(d(n+r))	O(n+r)

全部结果一致：13 27 38 49 49 65 76 97

选择依据：看初始状态（有序?）、稳定性要求、空间限制

图 8-11 ★ 汇总：8 种算法结果一致（见右侧表格第一列），代价大不相同。

附录 A 习题答案

1.1 习题答案

1. 线性结构要求每个元素至多一个前驱、至多一个后继。树形中一个结点可有多个孩子（后继上限突破 1），图状中前驱与后继都不限，故均非线性。
2. 不能。边数只是数量特征，逻辑结构看的是“几对几”的对应关系：线性是一对一，树是一对多。边数相同而关系不同，正是“边数不能用来分类”的反例。
3. 逻辑结构 ↔ S（关系），存储结构是 D 与 S 在计算机中的映像，运算 ↔ **P**（基本操作集合）。

1.2 习题答案

1. 循环条件是 $i^2 < n$，故 i 最多取到 $[\sqrt{n}]$，$T(n) = \mathbf{O(\sqrt{n})}$。
2. 共 $\log_2 n$ 层，每层合并代价合计 n，故 $T(n) = n \cdot \log_2 n = \mathbf{O(n \log n)}$（n = 2^k 时严格成立）。
3. 步数满足 $f(n) = 2f(n-1) + 1$、$f(1) = 1$，解得 $f(n) = 2^n - 1$ → 时间 $O(2^n)$；而任一时刻递归栈里最多 n 层记录（先一路压到 hanoi(1)，再逐层返回），故空间 **O(n)**。

2.1 习题答案

1. 表尾之后插入移动 **0** 个（只需 length++ 再赋值）；表头插入移动 **n** 个。
2. 删除第 2 个（12）要把第 3~7 个前移，共移动 **6** 个：结果 (25, 47, 89, 36, 14, 20)，表长 6。
3. 元素**等长**且存放在**起始地址已知的一段连续单元**里——这样才能用 $addr(a_i) = addr(a_1) + (i-1) \times L$ 直接算出地址。

2.2 习题答案

1. 之后插入只需 `s->next = p->next; p->next = s;`，与表长无关；之前插入必须先找到 p 的前驱，而单链表只能从头遍历，故 O(n)。
2. 需从头结点起后移 i − 1 次找到前驱，平均 n/2 次，量级 **O(n)**。
3. `s->prior = p->prior; s->next = p; p->prior->next = s; p->prior = s;` ——前两条把 s 挂好，后两条才改 p 侧的旧链。

2.3 习题答案

1. 1 vs 2 → 取 1；3 vs 2 → 取 2；3 vs 4 → 取 3；5 vs 4 → 取 4；5 vs 6 → 取 5；LB 剩 6, 8, 10 整段接入。共比较 **5** 次（= m + n − 1 的下界情形之一，此处 5 = 3 + 2）。
2. $A = 3x^4 + 2x^2$、$B = -2x^2 + 5x^6$，指数 2 的项系数 2 + (−2) = 0 → **丢弃该项**；结果 $= 3x^4 + 5x^6$。
3. 两个输入链表都应按指数**降序**排列（归并规则改为“取指数大者”），或在归并后整体逆置。

2.4 习题答案

1. 初始 `p = L->next; L->next = NULL;` ① `q = p->next; p->next = L->next; L->next = p; p = q;`（摘下 a）② 同样三步摘下 b → 表变 b → a；③ 摘下 c → c → b → a；最后 p = NULL，循环结束。

2. 长表（9）先走 9 − 6 = **3** 步，然后两表各走至多 6 步同步比较；总步数不超过 6 + 9 = 15。
3. 快慢指针：slow 每次走 1 步、fast 每次走 2 步。若 fast 或其 next 到达 NULL 则无环；若两者相遇则有环。找环入口：相遇后另置一指针从**头结点**出发，与 slow 同步每次一步，再次相遇处即入口（可由“头到入口距离 = 相遇点再绕一圈的整数倍”推得）。

3.1 习题答案

1. push a, pop a；push b, push c, pop c；pop b；push d, pop d。
2. push 前 4 个成功，第 5 次触发**上溢**（判满 top − base == 4 成立，操作被拒）；随后 pop 3 次，栈内剩 **1** 个元素。
3. 三次出队后队列空，`front == rear`，两者都指向**头结点**（最后一次出队时因 p == rear 而执行了 rear = front）。

3.2 习题答案

1. 队长 = (3 − 6 + 8) % 8 = **5**；容量上限 M − 1 = 7，故还能入队 **2** 个。
2. 初始 f = r = 0，队长 0 → 入：f=0, r=1, 长 1 → 入：f=0, r=2, 长 2 → 出：f=1, r=2, 长 1 → 入：f=1, r=3, 长 2 → 入：f=1, r=0, 长 3（此时 (r+1)%4 = 1 == f，队满，容量 3 = M − 1）。
3. 链队列的结点是动态申请的，队尾 rear 后面随时可接新结点，不存在“数组到界”的限制，因此没有假溢出；代价是每个结点要额外存指针。

3.3 习题答案

1. A→B、A→C、B→C，共 3 = $2^2 - 1$ 步。栈深变化：hanoi(2) → hanoi(1) 回到 hanoi(2) → 第二次 hanoi(1)，最大栈深 2。
2. T(2) = 2·1 + 1 = 3，T(3) = 2·3 + 1 = 7，T(4) = 2·7 + 1 = **15**，与 $2^4 - 1$ 相符。
3. `push(根); while(栈不空){ p = pop(); visit(p); if(p->rchild) push(p->rchild); if(p->lchild) push(p->lchild); }`——先压右再压左，保证左子树先被访问，与递归的先序一致。

3.4 习题答案

1. 读到 `]` 时栈顶是 `(`，类型不匹配 → 属于“栈顶不匹配”（括号交叉）情形，立即判失败。
2. `a b c + * d e / -`。
3. 依次：3 − 1 = 2；2 × 2 = 4；6 / 4 = 1.5；1.5 + 5 = **6.5**。弹栈计算共 4 次。

4.1 习题答案

1. j=1 → 1；j=2（前 1 个 “a”，最长相等前后缀 0）→ 1；j=3（“aa”，1）→ 2；j=4（“aab”，0）→ 1；j=5（“aaba”，1）→ 2；j=6（“aabaa”，前缀 aa = 后缀 aa，2）→ 3；j=7（“aabaaa”，前缀 aa = 后缀 aa，2）→ 3。故 next = **[1, 1, 2, 1, 2, 3, 3]**。
2. 模式 `abaab` 的 next = [1,1,2,3,1]。第一次失配发生在模式第 4 位（`a` 对 `b` 失配），j 滑到 next[4] = **3**，即把模式的第 3 位 `a` 对齐到主串的失配字符继续比较，主串指针不动。
3. 每一趟起点对最多比较 m 个字符后失配，而起点最多右移 n 次，故总比较次数上界为 (n − m + 1)·m = O(n·m)。

4.2 习题答案

1. $n(n+1)/2 = 6 \times 7 / 2 =$ **21** 个；完整需 36 个，节省 $(36-21)/36 \approx$ **41.7%**。
2. $k = 5 \times 4 / 2 + (3-1) = 10 + 2 =$ **12**。
3. $1000 \times 3 \times 4 =$ **12000 字节**（另加行列数与总数三个域）；完整存储 $10^6 \times 4 = 4$ MB，节省约 **99.7%**。

4.3 习题答案

1. `((a,b),(c,d))`：长度 **2**、深度 **2**；`(a,(b,(c,(d))))`：长度 **2**、深度 **4**；`()`：长度 **0**、深度 **1**。
2. 不有限。D 的第 3 个元素是 D 自身，展开深度时得到 Depth(D) = 1 + max(…, Depth(D))，方程无解——它是递归表，深度不存在。
3. 头结点 → [tag=0 | a | ↓] → [tag=1 | →子表 | ∧]，子表为 [tag=0 | b | ↓] → [tag=0 | c | ∧]。横向用 link 串同层元素，纵向用 sublist 钻入子表。

5.1 习题答案

1. 需要扫描**全部 n 个** parent 域：叶子是“没有任何结点的 parent 指向它”的结点，只能靠统计每个下标被引用的次数判断，故必须看完整张表。
2. A: firstchild = B, nextsibling = ∧；B: fc = E, ns = C；C: fc = ∧, ns = D；D: fc = H, ns = ∧；E: fc = ∧, ns = F；F: fc = ∧, ns = G；G: fc = ∧, ns = ∧；H: fc = ∧, ns = ∧。（注意 B 的三个孩子 E、F、G 串成一条兄弟链，B、C、D 是 A 的孩子，也串成兄弟链。）
3. 转换时各棵树的根被“下一个兄弟”关系串起来，因此第一棵树的根成为二叉树的根，其余根依次挂在它的右链上——右链结点数 + 1（含根本身）即森林中树的棵数。

5.2 习题答案

1. 最小深度 $\lfloor \log_2 257 \rfloor + 1 = 8 + 1 =$ **9**（完全二叉树时最矮）；最大深度 **257**（退化成单支链）。
2. $n = 1000$ 为偶数，故 $n_1 = 1$（有一个结点只有左孩子），$n_0 = n_2 + 1$ 且 $n_0 + n_1 + n_2 = 1000 \rightarrow 2n_2 + 1 + 1 = 1000 \rightarrow n_2 = 499$，叶子 $n_0 =$ **500**。
3. 双亲 = `⌊(i − 1)/2⌋`（$i > 0$）；左孩子 = `2i + 1`；右孩子 = `2i + 2`（越界即不存在）。

5.3 习题答案

1. 根 A；中序里 A 左边 DBE 为左子树、右边 FC 为右子树；先序中紧随 A 的 B 是左子树根，中序里 B 左 D、右 E → B 有左 D 右 E；先序余下 C 是右子树根，中序里 C 左 F → C 有左 F。后序 = **D E B F C A**。
2. 完全二叉树层序 1~7 即结点编号，中序（左根右）为 **4 2 5 1 6 3 7**。
3. 反例：先序 AB、后序 BA 对应两棵不同的树——A 有左孩子 B，或 A 有右孩子 B。只有一个孩子时左右无法区分。

5.4 习题答案

1. 二叉树：A 左孩子 B；B 右链 B → F → H；B 左孩子 C，C 右链 C → D → E；F 左孩子 G；H、G 无孩子。先序 = **A B C D E F G H**；中序 = **E D C B G F H A**。

2. **3 个**（三棵树的根依次挂在右链上）。
3. 因为中序“左根右”中的“左”是第一个孩子及其子孙，访问它等于先访问完该结点的最左子树；而“右”是兄弟链——一棵树的后根要求“孩子先于自己”，与中序在“孩子 → 自己 → 兄弟”上的次序恰好一致。

5.5 习题答案

1. 合并 2+4 = 6 → 森林变 {5, 6, 7, 8}；再 5+6 = 11 → {7, 8, 11}；再 7+8 = 15 → {11, 15}；再 11+15 = 26。非叶结点权值之和 = 6 + 11 + 15 + 26 = **58**，即 WPL = 58。叶子深度：5、7、8 在第 2 层，2、4 在第 3 层，校验 5×2 + 7×2 + 8×2 + 2×3 + 4×3 = 10 + 14 + 16 + 6 + 12 = 58 √。编码长度分布：3 个字符 2 位、2 个字符 3 位。
2. 叶子 = (15 + 1)/2 = **8** 个，合并 = 15 − 8 = **7** 次（= n − 1）。
3. 构造过程保证“频率高（权大）者离根近”，且每次合并都是当前最小的两者——任何最优前缀码都必须把最小的两个符号放在最深且同层的位置，故可经交换变换成哈夫曼码而不增加码长。要求：各符号出现频率**已知且固定**（静态），若频率随时间变化则应改用自适应编码。

5.6 习题答案

1. n = 7，起点 ⌊7/2⌋ = 3，自 3 号往前筛。① 筛 3 号（65，孩子 13、27）：与较小的孩子 13 交换 → 49 38 **13** 97 76 **65** 27。② 筛 2 号（38，孩子 97、76）：已满足，不动。③ 筛 1 号（49，孩子 38、13）：与较小的孩子 13 交换 → **13** 38 **49** 97 76 65 27；49 沉到 3 号位后要继续筛，它的孩子是 65、27 → 再与 27 交换 → 13 38 **27** 97 76 65 **49**。最终 **13 38 27 97 76 65 49**，逐格验：13 ≤ 38,27；38 ≤ 97,76；27 ≤ 65,49，全部成立。
2. 高度为 h 的堆中，需下沉 i 层的结点至多有 ⌈n/2^(i+1)⌉ 个，故总代价 Σ i·⌈n/2^(i+1)⌉ = n·Σ i/2^(i+1) = n·O(1) = **O(n)**（级数 Σ i/2^i 收敛于 2）。
3. 维护容量 100 的**小顶堆**：新数与堆顶比，更大则替换堆顶并下沉，否则丢弃。每来一个数 O(log 100) = O(1)，n 个数共 **O(n)**，空间 O(100)。堆顶始终是这 100 个里最小的，即“第 100 大”的门槛。

5.7 习题答案

1. 找到根 r 后取 **−parent[r]** 即集合大小（因为根存的是元素数的相反数）。若约定 parent[i] = i 表示根，则需另设 size[] 数组。
2. 不优化时按输入顺序挂，树高 = **7**（一条链）；按大小合并后树高 ≤ ⌊log₂8⌋ = **3**。
3. 每条边都要做一次 O(n + e) 的遍历判连通，总复杂度升到 **O(n•e)** 级别，失去 Kruskal 的优势（应保 O(e log e)）。

6.1 习题答案

1. 度之和 = 2 × 15 = 30；已给出的 6 个顶点贡献 2 × (2 + 4 + 6) = 24，余下 30 − 24 = 6 由度为 3 的顶点分担，故有 6 / 3 = 2 个，共 **8 个顶点**。（最大度 6 ≤ n − 1 = 7，简单图条件下成立。）
2. v1 的邻接点是 v0、v2、v3（度 3）；v2 的邻接点是 v0、v1、v3（度 3）。边 (1,2) 在 v1 处记一度、在 v2 处又记一度，所以“度之和 = 2 | E | ”里它被算了两次；但**边数**只算一条——这正是握手定理中“2 倍”的来源。
3. n 个顶点的简单图中，任一顶点的度只能取 0, 1, …, n − 1 这 n 个值；但度 0（与所有点不连）与度 n − 1（与所有点相连）不可能同时存在，故实际最多只有 n − 1 种取值。n 个顶点分进 n − 1 个抽屉，必

有两点同度（抽屉原理）。

4. Σ 入度 = 0 + 1 + 2 + 2 + 1 = 6，Σ 出度 = 2 + 2 + 1 + 0 + 1 = 6，两者相等，故有 **6 条弧**。数据本身不排除自环；但若有自环，该顶点会同时贡献 1 个入度和 1 个出度，仅由这两个序列无法判定有无自环——需要看邻接矩阵的对角线。
5. 无向连通图至少 **n − 1** 条边（此时是一棵树）；强连通有向图至少 **n** 条弧（n 个顶点排成一个有向环）。

6.2 习题答案

1. 顶点表 4 行，每行含 firstout / firstin。v0：出链 <0,1> → <0,2>，入链 <2,0>；v1：出链 <1,2> → <1,3>，入链 <0,1>；v2：出链 <2,3> → <2,0>，入链 <1,2> → <0,2>；v3：出链为空，入链 <2,3> → <1,3>。共 6 个弧结点，每个结点被两条链共享。
2. 行是出度、列是入度：OD = 4，ID = 2，TD = 6。
3. 边结点 e 个（每条边只存一次），链指针 2e 个（每边 ilink、jlink 各一）。
4. 邻接矩阵直接取 arc[i][j] 一个元素，O(1)；邻接表必须沿 v 的边链逐个比较头点编号，最坏 O(TD(v))，稠密图上接近 O(n)。
5. **邻接表**（无向图）：空间 O(n + 2e) 且判边需扫链。若为邻接多重表，空间应是 O(n + e) 个边结点，与题设不符。

6.3 习题答案

1. 从 v1：DFS = 1 → 2 → 5 → 3 → 4 → 6，生成树边 (1,2)、(2,5)、(5,3)、(5,4)、(4,6)；BFS = 1 → 2 → 3 → 5 → 4 → 6，生成树边 (1,2)、(1,3)、(2,5)、(5,4)、(4,6)。注意 v5 在 BFS 中是由 v2 首次到达的，所以 (3,5) 不是树边——两棵树的边集不同，但都是 5 = n − 1 条边。
2. 删去 (4,6) 后 v6 成为孤立点，图有 **2 个连通分量**（{v1,v2,v3,v4,v5} 与 {v6}），故从 v2 出发访问不完，需再启动 1 次，共 **2 次**。
3. 邻接表下每个顶点入队出队一次（O(n)），每条边在其两端的链上各被检查一次（O(e)），合计 O(n + e)；邻接矩阵下检查一个顶点的全部邻接点须扫完一行 n 个元素，n 个顶点共 $O(n^2)$，与边数无关。
4. DFS 序列 = 1 → 2 → 3 → 4（v1 的邻接点 v2、v4；先入 v2 → v3，v3 的邻接点 v1 已访问，回溯后访问 v4）。强连通分量 **2 个**：{v1, v2, v3} 构成环，v4 只进不出、自成一个。
5. 错。当图本身就是一棵树时，两者得到的生成树都等于原图，完全相同；一般地，只要每个顶点在两种算法下都只有一条“首次到达”的边，两棵树也可能一致。正确的说法是：两棵树**不一定**相同。

6.4 习题答案

1. 初值（以 v1 的出边建表）：lowcost = {v2: 6, v3: 1, v4: 5, v5: ∞, v6: ∞}，括号内前驱均为 v1。第 1 轮取 v3(1)：v2 经 v3 为 5 < 6 → 改 5；v4 经 v3 为 5 = 5 → **不改**；v5 → 6；v6 → 4，候选变为 {v2: 5, v4: 5, v5: 6, v6: 4}。第 2 轮取 v6(4)：v4 经 v6 为 2 < 5 → 改 2；v5 经 v6 为 6 = 6 → 不改。第 3 轮取 v4(2)：无改进。第 4 轮取 v2(5)：v5 经 v2 为 3 < 6 → 改 3。第 5 轮取 v5(3)。总权 1 + 4 + 2 + 5 + 3 = **15**。
2. 两条权 1 的边 v1—v2、v3—v4 必然先被选入（各自连接不同分量），此时顶点被分成两个分量 {v1, v2} 与 {v3, v4}。接下来考察三条权 2 的边：**第一条**连接这两个分量，选入后凑够 n − 1 = 3 条边，生成

树已成；此时图已连通，**其余两条**无论连接哪两个顶点，其两端必在同一分量内，选入即成环，按环路性质（它是所在回路中不劣于其他边的选择）舍弃。故第三条边舍弃的理由是“两端已连通”，与它的权值大小无关。

3. 能同时出现——只要这两条边不构成回路（例如它们没有公共顶点，或虽有公共顶点但加入后仍无环），Kruskal 会把两条都选入。不能同时出现的情形是：这两条边与已选边构成一个回路，此时二者只能取其一。因此“并列最小边”既可能全选也可能只选一部分，取决于它们是否成环。
4. Prim 第 1 轮把 v1、v3 连成一条边，已是含 2 个顶点的树，此后每轮**加一个顶点、加一条边**，不可能成环，故中间结果始终是树；Kruskal 选完 v1—v2 与 v3—v4 两条边后得到的是**两片互不连通的树**（森林），直到选入 v2—v3 才合成一棵树。
5. 每条边权 1 时，任何生成树的代价都是 n − 1，因此 **G 的每一棵生成树都是 MST**：MST 代价唯一等于 n − 1，而 MST 的个数等于 G 的生成树个数（可按矩阵树定理计数）。

6.5 习题答案

1. 源点 v2 时：D = [∞, ∞, 0, 50, ∞, 60]，v3 为 v2 → v3（50），v5 为 v2 → v3 → v5（60）；v0、v1、v4 均不可达（v2 无出边指向它们所在的部分）。
2. 删去 v4→v3 后 v3 只能经 v2 到达，D[v3] = 60、路径 v0 → v2 → v3；D[v5] = 70、路径 v0 → v2 → v3 → v5。
3. 反例：三个顶点，弧为 v0→v1 = 2、v0→v2 = 3、v2→v1 = −5。Dijkstra 第 1 轮在 U 中取到最小的 v1（D = 2）并定死；第 2 轮定死 v2（D = 3）时才发现 3 + (−5) = −2 < 2，但 v1 已并入 S、不再复查，于是输出 D[v1] = 2，真实值为 −2。此图中 v1 无出边，故不含任何回路，最短路径确实存在——错误纯粹来自“先定死就不再改”在负权下不成立。
4. 两者同为 $O(n^3)$ 量级，运行次数上无本质差别；还应考虑编程复杂度（Floyd 三重循环即可）、是否需要 Path 回溯，以及边权是否非负——若含负权，Dijkstra 方案根本不可行。
5. 不能。D[i][i] = 0 只说明图中**无负环**；图中完全可能有正权环或零权环，只是绕回自身不会使路径更短，不影响最短路径的定义。

6.6 习题答案

1. 第 1 步降序进栈得栈 C2, C1（顶为 C1）→ 弹 C1 输出，删 <C1,C3>、<C1,C4>，C3 入度 1→0 进栈、C4 入度 2→1；弹 C3 输出，删 <C3,C6>，C6 入度 2→1；弹 C2 输出，删 <C2,C4>、<C2,C5>，C4 入度 1→0 进栈、C5 入度 1→0 进栈；弹 C5 输出，删 <C5,C6>，C6 入度 1→0 进栈；弹 C6 输出；弹 C4 输出。序列为 **C1 → C3 → C2 → C5 → C6 → C4**（注意 C4 最后：它入度归零最晚）。
2. **11 个是错的直觉，正确答案是 7 个**。C4 与 C5 在约束上并不对称：C5 还有一条出弧 <C5, C6>，而 C4 没有后继，所以“一半满足 C4 先、一半满足 C5 先”的对称性不成立。把 6 个顶点的全部 6! = 720 种排列逐一代入约束（含新加的 C4 → C5）穷举，合法序列为 **7 个**。这类计数题不要靠对称性推算，要么枚举，要么按“最后一个位置的可选顶点”递推。
3. a3 = <v1, v3> 由 2 改为 5：ve(v3) = max{3 + 5 = 8, 2 + 4 = 6} = **8**；ve(v4) = 6 不变；ve(v5) = max{5, 8 + 2 = 10, 7} = **10**，工期变为 10。逆向：vl(v5) = 10，vl(v4) = 9，vl(v3) = 8，vl(v2) = min{8 − 4 = 4, 10 − 3 = 7} = 4，vl(v1) = min{8 − 5 = 3, 9 − 3 = 6} = 3，vl(v0) = min{3 − 3 = 0, 4 −

2 = 2} = 0。关键活动：a1（e = 0，l = 0）、a3（e = 3，l = 3）、a7（e = 8，l = 8）——关键路径变为 **v0 → v1 → v3 → v5**，长度 3 + 5 + 2 = 10。

4. ve(j) 依赖 j 的**所有前驱**都已求出，故必须沿拓扑序推进；vl(k) 依赖 k 的**所有后继**都已求出，故必须沿逆拓扑序回推。次序颠倒会用到尚未算出的值。
5. 工期仍是 **10 天**。第一条关键路径缩短为 8，但第二条仍是 10，工程完成时间取所有路径的最大值。必须同时压缩两条关键路径上的活动（或压缩它们的公共活动）才能缩短工期。

7.1 习题答案

1. [1..9] mid=5 → 23 < 31 → low=6；[6..9] mid=7 → 31 = 31 命中。共 **2** 次比较。
2. 判定树：根 5（第 1 层）；2、7（第 2 层）；1、3、6、8（第 3 层）；4、9（第 4 层）。失败区间共 n + 1 = 10 个外部结点：结点 1 下 2 个、结点 3 左 1 个、结点 6 下 2 个、结点 8 左 1 个（都在第 4 层），结点 4 下 2 个、结点 9 下 2 个（都在第 5 层）。失败时比较次数 = 外部结点层数 − 1，故 ASL(失败) = (6 × 3 + 4 × 4) / 10 = **3.4**。
3. 取每块 s ≈ [√27] = **6** 个（共 [27/6] = 5 块），ASL ≈ (5+1)/2 + (6+1)/2 = 3 + 3.5 = 6.5 最小。一般结论：块数 ≈ 每块元素数 ≈ √n。

7.2 习题答案

1. 树形：50 为根，左 30(20, 40)、右 70(60, 80)；中序 = **20 30 40 50 60 70 80**（递增，可作自检），树高 **3**。
2. 50 有左右子树，用**中序前驱 40**（左子树最大）顶替 50，再删去原 40 结点；结果根为 40，左 30(20)、右 70(60, 80)。用后继 60 亦可，形状不同但都合法。
3. 插 10、15 后 10 的 bf = −1；插 5 后 10 的 bf = 1（仍平衡）；插 8 后 5 的 bf = −1、10 的 bf = 2，失衡位置在**左孩子的右子树 → LR**。先对 5 左旋（8 上升、5 成为 8 的左孩子），再对 10 右旋。最终根为 8，左孩子 5、右孩子 10，三者的平衡因子都在 ±1 内。

7.3 习题答案

1. 3 阶 B 树每个结点至多 2 个关键字、至多 3 棵子树，故第 1 层 1 个结点、第 2 层至多 3 个、第 3 层至多 9 个，共至多 1 + 3 + 9 = 13 个结点、每结点 2 键 → 最多 **26** 个关键字。
2. 15, 18 → [15,18]；插 22 → 溢出，18 上移成根，叶 [15]、[22]（分裂 1 次）；插 25 → 叶 [22,25]；插 30 → [22,25,30] 溢出，25 上移进根 → 根 [18,25]（分裂 2 次）；插 35 → 叶 [30,35]；插 40 → [30,35,40] 溢出，35 上移进根 → 根 [18,25,35] 溢出 → 中位数 25 上移成新根，左 [18]、右 [35]（分裂 3、4 次）。最终：根 [25]，第二层 [18]、[35]，叶子 [15] [22] [30] [40]，共分裂 **4** 次。
3. B+ 树的数据**全部在叶子上**，任何关键字的查找都必须从根走到叶子才结束；B 树的关键字可能出现在内部结点，命中即返回，故路径长度随命中的结点层次而变。

7.4 习题答案

1. 黑高为 h 的红黑树至少含 **2^h − 1** 个结点：每条路径至少 h 个黑结点，取“全黑且满”的情形最小。h = 2 时至少 **3** 个结点（根 + 两个孩子，全黑）。红结点只会增加结点数，不会减少。
2. 插 10（根，染黑）；插 20（红，挂在 10 右，RR 外侧但无叔结点）→ 对 10 **左旋**并换色，得根 20 黑、左 10 红；再插 30（红，挂在 20 右）→ 此时 20 的右孩子 30 红、左孩子 10 红，无红红相邻，

五条性质成立。最终根 20(黑)，左 10(红)，右 30(红)。

3. 由性质 5，任一结点到其后代叶子的每条路径黑结点数相同 = b；由性质 4，红结点不能相邻，故每条路径上红结点数至多为 b（红黑交替），于是最长路径 ≤ 2b；而最短路径至少全是黑结点 = b。故最长 ≤ 2 × 最短。

7.5 习题答案

1. 18 mod 13 = 5 → 5（1 次）；31 mod 13 = 5 → 6（2）；44 mod 13 = 5 → 7（3）；10 mod 13 = 10 → 10（1）；22 mod 13 = 9 → 9（1）。合计 1+2+3+1+1 = 8 → ASL = 8/5 = **1.6**。
2. 桶 5 的链为 44 → 31 → 18（头插），比较次数分别 1、2、3；桶 9 为 22（1）；桶 10 为 10（1）。合计仍是 8 → ASL = **1.6**。本题冲突集中在同一串同义词上，两种方法结果相同；区别要等到出现“非同义词争抢地址”（如 7.5.2 中的 84）时才显现。
3. 若散列函数把全部关键字都映到同一地址，表就退化成一条链表（或一条长度为 n 的探测链），查找需 O(n)。平均 O(1) 的前提是**简单均匀散列假设**：每个关键字等可能地映到任一地址、且相互独立。

8.1 习题答案

1. 先按学号排一次（学号唯一，稳定与否影响不大），再按成绩做**稳定**排序：成绩相同的同学之间仍保持学号有序。若第二趟不稳定，成绩相同者的学号次序就被打乱了。
2. **简单选择排序**。每趟都要在无序区里选最小者，比较次数固定 Σ(n − i) = n(n − 1)/2；每趟至多一次交换，故至多 n − 1 次。
3. O(1)：直接插入、折半插入、简单选择、冒泡、希尔、堆排序；O(n)：归并排序（辅助数组）、基数排序（r 个队列 + 计数）、计数排序（O(n + k)）；介于两者之间：快速排序（递归栈平均 O(log n)、最坏 O(n)）。
4. **原写法**：插入 49b 时判 `49a > 49b` 为假，不后移，49b 落在 49a 之后 → **13, 49a, 49b**（先后未变，稳定）。**改成 `>=`**：`49a >= 49b` 为真，49a 被后移一格，49b 插到它前面 → **13, 49b, 49a**（先后反了）。两种写法的关键字序列都是 13 49 49，从结果上**完全看不出差别**。可见稳定性不取决于“排没排对”，只取决于**相等的那一次判定会不会让元素跨过对方**。

8.2 习题答案

1. 逐趟（取出的元素与比较次数，**不计**最后那次哨兵判定）：取 79 → 与 46 比 1 次，序列 46 79；取 56 → 与 79、46 比 2 次，得 46 56 79；取 38 → 与 79、56、46 比 3 次，得 38 46 56 79；取 40 → 与 79、56、46、38 比 4 次，得 38 40 46 56 79；取 84 → 与 79 比 1 次，不动。最终 **38 40 46 56 79 84**，共比较 1 + 2 + 3 + 4 + 1 = **11** 次（若把第 3、4 趟的哨兵判定也计入，则为 13 次——答题必须写明采用哪种约定）。
2. d=3：组 (46,38)、(79,40)、(56,84) → 组内插入得 **38 40 56 46 79 84**（第一组交换、第二组交换、第三组不动）。d=1：一次直接插入 → **38 40 46 56 79 84**。
3. 因为相距 d = n/2 的配对恰好是 (1, 1+d), (2, 2+d), …，每组恰好两个元素；随着 d 缩小，组内元素变多、组数变少，最后一趟 d = 1 即整体直接插入。

8.3 习题答案

1. key = 50。按 8.3.2 的填坑规则双指针相向扫描，依次把小于 50 的 20、10、40、30 填进左半、把大于 50 的 90、70、80、60 填进右半，两指针相遇于 a[5]，基准归位。结果 **20 10 40 30 50 70 80 60 90**——左侧四个数全 < 50，右侧四个数全 > 50。
2. 第 1 趟把 90 冒到第 9 位，第 2 趟把 80 冒到第 8 位。逐对模拟后第 2 趟结束序列为 **10 30 50 40 70 60 20 80 90**（注意前 7 个元素并未有序——冒泡每趟只保证末尾多一个就位）。
3. 随机数据下每次划分接近对半分，递归树高 O(log n)、总比较 O(n log n)，且常数小、缓存友好；而近乎有序时若仍取端点作基准，每次只能切出 1 个元素，退化为 $O(n^2)$，此时插入排序反而最好（O(n)）。

8.4 习题答案

1. 趟 1：比较 4 次，选出 6，与 15 交换；序列 6 27 15 40 22。趟 2：比较 3 次，选出 15，与 27 交换 → 6 15 27 40 22。趟 3：比较 2 次，选出 22，与 27 交换 → 6 15 22 40 27。趟 4：比较 1 次，选出 27，与 40 交换 → 6 15 22 27 40。合计比较 4 + 3 + 2 + 1 = **10** = n(n − 1)/2，交换 **4** 次。
2. n = 6，起点 [6/2] = 3。筛 3 号（10，孩子 2）→ 交换；筛 2 号（8，孩子 6、4）→ 与 4 交换；筛 1 号（12，孩子 4、2）→ 与 2 交换，12 沉到 3 号位后再与孩子 10 比较 → 交换。最终 **2 4 10 6 8 12**（逐格验：2 ≤ 4,10；4 ≤ 6,8；10 ≤ 12）√。
3. 堆排序无论输入如何，都要建一次 O(n) 的堆，并做 n − 1 次 O(log n) 的筛选，没有“提前发现有序”的机制；而冒泡有 flag，遇到已有序输入时一趟零交换即结束，故最好 O(n)。

8.5 习题答案

1. 个位趟：按个位分配得 Q0:{20} Q1:{51} Q3:{13} Q6:{66,76} Q8:{8} Q9:{49} → 收集 **20 51 13 66 76 8 49**；十位趟：Q0:{8} Q1:{13} Q2:{20} Q4:{49} Q5:{51} Q6:{66} Q7:{76} → 收集 **8 13 20 49 51 66 76**，共 2 趟（最大位数 2）。
2. 统计 count[1..9]（按值 0~9）：0→0，1→2，2→1，3→1，4→1，5→1，6→1，7→0，8→0，9→1；前缀和：0, 2, 3, 4, 5, 6, 7, 7, 7, 8。末项 count[9] = 8 = n √。结果 1 1 2 3 4 5 6 9。
3. 链表归并不需随机存取：Merge 时用**指针**把两表的结点依次串进结果链即可，不需要辅助数组；递归只是 O(log n) 的指针参数。因此链表归并的额外空间只有递归栈 O(log n)（自底向上迭代版可到 O(1)）。

8.6 习题答案

1. 需 [k^3] ≥ 12，即 k ≥ 12^(1/3) ≈ 2.29 → 至少 **3 路**（3 路时 $[\log_3 12] = 3$ √）。
2. 哈夫曼合并：2 + 4 = 6；6 + 6 = 12；8 + 10 = 18；12 + 18 = 30。总读写量 = 6 + 12 + 18 + 30 = **66**。归并顺序：先并 (2,4)，再与 6 并，同时把 (8,10) 并成 18，最后 12 + 18。
3. 磁盘 I/O 以“块”为单位、每次启动（寻道 + 旋转）开销固定。缓冲区越大，一次读入的连续块越多、可以预读与延迟写，把许多次小读写合并成少数大块顺序读写，从而减少**启动次数**并提高吞吐。

8.7 习题答案

1. (a) **基数排序（LSD，4 字节 8 趟或 4 趟 16 位）**或外部排序——512 MB 勉强装得下 10^8 个 32 位整数（400 MB），基数排序 O(d·(n+r)) 且不需比较，比 O(n log n) 更快；若内存吃紧则外部排序。(b) **归并排序**（稳定 + O(n log n)）。(c) **容量 10 的小顶堆**（或快速选择算法 O(n)）——全排序是浪费。
2. 决策树模型：n 个元素有 n! 种可能的排列，每种都必须落到不同的叶子上，故树至少有 n! 个叶；高度为 h 的二叉树至多 2^h 个叶，故 2^h ≥ n! ⟹ $h \geq \log_2(n!) = \mathbf{\Omega(n \log n)}$（斯特林公式）。而比较排序的最

坏比较次数 ≥ 树高。

3. 因为它的**常数因子最小**、内层循环紧凑、局部性好，平均性能优于同为 O(n log n) 的堆排与归并；最坏情形可以通过**随机选基准 / 三数取中 / 小区间切换插入排序**等手段把概率压到极低，工程实现（如各语言的 qsort / sort）都这么做。